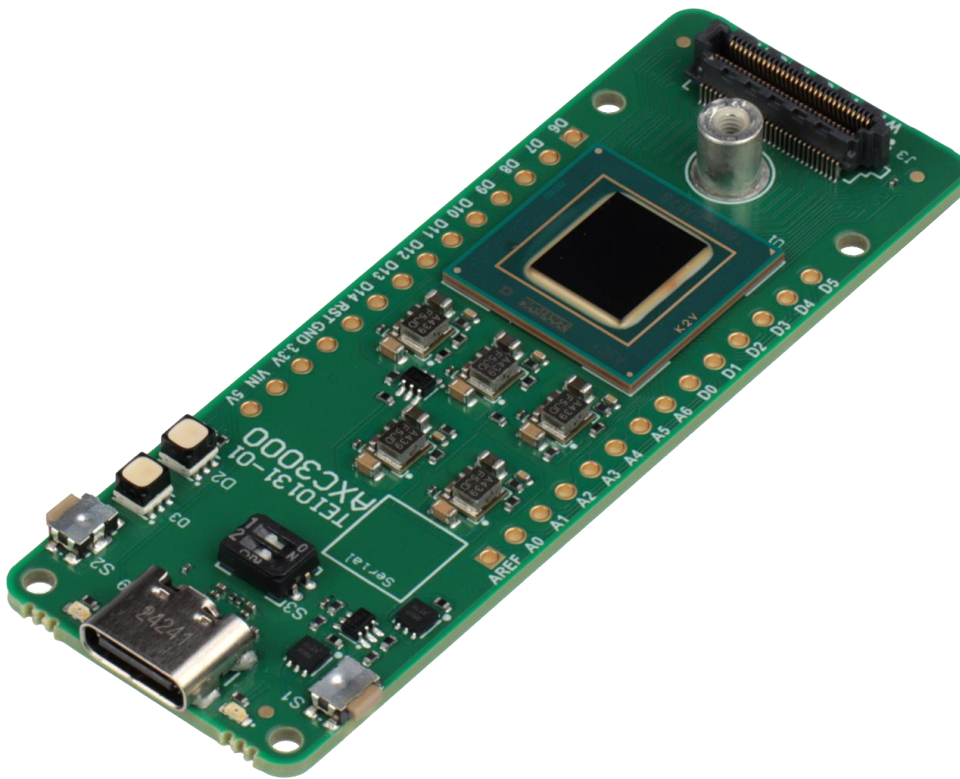




AI Solutions with Agilex[®] 3 and FPGA AI Suite



Software and hardware requirements to complete all exercises

Software Requirements: Quartus[®] Prime Pro version 25.1.1

Hardware Requirements: AXC3000 Agilex-3 Series Board

Document Control

Document Version:	Version 1.0
Document Date:	10/09/2025
Document Author(s):	Steven Kravatsky, Erika Peter, Naji Naufel, ESC Team
Document Classification:	Released
Distribution of Document Distribution:	This document is still under development. All specifications, procedures, and processes described in this document are subject to change without prior notice
Prior Version History:	0.1

1	FPGA AI SUITE AI WORKSHOP FOR ALTERA FPGA	4
2	INTRODUCTION TO FPGA AI SUITE	5
3	FPGA AI SUITE VS OTHER AI SOLUTIONS	12
4	WORKSHOP STEPS	14
5	DESIGN REQUIREMENTS	15
6	SOFTWARE AND NETWORK TOPOLOGY	16
7	BUILD, TRAIN, TEST, AND EXPORT THE MODEL	22
8	PREPARE THE FPGA PROJECT	31
9	VISUALIZATION USING THE ONNX FILE	38
10	FPGA DESIGN SPACE EXPLORATION	41
11	SOFTWARE EMULATION USING FPGA AI SUITE	60
12	GENERATE DLA IP CORE	66
13	INTEGRATE DLA IP CORE INTO FPGA	69
14	TEST THE DESIGN	75
15	VERIFY THE DESIGN REMOTELY	78
16	VERIFY THE DESIGN LOCALLY (CLOUDLABS BUILD)	86
17	VERIFY THE DESIGN LOCALLY (LOCAL BUILD)	96
18	CONGRATULATIONS ON WORKING THROUGH THIS WORKSHOP!	103
19	LEGAL DISCLAIMER	104

1 FPGA AI Suite AI Workshop for Altera FPGA

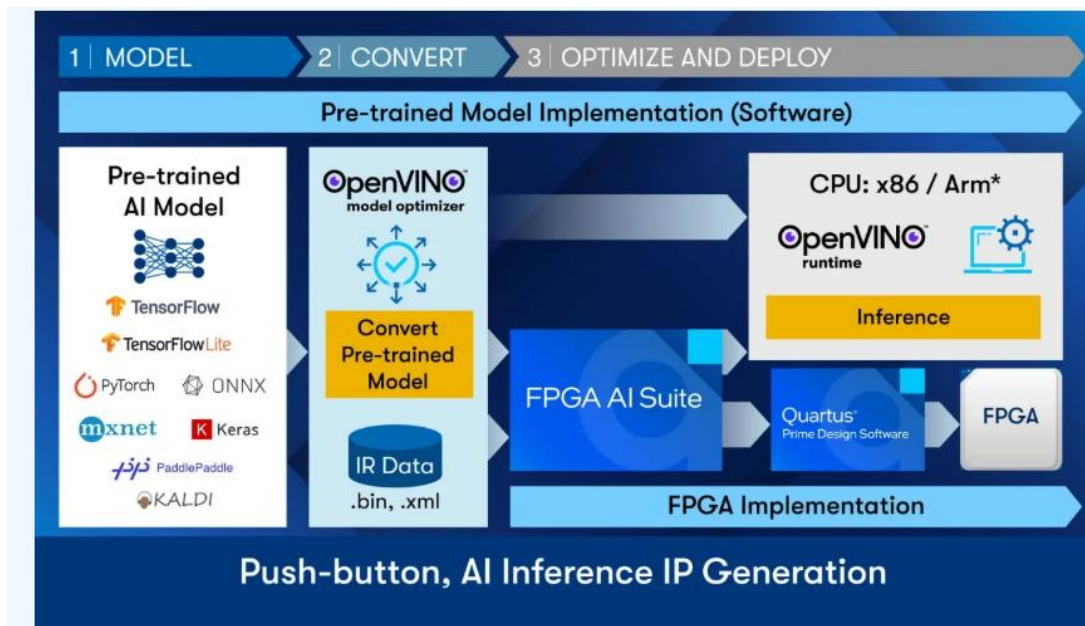
This FPGA AI Suite AI workshop is created for participants with no prior knowledge of AI and covers the following topics:

- Introduction to FPGA AI Suite: What it is and comparison to other AI implementations
- Step-by-step workshop flow for AI machine learning (ML) including:
 - Design requirements.
 - Software and topology
 - Build the model
 - Train the neural network model Visualize the model
 - FPGA Design Space Exploration
 - Generate and integrate the IP core in a reference design
 - Program and test the design on hardware

2 Introduction to FPGA AI Suite

FPGA AI Suite is a tool suite that allows users to take trained deep learning models and then deploy them for inference in Altera FPGAs. The flexible and configurable FPGA AI Suite focuses on ease of use by leveraging standard AI front ends, (such as TensorFlow, Python and others) and the Intel OpenVino hardware optimization tools to the do AI inference. The end-to-end AI workflow (as seen below) consists of 3 major sections: Model, Convert, and Optimize and Deploy.

- **Model:** Import pretrained models from traditional sources.
- **Convert:** OpenVino converts the pre-trained model to create two intermediate files (bin & xml)
- **Optimize and deploy:** FPGA implementation stage. This is the stage where FPGA AI Suite and Quartus are used to target a FPGA. Guidance is provided by the designer on required performance and/or resource utilization



Generating Inference IP is a push-button experience in FPGA AI Suite

2.1 Performance and Resource estimation

FPGA AI Suite does area and performance estimation, without needing to do a FPGA compile. The FPGA AI Suite can also generate an optimized architecture. It then creates a file that can be used by the Quartus software to program a device

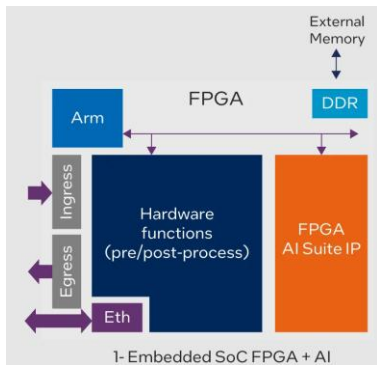
2.2 FPGA AI Suite Implementations

FPGA AI Suite

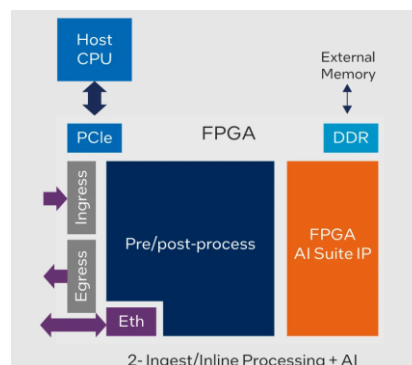
- supports different implementations including **hostless** and **hosted**, **DDRfree** and **non DDRfree**
- provides different **reference designs** for a starting point
- supports many Altera different families such as the Agilex 5, Agilex 7, etc.

The focus for this workshop is on the **Agilex 3** devices.

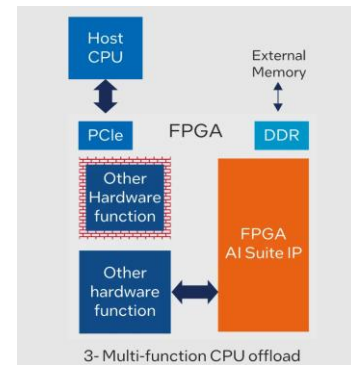
As an example, the FPGA AI Suite can support the following types –of applications:



AI Accelerator



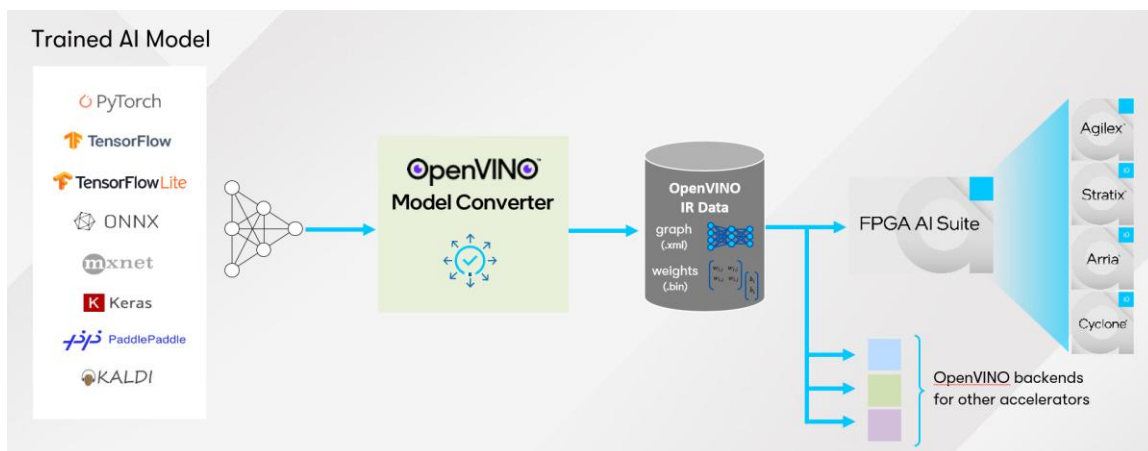
AI Accelerator + Direct Ingest & Data Streaming



AI Accelerator + Additional HW

2.3 FPGA AI Suite Tool Flow

The flow of the tool is from left to right. This includes The Trained Ai model, OpenVino Model Converter, and FPGA Suite and FPGAs.



2.3.1 Trained AI model

- This can be a proprietary or an open-source model. For this workshop, the open source LeNet-5 model written in Pytorch is used. A LeNet-5 model, which is a simple (7 layer) convolution neural network, is equivalent to a “Hello World” design.

2.3.2 OpenVino Model converter:

The converters functions include:

- Format conversion. Format conversion means you have the flexibility to convert a wide range of AI framework models i.e. PyTorch, TensorFlow, TensorFlow Lite, etc.
- Basic optimization to the AI graph topology to try to reduce computational requirements. These optimizations, which are generic optimizations, are not specific to a hardware device.
- Generation of files, which are called intermediate files. These files are .xml (graph which is the neural network) and .bin (weights). Weights are the parameters that the model modified to get more accurate results

2.3.3 FPGA AI Suite

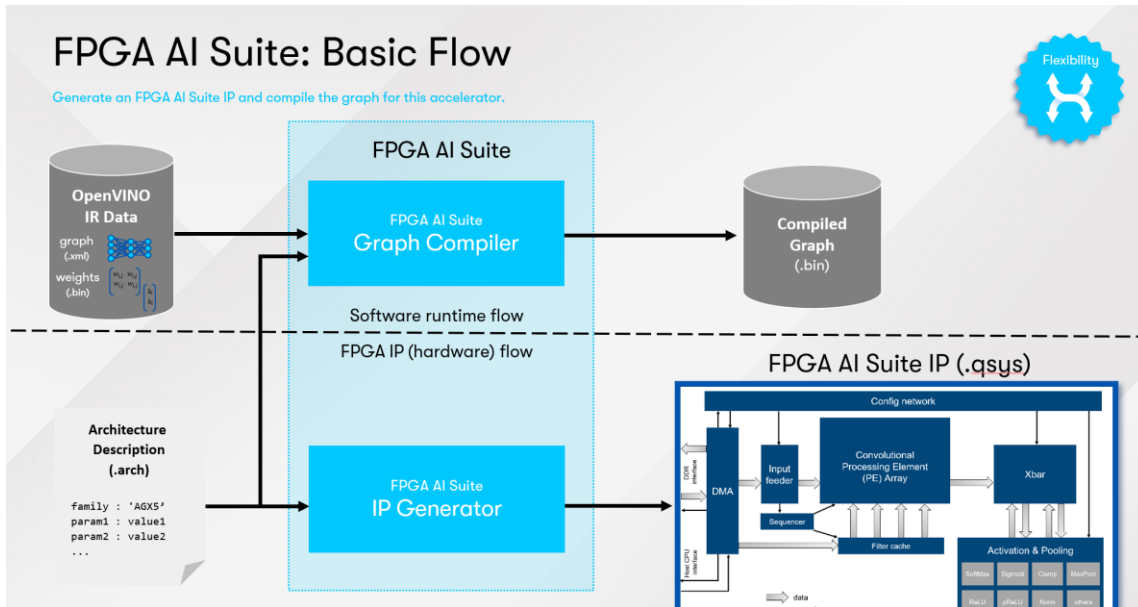
- Creates the AI IP and compiles the graph for a custom accelerator.

2.3.4 FPGAs

- These are stand-alone, hardware devices that will contain the graph and weights. In this case, the target device will be the new Agilex 3 device.

2.4 OpenVINO / FPGA AI Suite Interaction

Details of connections between the OpenVino and the FPGA AI suite are described below.



A high-level explanation:

The FPGA AI Suite creates a **custom IP block that functions as an AI accelerator**. It compiles the graph (AI model) to create execution instructions for the accelerator. These instructions are typically run using the OpenVINO runtime control software. This could run on an external PCIe connected CPU, an integrated ARM processor or a Nios V soft core processor.

A more detailed explanation:

The AI accelerator has a temporal overlay architecture. The IP can be configured to perform a different function for each pass through it. There are two main functional units, the Convolutional Processing Element (PE) Array and the Activation and Pooling block. The particular function that the AI accelerator implements during a pass is determined by the execution instructions.

The OpenVINO runtime software is going to orchestrate configuring the IP layer by layer and the AI graph computation is going to involve many passes through that IP to compute all of these graph layers. The runtime gives the IP a different configuration each pass through so that the correct computation is performed by the overlay for that layer. If some of the layers cannot be offloaded to the FPGA fabric because of the **parameterization** we've chosen for the inference IP, the open VINO runtime will handle those and run them on the CPU.

2.5 IP Generator and Graph Compiler

The FPGA AI Suite acts as a Graph (Model) Compiler and an IP Generator

The **IP Generator** creates the custom **AI accelerator**. It can be customized for performance, resource usage or latency. The customization is provided by the .arch architecture file.

The AI accelerator is not dedicated to any one AI model.

The **Graph Compiler**, creates a **Compiled Graph**. This is a set of execution instructions derived from the AI model (the .xml)

The Graph Compiler also must be aware of the architecture file settings so it can generate appropriate instructions for the **IP accelerator**.

FPGA AI Suite Graph Compiler (also known as DLA Compiler, where DLA stands for deep learning accelerator) does the following:

- generates optimized architecture based on the OpenVino graph (.xml) file
- estimates the performance (i.e. frames per second) of the model
- estimates the FPGA resources for a specific IP parameterization
- produces compiled binaries suitable for execution
- runs command line scripting

2.6 The Architecture Description File

This human readable parametrizable .arch file contains many parameters including:

- Hardware: Target family but not the specific device. In this case, it will be Agilex 3.
- Architecture i.e. number of processing elements (Convolutional blocks), enable/disable activation functions (i.e. used for non-linearity), and pooling functions
- On-chip memory for filter weights and other values.
- Numerical precision (i.e. integer 8 or floating point as fp11, fp12, fp13 or fp16) which in turn affects the inference accuracy
- Memory interface
- and more

2.7 Why Target FPGAs?

This workshop targets an Altera FPGA for AI inference device

2.7.1 Why target Altera FPGAs for AI inference?

There are many reasons for this:

- FPGAs are ideal for custom AI Solutions
- Build high performance custom AI network from the ground up
 - customizable memory hierarchy
 - configurable compute capability
 - programmable data path
- Multiple I/O standard support for data ingest/egest
- Real time with low deterministic latency
- Ideal for edge deployment with low power and form factors
- Lower TCO (total cost of ownership) and TTM (time to market) benefits
- Faster time to market with evolving AI networks
- FPGA reconfigurability provides flexibility for customization as the use case evolves
- Adapt to new networks, data types
- Long product life cycles

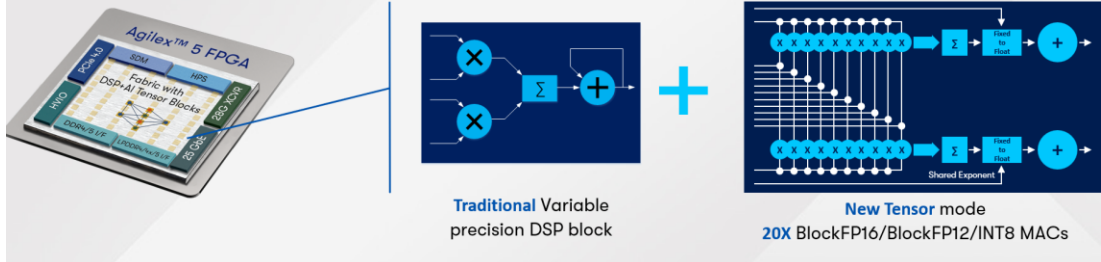
A follow up question to using Altera FPGAs might be: Why would the Altera Agilex devices, such as Agilex 3 and Agilex 5 device families be specifically useful for AI inference?

2.7.2 Agilex 3/5 AI Tensor Block

AI is very math intensive. Agilex 3 and Agilex 5 devices contain hardened DSP blocks with AI Tensor modes. These Tensor mode blocks are highly efficient at implementing the AI math functions.

The AI Tensor Block provides **deterministic** latency and offers the following benefits:

- Integrates AI compute **within flexible FPGA fabric**
- Focuses on **matrix/vector-mult** patterns and introduces **new BlockFP16 and BlockFP12** formats maximizing power/area efficiency
- Completely **backward compatible** with existing FPGA DSP



3 FPGA AI Suite vs other AI solutions

There are different ways to implement AI in an Altera device. A few of them are discussed below, where you can see the comparisons between them.

3.1 TinyML

TinyML has the following features:

- Very low-level hardware implementation of machine learning.
- Method for implementing machine learning (ML) on extremely low power, small size, edge-AI applications devices.
- Capable of interfacing with Nios V (soft-core processor) or ARM (hard-core processor).
- Free to use (no cost).
- Use Python -TensorFlow software (or other open-source ML software) to create a model, prepare the dataset, and run the training.
- Specific, known models i.e. LeNet-5, Rosetta-Net, etc. need to be known before implementation.
- Uses Windows, WSL or Linux for operating systems

3.2 ONE WARE Studio

ONE WARE Studio has the following features:

- Software studio that offers a higher level of abstraction as compared to Tiny ML.
- Creates custom AI models based on the GUI settings. The resulting model is tailored to your specifications, hardware and dataset rather than using pre-designed models like LeNet-5 or Resnet-50.
- No prior knowledge of AI models is required.
- Often creates AI models that are more efficient than custom hand-created AI models
- Compatible with Nios V (FPGA)
- Generates RTL (VHDL) code
- Exported code can be used in the Quartus Platform Designer for integration with other FPGA IPs
- Operates under Windows OS

3.3 FPGA AI Suite

FPGA AI Suite has the following features:

- AI inference development platform
- A way to convert a pre-trained AI model and then implement the result in Quartus
- Knowledge of specific AI models is needed.

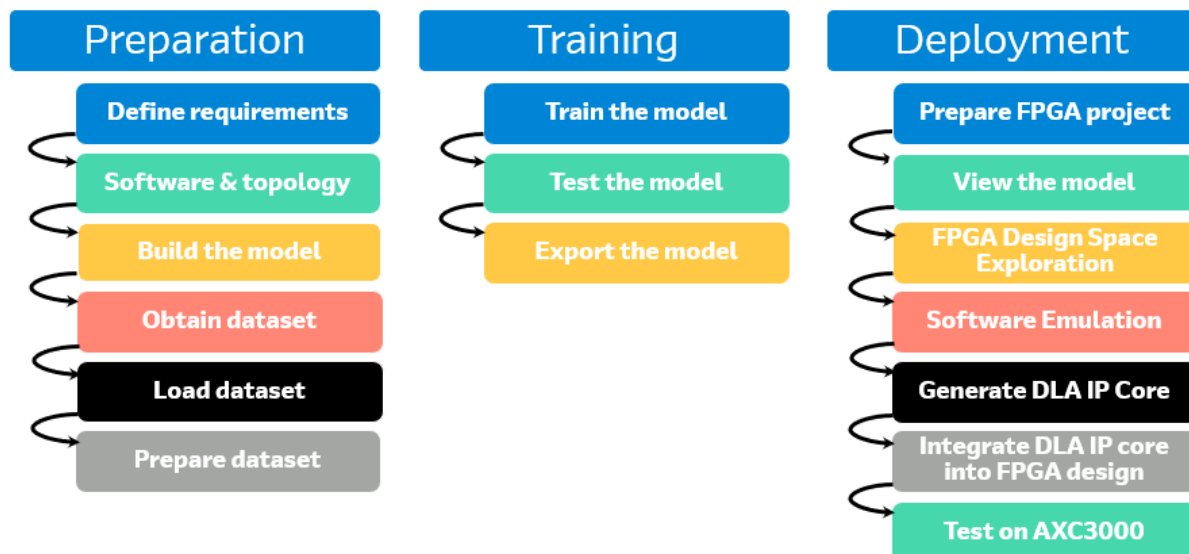
- Provides a predicted estimation of device resource utilization and performance.
- Commonly utilized on Altera's Agilex and Stratix device families.
- Capable of delivering high-performance solutions, achieving high frames per second (fps).
- Command-line based software
- Operates on Linux OS or WSL.

4 Workshop steps

The following steps describe the flow of the AI workshop.

- Design requirement.
- Software and network topology.
- Build the model
- Obtain a dataset.
- Load Dataset.
- Prepare data set.
- Train the model.
- Test the model.
- Export the model.
- Prepare FPGA project.
- View the model.
- FPGA Design Space Exploration.
- Software Emulation.
- Generate the DLA IP Core.
- Integrate the DLA IP Core into the FPGA example design.
- Test the design on the AXC3000 hardware.

The overall workflow of this workshop can be visualized in three stages, as illustrated in the following diagram. You will be guided through each of these stages step by step.

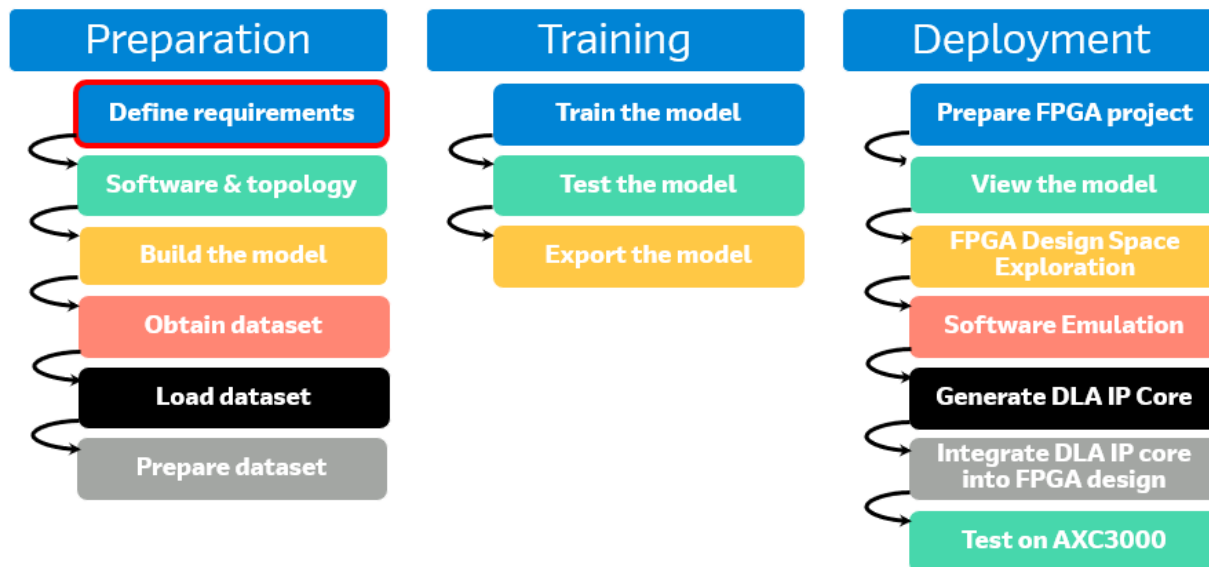


5 Design Requirements

The first step for any design process is to clearly define the requirements. This step is very important because it affects

- Design
- Dataset selection.
- Choice of models to be implemented.
- Target device.
- Results.

This foundational step is visually represented in the red box of the workflow diagram.



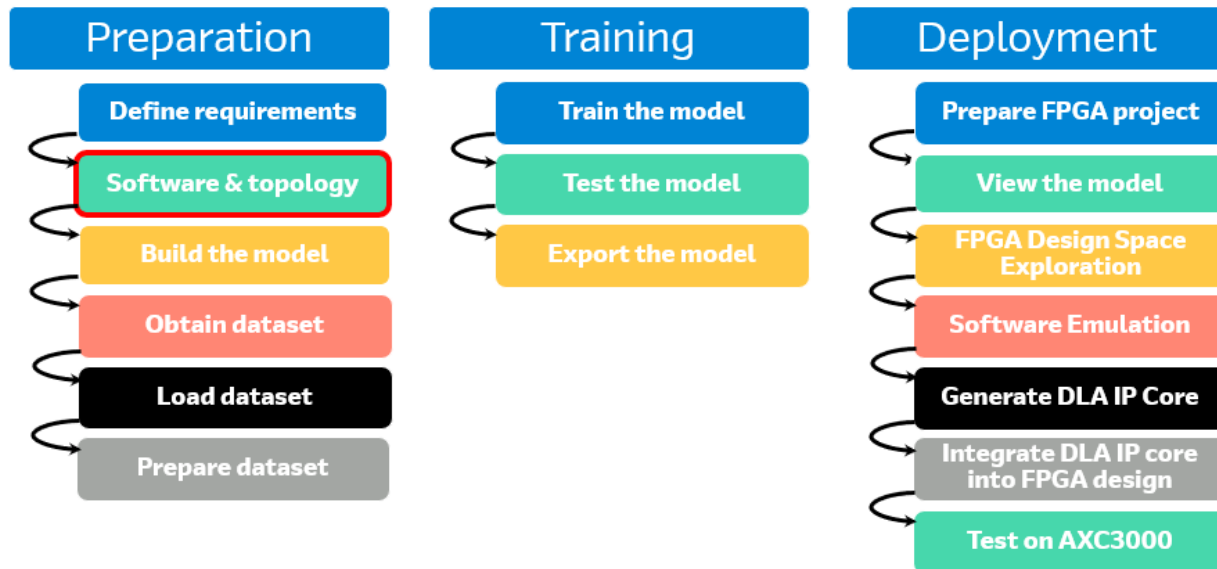
The design requirements for this workshop are as follows:

- Utilize ML to classify a single digit from the number 0 to 9, resulting in 10 distinct output categories.
- Use a small CNN LeNet-5 model. This model is less complex than other CNN models but still shows the ML flow. LeNet-5 is a good model for digit classification.
- Use the database from MNIST (Modified National Institute of Standards and Technology) that contains a large collection of handwritten digits (ie 60,000) to determine classification of the digits. The MNIST database is the equivalent of the “Hello World”. The MNIST is smaller than other datasets.
- The image ‘4’ represents a sample of a digit from this MNIST dataset.



6 Software and Network Topology

The next step in the workflow is to determine the network topology and select the appropriate framework (software). This step is visually represented in the red box with the workflow diagram.



6.1 PyTorch Software framework

A software framework for AI is a structured set of tools, libraries, and components designed to simplify the development, training, deployment, and management of artificial intelligence models and applications. It provides reusable abstractions and standardized workflows, so developers don't have to build core functionality from scratch.

There are different software frameworks available that include PyTorch, TensorFlow, ONNX, Keras, and others. The choice of the framework depends on factors such as the application that it is used for, previous user experience, what is already used in a company and more. PyTorch, which is used in this workshop, can be used for quick research prototyping.

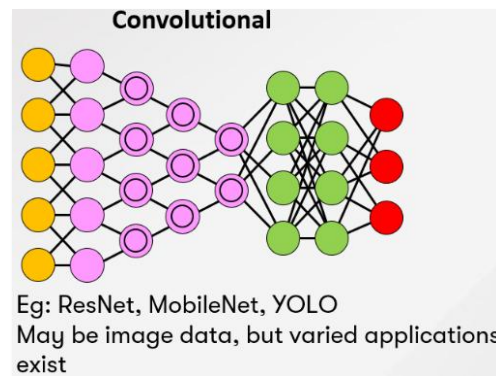
PyTorch software framework was originally released in January 2017. It was created by Facebook's AI Research Lab (FAIR). Before PyTorch, FAIR had developed Torch, a scientific computing framework in Lua (2002–2016). Torch was fast but its Lua base limited adoption. PyTorch was developed as its Python successor, leveraging Python's ecosystem and dynamic graph execution.

PyTorch provides researchers with a framework that is both flexible (for rapid prototyping) and efficient (for large-scale deep learning).

PyTorch began as Facebook’s answer to Torch’s limitations and quickly became the leading deep learning framework due to its dynamic graph design, Python integration, and ease of use. From research prototyping to large-scale industrial deployment, it has progressed into a full-stack ecosystem with global adoption and community-driven governance under the PyTorch Foundation.

6.2 Convolutional Neural Network Topology

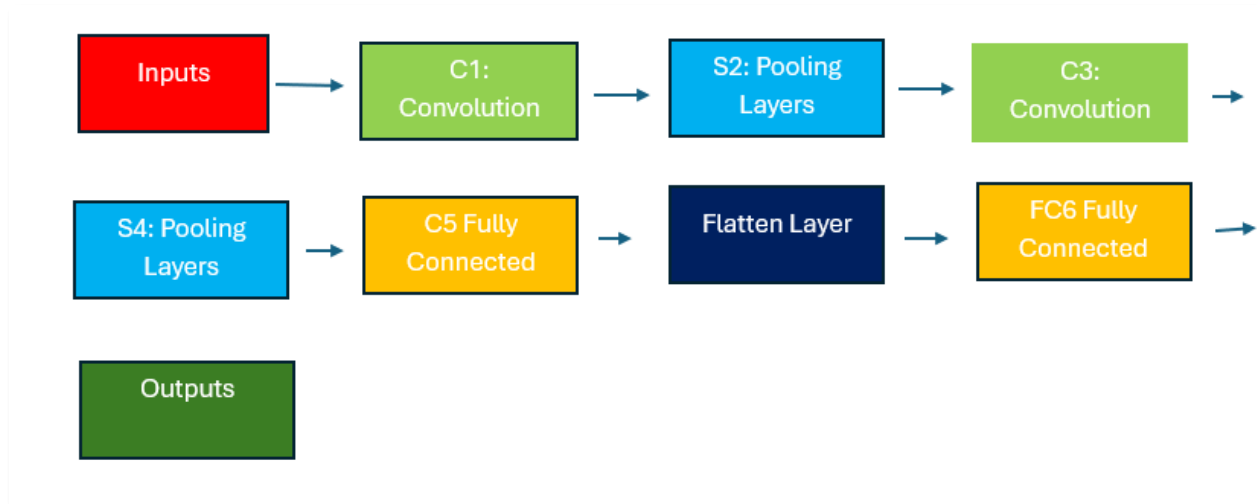
This workshop will use a Convolutional Neural Network (CNN) topology, specifically a [LeNet-5 model](#). This model is commonly used for image detection. CNN topologies are a preferred choice due to their effectiveness in processing visual data. The topology of CNN is shown below.



From the previous figure, the input layers are yellow “neurons” (where the data is input), Hidden layers (i.e. the “calculation layers and classification layers”) would be the purple and green “neurons”. Purple neurons represent the convolutional and pooling layers which are the computational layers. The green neurons, which are the fully connected layers, are the classification layers. The output layer is the red “neurons”.

In the requirements section, it was stated that the LeNet-5 model was selected due to model simplicity.

The LeNet-5 model is shown below.



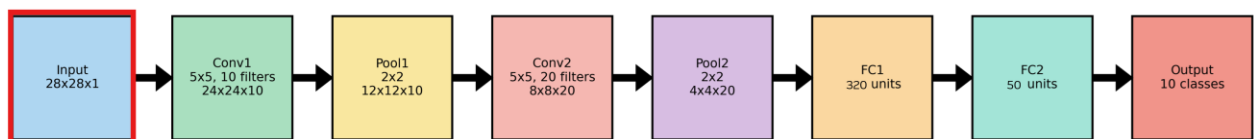
LeNet-5 seems to imply 5 layers (i.e. the number 5 in the word LeNet-5), but it has 7 layers. The Input layer does not count. There are 2 convolutional layers (C1 and C3) where the features of the image are extracted, 2 pooling layers (S2, and S4) where there is down sampling, 1 fully connected layer (where the data is linked), and 1 output layer (where the data is output).

The LeNet-5 model receives at its input layer, 28x28 pixel monochrome images and produces the probability of it being in a particular category (i.e. a three or a five for digit recognition) by the time the data has propagated through to its output layer.

6.3 Lab Model Specifics

The section below provides the specific details needed to understand the network that is created with the PyTorch framework in this lab.

6.3.1 Input Layer



The input layer receives images that are 28 pixels wide by 28 pixels by 1 channel. The single channel indicates that it is monochrome.

The matrixes are the same in height and width i.e. 28×28 , because it makes math easier to compute. If the model was processing color images, then the channel value would be set to 3, to represent red, green, and blue. There is a total of $28 \times 28 \times 1 = 784$ features for each image.

6.3.2 Conv1: 2D Convolution layer



The first convolutional layer (C1) does the dot product (math) where there are 10 filters with kernel (matrix) of 5×5 . Kernel of 5×5 means that it is a filter of 5×5 that will travel across the image from left to right and from top to bottom.

There are 10 filters because this number of filters is a reasonable starting condition for using MNIST dataset. With these 10 filters, it would look for the basic features of the numbers such as horizontal, vertical lines, and diagonal lines.

Stride of 1,1 means that the matrix (kernel) moves stepwise by 1 through the data for convolution.

Activation infers that when a section of the image is seen, the “neurons” will respond (be activated). There are different modes of activation. In this case, the activation is ReLu (Rectified Linear unit), which introduces non-linearity. Should there be a non-negative number, the result is zero, but if it is positive, the result will be positive.

The 5x5 kernel traverses the 28 x 28-pixel array horizontally from left to right and vertically from top to bottom. Each step of the kernel produces an entry in the resultant array. There can be only n number of steps horizontally, where $n = \text{the maximum array width} - \text{the width of the kernel} + 1$. The same is calculated for the vertical steps. There are ten filters employed, each looking for different features in the image. The output from this layer is ten 24 x 24 arrays of filtered data.

6.3.3 Pool1: Pooling Layer



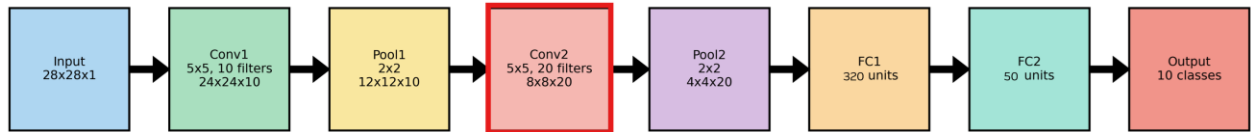
The LeNet-5 pooling layer performs down sampling via **averaging**, reducing resolution and introducing robustness to small input variations.

Each pooling layer takes small neighborhoods of the input feature maps (typically 2×2) and reduces them to a single value.

This lowers the spatial resolution of feature maps, cutting width and height roughly in half. It also reduces the number of parameters and computation in subsequent layers.

The output of this layer is ten 12 by 12 arrays, since the subsampling is set to 2.

6.3.4 Conv2: 2D Convolution layer

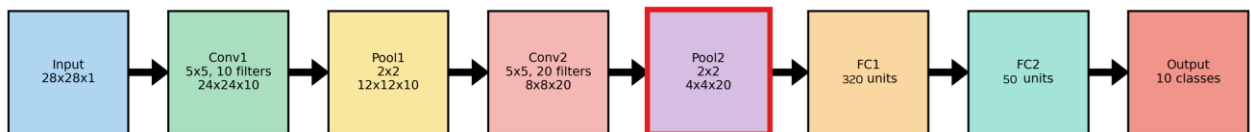


The 2nd convolutional layer extracts more features. It is very common to have multiple convolutional and pooling layers in a model.

There are 20 filters with a kernel size (filter) of 5×5 and a stride of (1,1). This also uses a ReLU activation function. The output parameters are 8,8,20.

Pool2: Pooling Layer

This pooling layer again performs down sampling via **averaging** 2×2 areas of pixels to a single one.



6.3.5 Fully Connected Layers (FC1 and FC2)



The output from the pooling layer is flattened from 20 4×4 arrays into a single dimension 320×1 array. Classification occurs in these layers. The first fully connected layer produces 50 classification units which are then reduced to the final 10 units used in the output layer. Each unit represents one handwritten digit, from zero to nine. The value contained in the unit represents the probability of it recognizing that digit. Values range from zero to one.

6.3.6 Softmax function

The output is a 10×1 vector with each location containing an FP32 value. The softmax function maps this vector of real numbers into a probability distribution that has the following properties:

- Output values are between 0 and 1
- Sum of all the outputs equals 1

An example of this is:

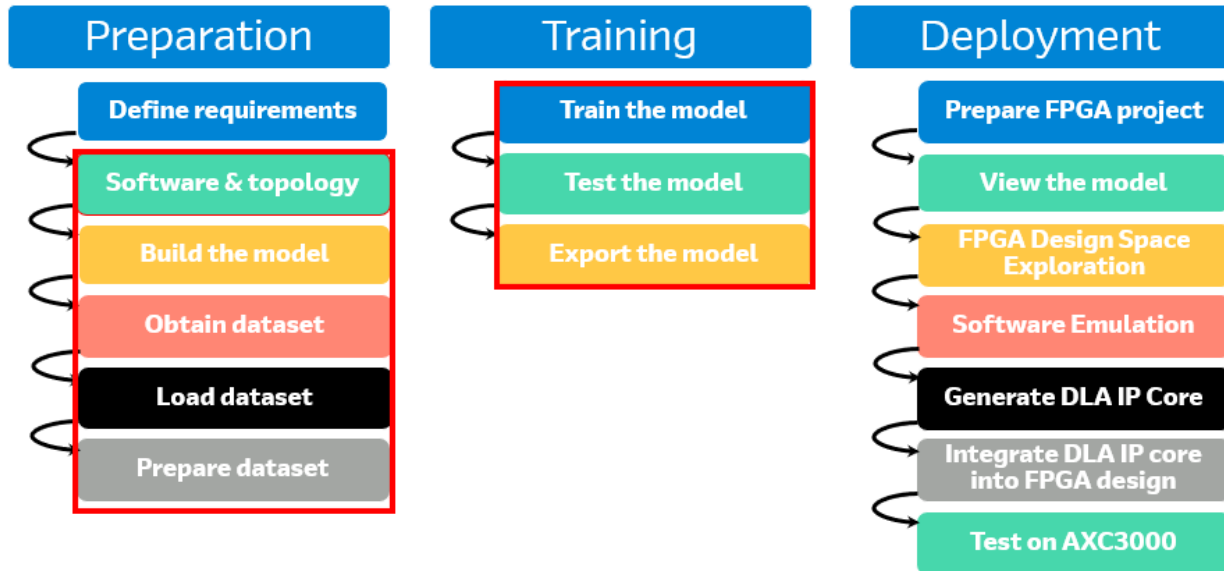
- Input vector = [2.0, 1.0, 0.1]
- Output vector = [0.659, 0.322, 0.032]

To calculate softmax, add the input vector values together ie $2+1+0.1 = 3.1$. Then, divide each value of input by the total ie $2.0/3.1 = 0.659$, $1.0/3.1 = 0.322$ and $0.1/3.1 = 0.032$

The probability of the three classes can be calculated by making the output vector as % as 65.9%, 32.2%, and 3.2% respectively.

7 Build, train, test, and export the model

This section of the lab will be completed in the Colab environment. The model is created, and the dataset is obtained, loaded, and prepared in this section of the lab. Finally, the model is trained, tested, and exported.

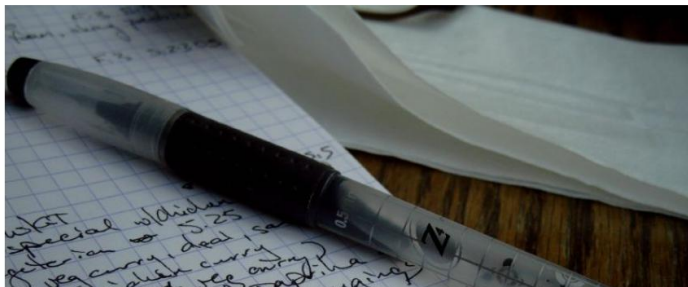


The lab is based on an article written by Gregor Koehler; [MNIST Handwritten Digit Recognition in PyTorch](#). Before continuing, please read the article. It explains the flow of the code, from importing MNIST datasets, training the model and evaluating its performance. Click on the image below to access the article.

Gregor Koehler / Feb 17 2020
with Andrea Amantini and Philippa Markovics
Remix of PyTorch Environment by Martin Kavalár

MNIST Handwritten Digit Recognition in PyTorch

In this article we'll build a simple convolutional neural network in PyTorch and train it to recognize handwritten digits using the MNIST dataset. Training a *classifier* on the MNIST dataset can be regarded as the *hello world* of image recognition.

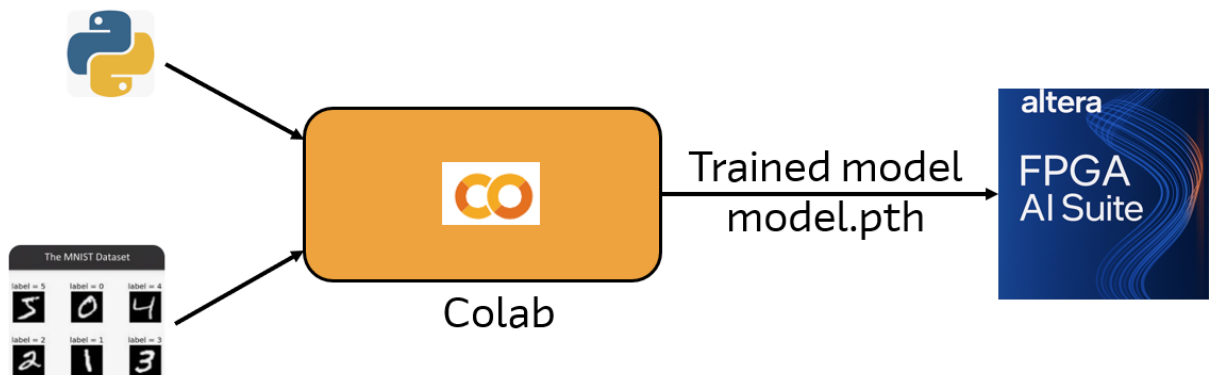


An existing LeNet-5 model, based on the article above, will be used for this lab. It was written in Python and it makes use of the PyTorch software framework.

This lab will use Colab, a cloud-based development environment, hosted by Google. The AI workshop will use it to train and verify the CNN model.

The Python notebook is loaded into the Colab environment. The code is then executed. The MNIST training and testing datasets are then loaded. After training and testing successfully complete, a trained model will be exported as a .pth file. This will be modified into two intermediary (bin & xml) formats and imported in the Altera FPGA AI suite

Python Notebook



MNIST data set

Google Colab is a cloud-based interactive computing environment provided by Google. It is built on top of Jupyter Notebooks and allows users to write and execute Python code directly in the browser.

A summary of the Python Notebook code includes:

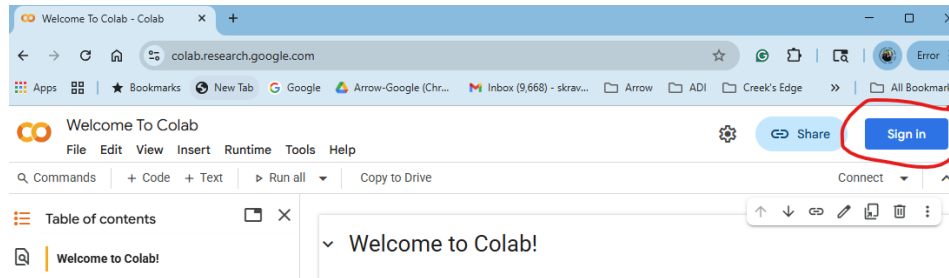
- Import an existing PyTorch environment
- Prepare the dataset.
 - Define the batch sizes for training and testing
 - Import the MNIST images and labels for training and testing
 - View six of the database images
- Build and initialize the network
- Train and test the model
- Evaluate the model's performance
- Exports the model, including weights and biases as a .pth file.

7.1 Open Colab

7.1.1 Click on the following link to open Colab in a browser

<https://colab.research.google.com/>

7.1.2 Sign in using your Google credentials



7.2 Load the PyTorch Notebook

The model is provided as part of the source files for this lab.

Navigate to the lab in the cloned workshop repository and open it in Colab.

7.2.1 Load the Jupyter notebook

Jupyter notebook is a web-based application that can be used for coding, machine learning and more.

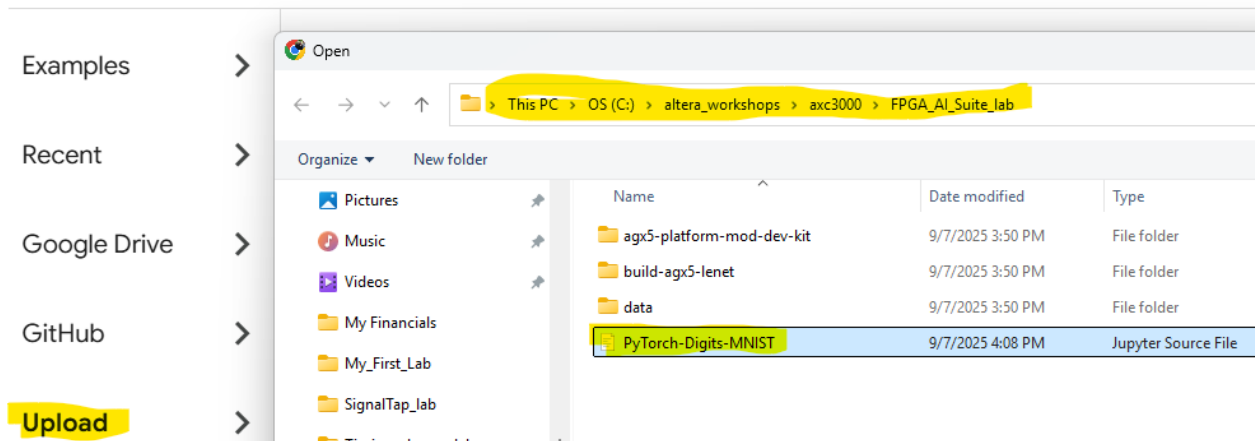
7.2.2 File → Open notebook → Upload.

7.2.3 Navigate to C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\

7.2.4 Select PyTorch-Digits-MNIST. Click Open.

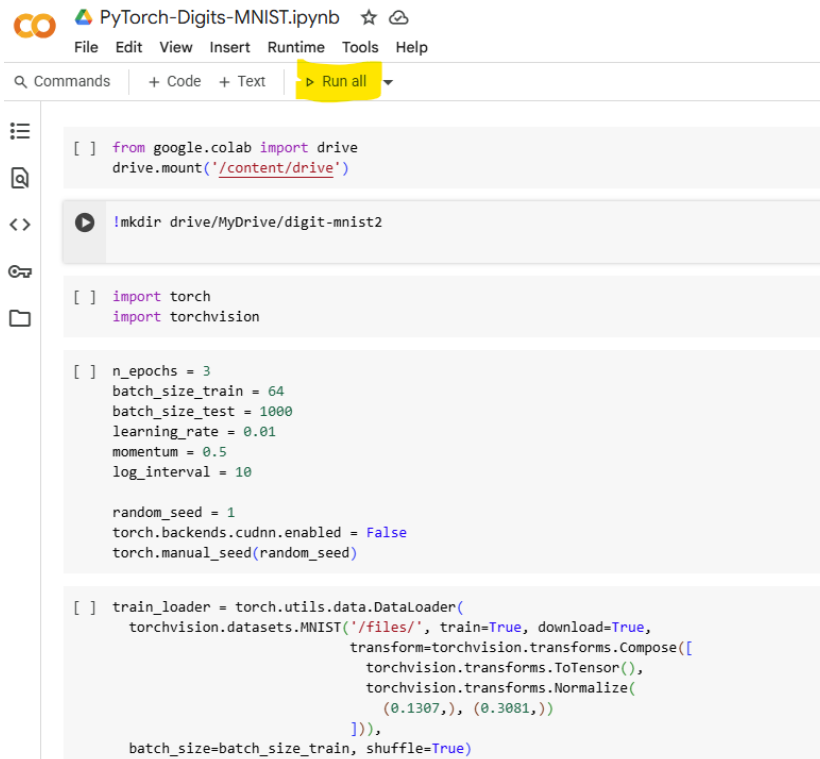
Just for reference, the MNIST dataset has 60,000 images used for training and 10,000 used for testing. When working with datasets, a portion of the dataset will need to be used only for testing.

Open notebook



7.3 Execute the code

7.3.1 A view of the environment is shown below. Click Run all.



The early section of the code imports the Pytorch library and the Torchvision library (used for image processing)

The line that states `n_epochs = 3` refers to the number of times that it goes through an entire pass through the dataset. A model needs to go through the dataset to get higher accuracy. This means that the model will go through the large MNIST dataset (60,000 items) 3 times.

Batch size refers to the number of images that will be processed (via forward and backpropagation) before the model weights are changed. Note that the batch size of the training function is much less (which is expected) than the batch size of the testing function. This is because the model needs to be carefully trained first.

Learning rate of 0.01 is a default rate. If the learning rate was much smaller such as 0.001, the learning time would be much slower

When **Run all** is selected, all the steps that have the arrow and the **[]** in front of them will be run.

7.3.2 Allow the notebook to access your Google Drive

Permit this notebook to access your Google Drive files?

This notebook is requesting access to your Google Drive files. Granting access to Google Drive will permit code executed in the notebook to modify files in your Google Drive. Make sure to review notebook code prior to allowing this access.

No thanks

Connect to Google Drive

The code can only be executed after allowing Google Drive access.

7.3.3 Select your account in the next pop-up window. Then click on continue in the two following pop-up windows.

7.4 Review the results

As the model is being trained, the results will look like (which are the same results as seen from the web site of [MNIST Handwritten Digit Recognition in PyTorch – Nextjournal](#))

```

Train Epoch: 3 [51840/60000 (86%)] Loss: 0.223256
Train Epoch: 3 [52480/60000 (87%)] Loss: 0.220552
Train Epoch: 3 [53120/60000 (88%)] Loss: 0.235116
Train Epoch: 3 [53760/60000 (90%)] Loss: 0.416173
Train Epoch: 3 [54400/60000 (91%)] Loss: 0.427750
Train Epoch: 3 [55040/60000 (92%)] Loss: 0.286725
Train Epoch: 3 [55680/60000 (93%)] Loss: 0.256722
Train Epoch: 3 [56320/60000 (94%)] Loss: 0.313162
Train Epoch: 3 [56960/60000 (95%)] Loss: 0.325091
Train Epoch: 3 [57600/60000 (96%)] Loss: 0.184333
Train Epoch: 3 [58240/60000 (97%)] Loss: 0.098307
Train Epoch: 3 [58880/60000 (98%)] Loss: 0.195419
Train Epoch: 3 [59520/60000 (99%)] Loss: 0.320958

Test set: Avg. loss: 0.0925, Accuracy: 9721/10000 (97%)

```

Note there are different sections being reported:

- number of epochs
- number of datasets
- loss and accuracy

There are 3 epochs (entire passes through the dataset), because the setting in the code was set to three. The model ran 3 times through the MNIST dataset of 60,000 handwriting.

Number

- This is reported for each batch size increment, up to 60,000 digits. For example, it shows results for 58,880 and 59,520 digits.

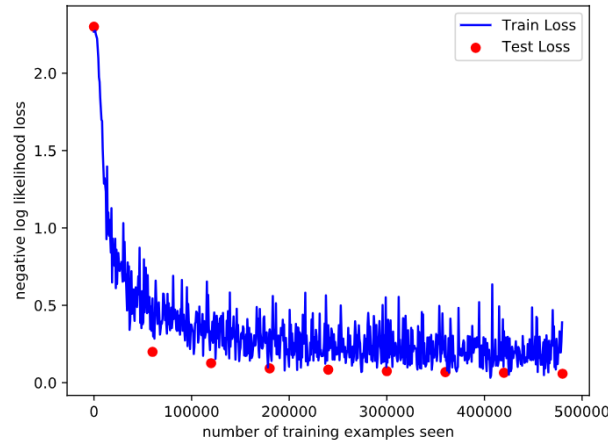
Loss

- This is training loss. Training loss is the calculation of how incorrect the model predicts the results. It does this calculation by performing a loss function. The loss function uses backpropagation in the model to modify the weights and biases. The training loss decreases as the model runs through epochs. This indicates that the model is learning. The goal for loss is to get as close to zero. The average loss is 0.0925 which is small.

Accuracy

- This tends to be the inverse of training loss. It indicates how correct the model predictions are. The goal for accuracy is to get as close to 100 % as possible. In this case, the average loss is 97% which is very close to 100%

If the losses are plotted (as seen at [MNIST Handwritten Digit Recognition in PyTorch – Nextjournal](#)) note how the training and test losses approach zero. This is the expected response, as it shows that model is “learning”.



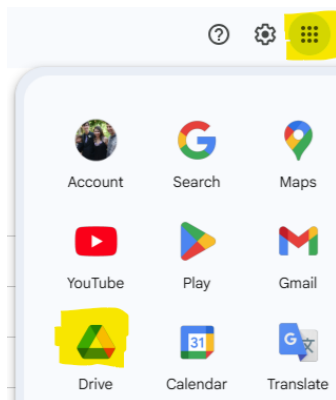
7.5 Export the model

7.5.1 Open **Google Drive** (cloud-based drive to share and access files) with the following steps

7.5.1.1 Open a **new tab** in the **browser**

7.5.1.2 Type `https:// google.com` this new browser tab.

7.5.1.3 In the new tab, select on the Google Apps icon (in yellow below) and then select Drive (also in yellow below).



7.5.2 Navigate to My Drive > digit-mnist2

My Drive > digit-mnist2 ▾

Type ▾	People ▾	Modified ▾	Source ▾
Name ↑	Owner		
 model.pth	 me		
 optimizer.pth	 me		
 t10k-images-idx3-ubyte.gz	 me		
 t10k-labels-idx1-ubyte.gz	 me		
 train-images-idx3-ubyte.gz	 me		
 train-labels-idx1-ubyte.gz	 me		

There are a number of different types of files in the directory shown above.

pth files are PyTorch files that contain the models' details such as the weights, biases and other parameter settings. The two gz files (shown above) with the word “image” in them refer to the actual data, while the files with word labels in in them refer to the labels of the data. Labels are needed so that the model can “check” if its predictions are correct or not. Should the predictions not be correct, the model will adjust parameters accordingly.

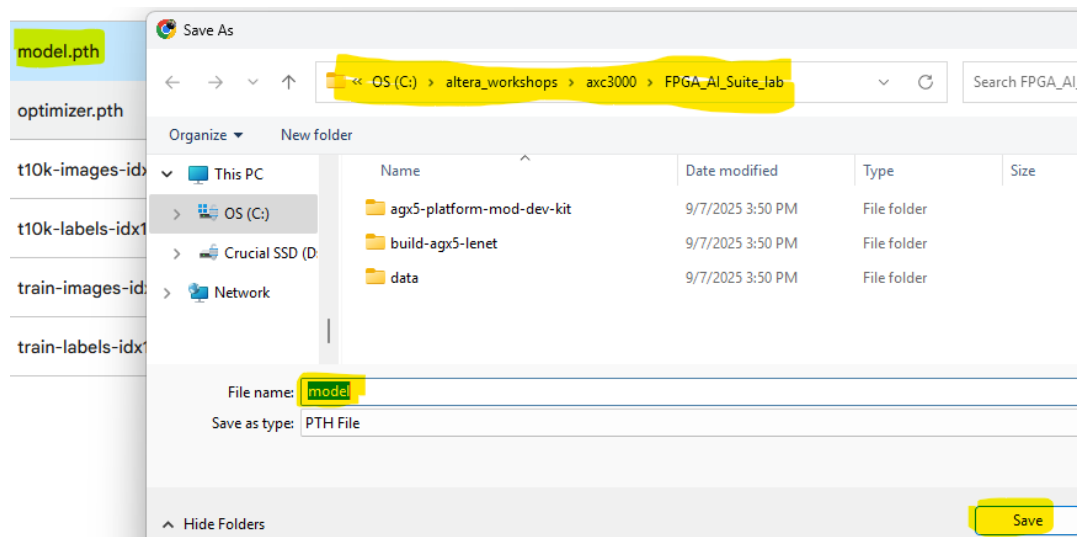
7.5.3 Export the model and place the model in the applicable directory.

7.5.3.1 Right click on **model.pth**.

7.5.3.2 Select Download. This file will be downloaded into the Downloads directory.

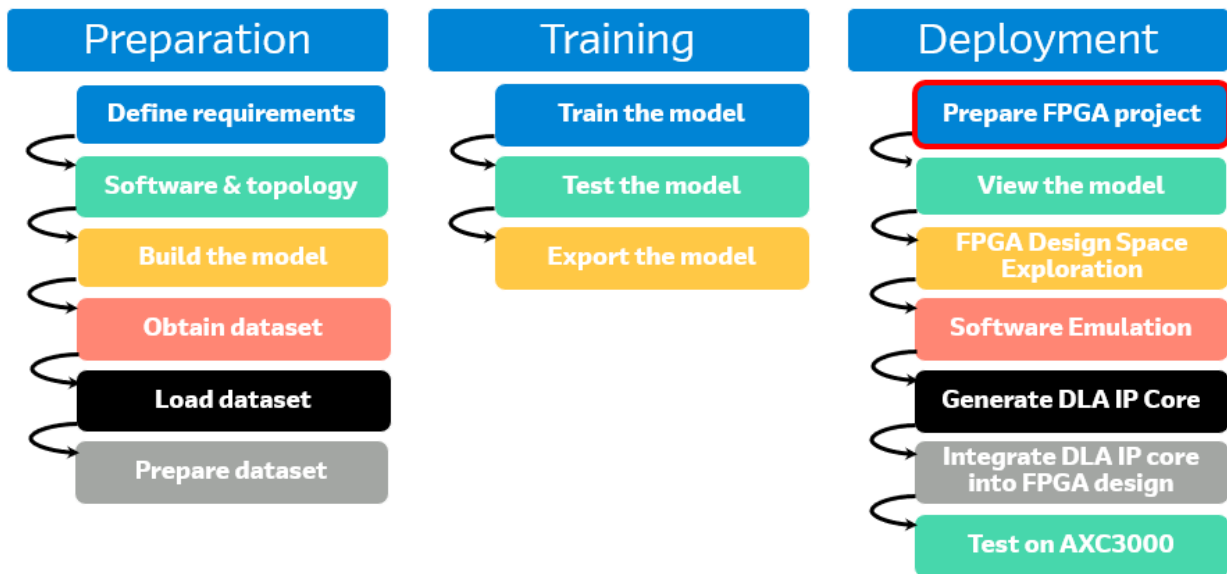
7.5.3.3 If the **Save As** option appears, then navigate to **Downloads** folder. If not, then the file has been automatically downloaded to the **Downloads** folder.

7.5.3.4 Go to the Downloads directory and copy the file to the following folder



pth file is a Pytorch file that contains the details of the model such as the weights, biases and other parameter settings.

8 Prepare the FPGA project



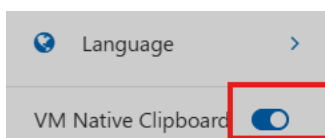
Deployment is an important stage because the AI model will be implemented in a stand-alone device.

The first stage in this deployment step is that the FPGA software (running under WSL and Linux) needs to be set up and then initialized.

For this workshop, it will run Ubuntu under WSL. The reason for WSL is that the other Agilex 3 workshops use Windows and thus only a Windows based Cloud Server is needed. However, if you are running the workshop as stand-alone or in future designs, you could use Linux instead of WSL.

Important: The commands that are required to be entered into **Ubuntu** or **Command Prompt shells** from sections **8.1.2** to **13.1.1** have been provided in a file named **lab_commands.txt**. They can be cut and pasted into their respective shells for convenience and accuracy. The file can be found in the lab folder.

CloudLabs users: To cut and paste within the VM, click on the More  button on the top right part of the screen. Then click on the VM Native Clipboard. Click OK on the Information screen. To cut and paste text, select the text, then right click on the selected text and press copy. To paste, right click in the shell.



8.1 Launch Ubuntu in a WSL shell

- 8.1.1 Ubuntu is a widely used Linux software, but here it is run under Windows OS. WSL, which stands for Windows Subsystem of Linux, is included in the Windows operating system. Type 'PowerShell' into the windows search bar. Select Windows PowerShell.



- 8.1.2 Type 'wsl -d Ubuntu-22.04' in the PowerShell

- 8.1.3 Ubuntu 22.04 is now available in the shell.

8.2 Initialize the environment.

- 8.2.1 Provide a path to the System Console installation in Windows. This will be used to download the FPGA image to the AXC3000 and extract the AI engine results.

```
$ export PATH="/mnt/c/altera_pro/25.1.1/syscon/bin:$PATH"
```

- 8.2.2 Configure the OpenVINO environment and FPGA AI Suite environments. The OpenVINO and FPGA AI suite environments must be set up in Linux before they are utilized. These commands could be added to the .bashrc file for convenience in future.

```
$ source ~/build-openvino-dev/openvino_env/bin/activate
```

```
$ source /opt/intel/openvino_2024.6.0/setupvars.sh
```

```
$ source /opt/altera/fpga_ai_suite_2025.1.1/dla/setupvars.sh
```

```
(openvino_env) altera@Arrow-Host:~$ source /opt/intel/openvino_2024.6.0/setupvars.sh
[setupvars.sh] OpenVINO environment initialized
(openvino_env) altera@Arrow-Host:~$ source /opt/altera/fpga_ai_suite_2025.1.1/dla/setupvars.sh
export COREDLA_VERSION=1
export COREDLA_ROOT='/opt/altera/fpga_ai_suite_2025.1.1/dla'
```

8.2.3 Run the following command to verify correct configuration

```
$ dla_compiler --march \
    $COREDLA_ROOT/example_architectures/AGX5_Performance.arch \
    -- fanalyze-area
```

```
((openvino_env) ) altera@BQ6BH74:~$ dla_compiler --march $COREDLA_ROOT/example_architectures/AGX5_Performance.arch --fanalyze-area
Architecture set to /opt/altera/fpga_ai_suite_2025.1.1/dla/example_architectures/AGX5_Performance.arch
Exporting input transform to file
Exporting output transform to file
Executing area estimate
Estimated area:
ALMs: 74222
ALUTs: 57384
Registers: 187891
DSPs: 266
M20Ks: 1162
Memory ALMs: 1960
M20K Width/Depth Breakdown:
(40,512): 546
(10,2048): 516
DSP Configuration: Tensor FP with ChainOut
```

The command tells the dla compiler (FPGA Ai Suite) to use an architecture file (AGX5_Performance.arch) from the directory path (COREDLA_ROOT/example_architectures) and then analyze it for area. This command is run to see if the software is set up correctly.

8.3 Copy the project files into the working directory

OpenVINO and FPGA_AI_Suite require that the files are resident in a Linux filesystem.

8.3.1 Create a working directory

```
$ mkdir ~/coredla_work && cd ~/coredla_work
```

8.3.2 Copy the project source files to the Linux environment.

```
$ cp -a /mnt/c/altera_workshops/axc3000/FPGA_AI_Suite_lab/* \
    ~/coredla_work
```

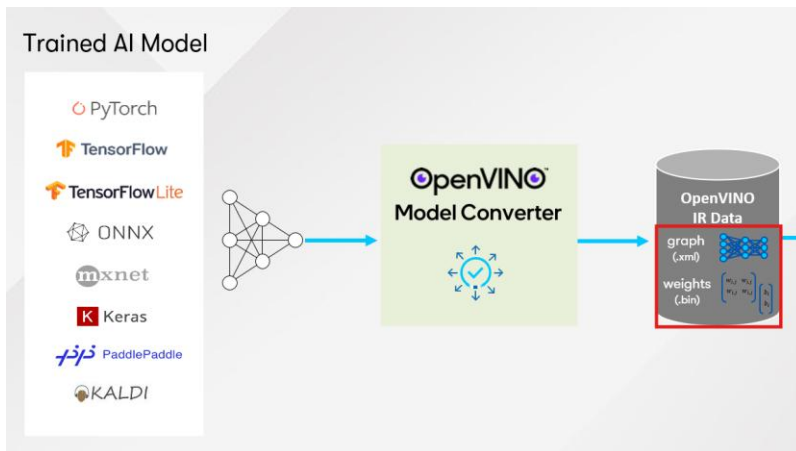
8.3.3 Install demo project and runtime content. This can take a few minutes.

```
$ source dla_init_local_directory.sh
```

8.4 Prepare the model for OpenVINO and FPGA AI Suite

OpenVINO can receive files generated by several different frameworks. In this project a Python script is used to convert the PyTorch.pth file into an ONNX format. The OpenVINO ovc utility converts it into two discrete files, bin and xml. The bin file contains the weights and biases that were determined during training and testing. The xml file contains the model's structure.

A graphical, top level view of this flow shows where the IR files are generated.

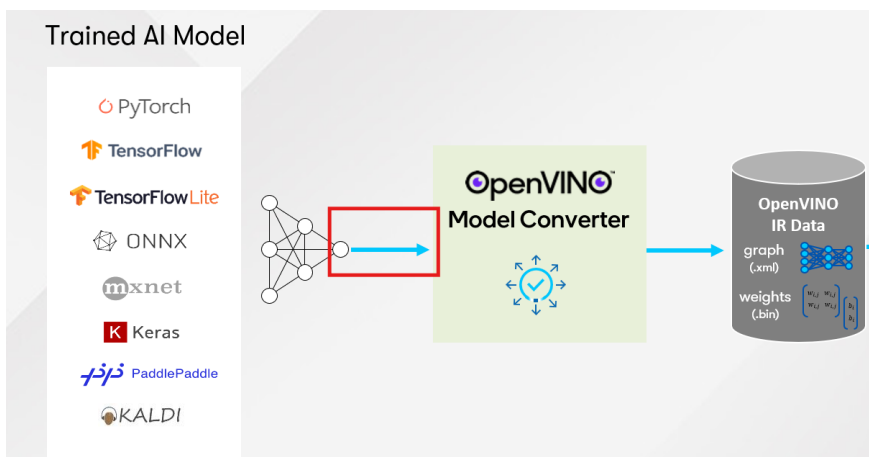


The following instructions will go through the individual steps of going from the trained AI model to OpenVINO IR files.

- 8.4.1 Convert the model from PyTorch to ONNX. Use a Python script to do this. This produces a file named Digit-classifier.onnx (Ignore the **error** shown in the shell. The ONNX file is generated.)

```
$ python convert-pt-2-onnx.py
```

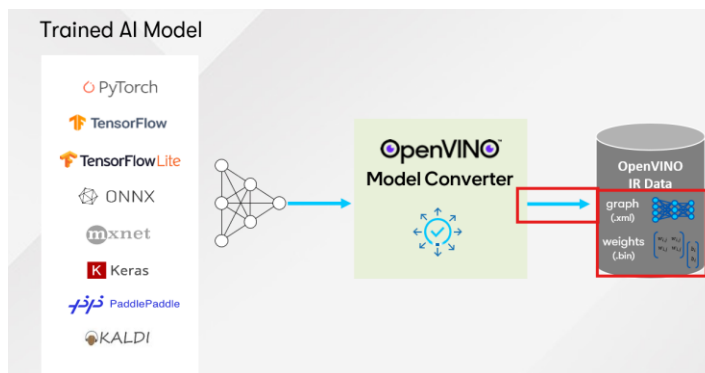
This step is seen as:



8.4.2 For reference, ONNX (which stands for Open Neural Network eXchange) is an open standard file that is used for machine learning. ONNX files can be exported and imported into other software tools. Convert from ONNX to bin and xml. This will be used by FPGA AI Suite to create an AI IP block in the Agilex 3 device. The option **--compress_to_fp16 False** maintains the FP32 accuracy of the weights and biases. This ensures accuracy before introducing FP16 optimizations.

```
$ ovc Digit-classifier.onnx --compress_to_fp16 False
```

This step is shown below:



An xml and bin file are produced by this step. The .xml file defines the model (graph), while the .bin file contains the weights and biases.

For reference, the xml file (eXtensible markup language) is a file that can be read by humans and computers. Xml is a “markup language which describes data, and enables the definition, transmission, validation, and interpretation of data between applications”. A markup language means the file contains tags. A tag in a file, which uses the <> in a file, defines data and even connections between the data. An example of a tag can be seen below.

XML files can be used for many different data-based applications. In this case, this .xml file describes the neural network, where it describes the layers, connections, settings and the link to the .bin file.

An example of an .xml file is listed below:

```
<xml version =
<model>
  <layers>
    <layer id = '0", name = "input" type = "Parameter">
      <output>
        <port id = "0", precision = "FP32">
          <dim> 1 </dim>
          <dim> 1 </dim>
          <dim> 28 </dim>
          <dim> 28 </dim>
        </port >
      </output>
    <layer id = '1', name 'Convolution'
      <input>
        <port id = "0", precision = "FP32">
          <dim> 1 </dim>
          <dim> 1 </dim>
          <dim> 24 </dim>
          <dim> 24 </dim>
        <weights size = "784" --this step refers to binary file (.bin)
      </input>
    <edges>
      <edge from layer 0 port 0 to layer 1 port 0>
    </edge>
  </model>
```

The values in the **<>** are **tags** (i.e. definition of what is being defined).

The **xml version** indicates the version of the xml software.

The **<layer>** refers to the building blocks of the model. There are 2 layers, layer id 0 and layer id 1, that are defined above. Precision is set to floating point. Both layers define dimension **<dim>** which specify the height and width of the incoming image.

Edges refer to connections between the building blocks.

As the .xml file does not contain the weights, the **.xml** file calls the **.bin** (binary file) for the them. The .xml file does **not** call the .bin file explicitly, but rather it refers to definitions in the .xml file.

By default, the name of the .xml file and the .bin file are the same, but the file suffix is different. By default, these files are also located in the same directory.

/ in **<>** indicates the end of the definition or section.

8.4.3 Copy the file from Ubuntu to Windows.

```
$ cp Digit-classifier.onnx /mnt/c/altera_workshops/\
axc3000/FPGA_AI_Suite_lab
```

9 Visualization using the ONNX file

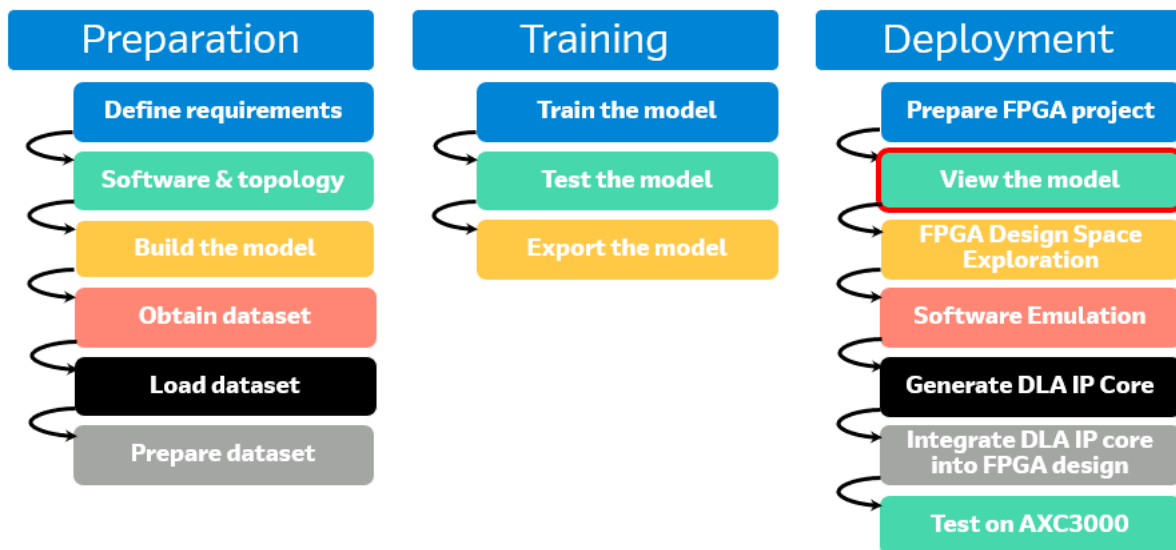
After you have set various settings in the PyTorch framework, trained the model, and experimented with the model, you might have the following questions:

- What does the model look like?
- How does the model compare to the LeNet-5 model covered in the background material?

There are several ways to visualize the model. One method is to view the results using the ONNX file. This section will focus on visualization using the previously exported ONNX file.

ONNX, which stands for the Open Neural Network eXchange, is an open standard used for machine learning. This file allows interoperability and portability between different frameworks (software). Different software can generate an ONNX file which can then the file can be visualized.

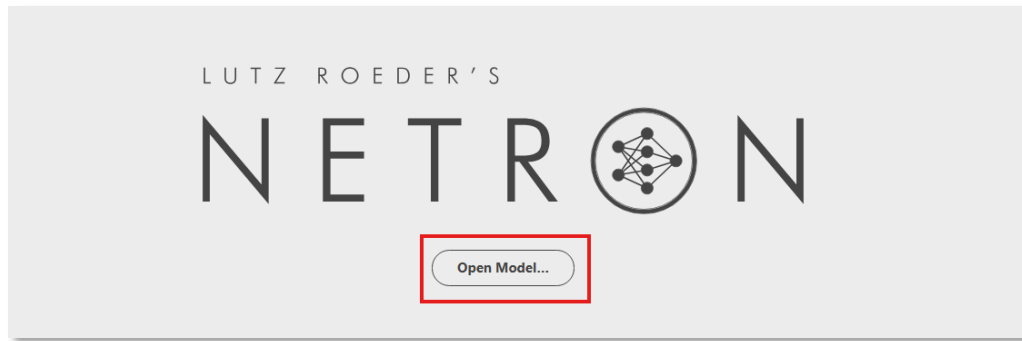
Netron is a utility that takes in an ONNX file and produces the model as a flow chart/graph.



9.1 Review the model using Netron

9.1.1 Go to the website Netron using the following link <https://netron.app>

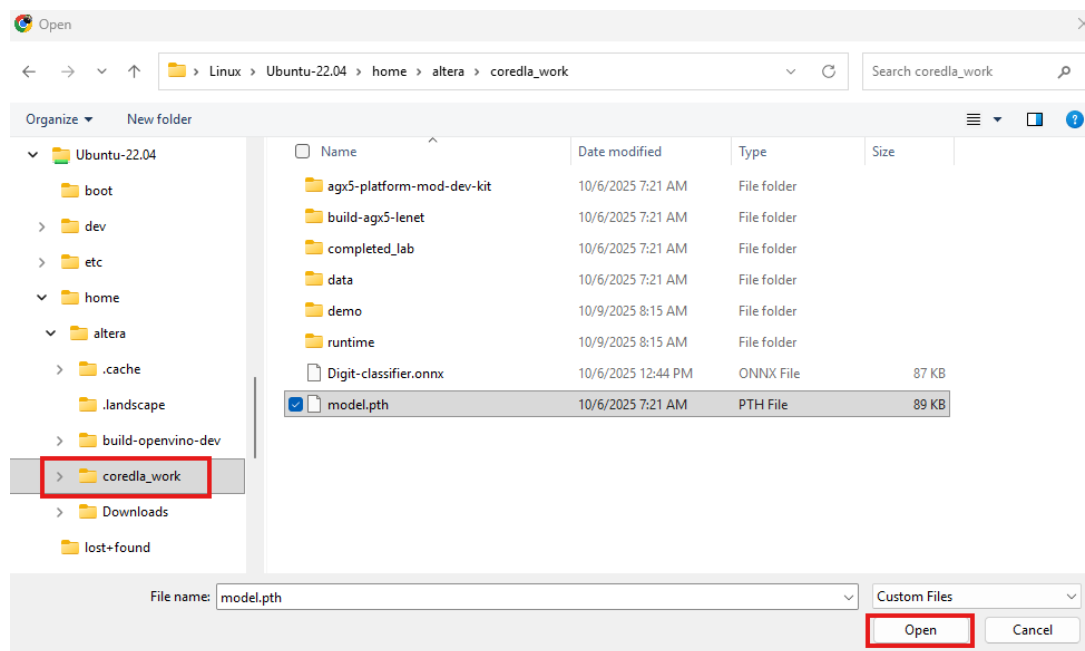
9.1.2 Click on Open Model.



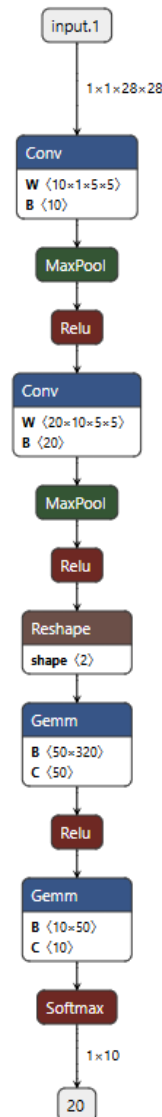
9.1.3 Navigate to the ONNX file at

\\wsl.localhost\Ubuntu-22.04\home\altera\coredla_work

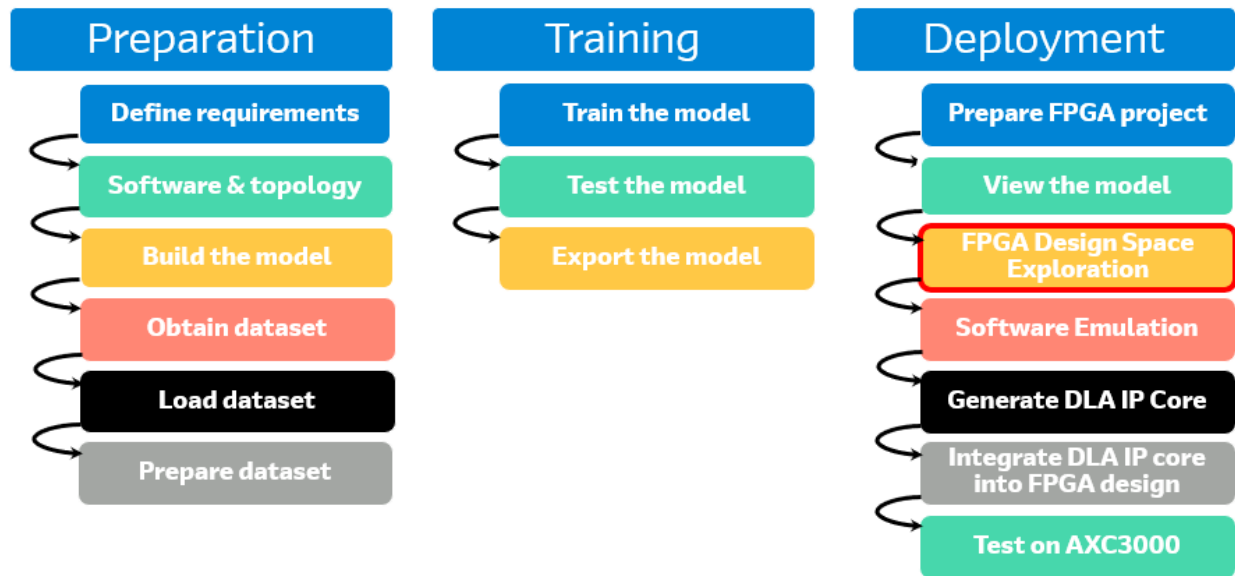
9.1.4 Select Digit-classifier.onnx



The graph showing the LeNet-5 model is as follows. . Notice how it matches the model. Click on each block for more details on its input and output formats, data types, strides and kernel shapes. You should be able to see the weights and bias from this graph. For example, if you select the first Conv (in blue color) there is a of W (Weights) and B (Bias) array dimensions are shown in the flow diagram.

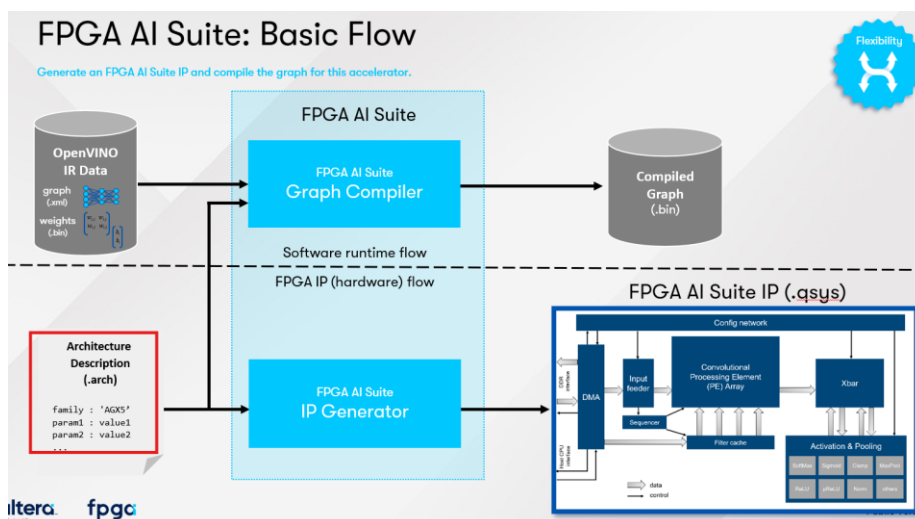


10 FPGA Design Space Exploration



10.1 Architecture files

Before the OpenVino intermediate files can be used in FPGA AI Suite, an architecture file is needed. An architecture file, as seen in the following red box, describes various parameters that are used to define how the AI Suite IP is created.



The .arch file contains many parameters including:

- hardware: Target family only. In this case, it will be Agilex 3.
- architecture i.e. number of processing elements (Convolutional blocks), enable/disable activation functions (i.e. for non-linearity) and pooling functions
- on-chip memory for filter weights and other values.
- numerical precision (i.e. integer 8 or floating point as fp11, fp12, fp13 or fp16) which in turn affects the accuracy of the inference
- external memory bus bandwidth
- and more

This .arch file will be used to generate custom, optimized IP that will be in turn be used on the hardware.

10.2 Example Designs

The question might be: Are there reference architecture files and design examples that could be used to get started?

Yes, there are various complete reference designs per FPGA family with different system configurations installed with the FPGA AI Suite. aa snippet of some of these can be seen below. The ones in the red box are for Agilex 5 (but these can also be targeted at Agilex 3 devices).

```
$ dla_build_example_design.py list
```

Identifier	Name	Quartus
a10_soc_m2m	Arria 10 Memory-to-Memory SoC Example Design	25.1
a10_soc_m2m_preset	Arria 10 Memory-to-Memory SoC Board Aware Example Design	25.1
a10_soc_s2m	Arria 10 Streaming-to-Memory SoC Example Design	25.1
a10_soc_s2m_preset	Arria 10 Streaming-to-Memory SoC Board Aware Example Design	25.1
agx5_soc_m2m	Agilex 5 Memory-to-Memory SoC Example Design	25.1
agx5_soc_s2m	Agilex 5 Streaming-to-Memory SoC Example Design	25.1
agx5e_modular_jtag	Agilex 5 Hostless JTAG Example Design	25.1
agx7_de10_pcie	Agilex 7 PCIe-Attached Example Design	25.1*
agx7_iseries_ddrfree	Agilex 7 Hostless DDR-Free Example Design	25.1*
agx7_iseries_ofs_pcie	Agilex 7 PCIe-Attached Example Design with OFS	24.3
agx7_n6001_ofs_pcie	Agilex 7 PCIe-Attached Example Design with OFS	24.3
agx7_soc_m2m	Agilex 7 Memory-to-Memory SoC Example Design	25.1
agx7_soc_m2m_preset	Agilex 7 Memory-to-Memory SoC Board Aware Example Design	25.1
agx7_soc_s2m	Agilex 7 Streaming-to-Memory SoC Example Design	25.1
agx7_soc_s2m_preset	Agilex 7 Streaming-to-Memory SoC Board Aware Example Design	25.1

*Can be compiled with newer versions of Quartus.

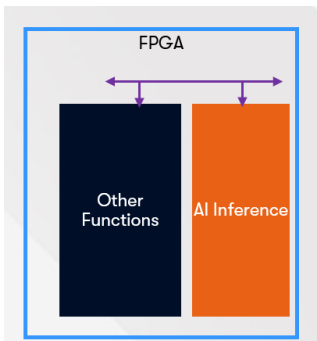
The FPGA Suite can be configured to be run in different combinations such as host with DDR or hostless DDR-free. In this case, the Agilex 3 design will be run as hostless DDR-free. **Hostless** means that rather than using an host processor, the inference execution operations will be run within the FPGA without a CPU.

DDR-free means there is no external DDR memory connected to the device. System Console will instead store the image features, via JTAG, on the on-chip memory. The scatter gather DMA will be stream the image features into the AI IP and stream the results from the AI IP. With no external memory, the weights, biases and execution instructions will be stored in memory blocks internal to the AI IP.

Some benefits of hostless DDR-free include:

- faster performance and less latency because data is stored internally in the device, rather than going off the device.
- simplify the design because the connections to the processor, configuration of the processor, connections to the external memory and configuration of the memory are not needed

An image of DDRfree with the AI inference can be seen below



10.3 Agilex 3 architecture

10.3.1 Birds eye view image and components

This is a representation of an Agilex 3 device



Within this configurable digital device, there are building blocks for fabric (used for combinational functions), memory (used for storage), clocks, and DSP (used for math functions). The HSIO, HVIO, LPDDR4 blocks are used to interface to external devices.

10.3.2 Agilex 3 family table.

The Agilex 3 device family is comprised of a variety of devices seen in the table below. The part that is used on AXC3000 development board is the 2nd largest Agilex 3 device, thus allowing considerable flexibility in implementing designs.

There is vertical migration of this specific device to the equivalent Agilex 5 device. Vertical migration means that by only changing the device, the board can remain the same. Agilex 5 devices, for example, were released before Agilex 3, so before the Agilex 3 AXC3000 board was released, testing could be done by targeting an Agilex 5 device.

Product Line	A3C025 ¹	A3C050	A3C065	A3C100	A3C135
Specification Options ²	Y, Z	Y, Z	Y, Z	W, Y, Z	W, Y, Z
Logic elements (LEs)	25,075	47,200	65,490	100,300	135,110
Adaptive logic modules (ALMs)	8,500	16,000	22,200	34,000	45,800
ALM registers	34,000	64,000	88,800	136,000	183,200
M20K memory blocks	65	123	169	262	353
M20K memory size (Mb)	1.27	2.40	3.30	5.12	6.89
MLAB memory count	450	800	1,050	2,000	2,300
MLAB memory size (Mb)	0.27	0.49	0.64	1.22	1.40
I/O PLL	2	2	2	4	4
Fabric-feeding IOPLL ³	5	5	5	8	8
Variable-precision digital signal processing (DSP) blocks	34	65	88	138	184
18 x 19 multipliers	68	130	176	276	368
Peak INT8 (TOPS)	0.47	0.90	1.21	1.90	2.54
LVDS pairs at 1.25 Gbps	48	48	48	96	96
LPDDR4 interfaces (x32)	0	1	1	2	2
MIPI D-PHY interface	0	7	7	14	14
Differential (RX or TX) pairs at 12.5 Gbps	0	0	0	4	4
PCIe [®] 3.0 x4 instance	0	0	0	1	1
High-speed I/O (HSIO)	96	96	96	192	192
High-voltage I/O (HVIO)	160	160	160	200	200
Secure Device Manager (SDM)	Provides SHA-384 bitstream integrity, ECDSA 256/384 bitstream authentication, AES-256 bitstream encryption, physically unclonable function (PUF) protected key storage, side-channel attack resistance, SPDMA attestation, cryptographic services, and physical anti-tamper support				
Hard processor system (HPS)	NA			Multi-core with 32-bit/64-bit dual-core Arm Cortex [®] -A55 processor up to 800 MHz with 32 KB I/D cache and 128 KB L2 cache, and up to 2 MB L3 cache, multi-channels direct memory access (DMA), 512 KB on-chip RAM, USB 3.1 x1, USB 2.0 OTG x2, TSN MAC x3, UART x2, SPI M x2, SPI S x2, I2C x2, I2C x5, NAND x1, SDMMC x1, oscillator timer x2, SP timer x2, watchdog x5, and GPIO x2.	
Transceiver	NA			PCIe hard IP up to PCIe 3.0 x4 endpoint (EP) and root port (RP) Transceiver channel count: up to 4 channels at 12.5 Gbps (NRZ) Ethernet IP: up to 4 x10 GbE hard IP (MAC, PCS, and FEC)	

Notes:

1. A3C025 does not support EMIF and MIPI.

2. Specification code definition:

Code	HPS	Crypto	EMIF, MIPI, PUF, SPDMA Attestation
W	Yes	Yes	Yes
Y	No	No	Yes
Z	No	No	No

EMIF: External Memory Interface
MIPI: Mobile Industry Processor Interface
PUF: Physically Unclonable Function
SPDMA: Security Protocol and Data Model

3. The fabric-feeding IOPLL count including system PLL at the transceiver bank. The system PLL can be repurposed for core fabric usage if not used for the transceiver.

1

10.3.3 DSP block architecture -TensorFlow mode and FP16 efficiency.

AI is very math intensive, and as such, the inference device must be able to support the math functionality. Agilex 5 and Agilex 3 devices (the target for this workshop here) can support this with the built in DSP functionality.

Within the Agilex 3 devices, the variable procession DSP blocks can support floating point (single precision FP32 and half precision 16-bit arithmetic FP16 and FP18), fixed point (9 x 9 to 54 x 54) and AI tensor mode.

Before describing Tensor Mode, let's first review what is fixed point and floating point.

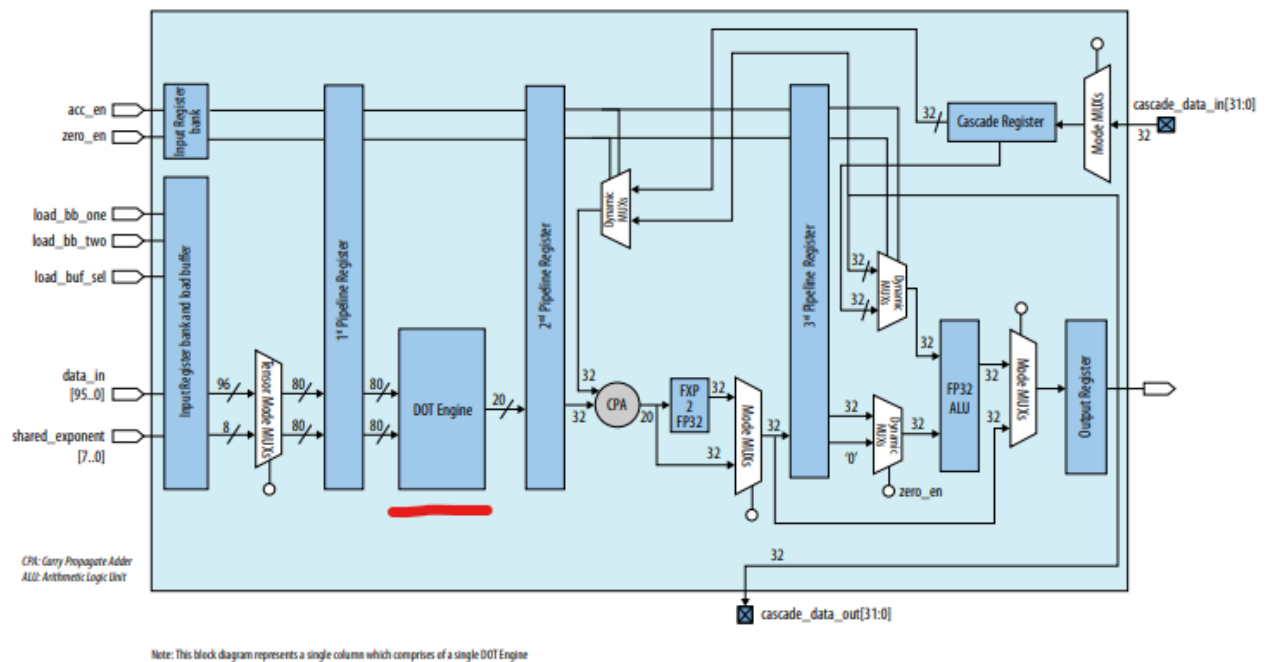
Fixed point and floating can be used for many applications such as basic math operations (such as addition, multiply and accumulation, Fixed point runs faster, uses less resources but has less precision than floating point. Floating point, on the other hand, represents more numbers than fixed point.

Just for a refresher, an example of a fixed point number with a precision of 3 can be 50.250 where the integer is 50 and the fractional part, after the decimal point, is 250.

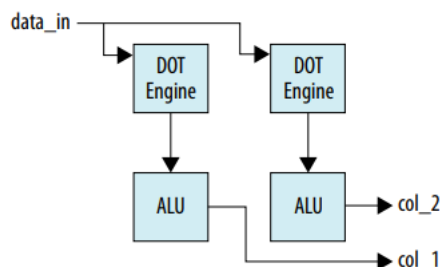
An example of a floating point can be 10exp6 which equals 1,000,000. Floating point formats use a base number (mantissa) and an exponent. Floating point can be used for very small or very large numbers because the exponents can replace the many values

in front or behind the decimal number. For example, the numbers 500×20 will be represented in floating point as $(5.0 \times 10^{\text{Exp } 2}) \times (2.0 \times 10^{\text{Exp } 1})$. The mantissas are 5.0 and 2.0 while the exponents are Exp 2 and Exp 1. Floating point multiplication multiplies the mantissa together and adds the exponents. In this example, the mantissa 5×2 (equals 10) and then the exponents are added together. $(\text{Exp } 1 + \text{Exp } 2) = \text{Exp } 3$. This results in $10 \times 10^{\text{Exp } 3} = 10^{\text{Exp } 4}$ (which equals 10,000)

Now, let's review Tensor Mode. Tensor Mode makes use of the matrix-matrix or vector-matrix multiplications and can be used for math intensive AI calculations. The Tensor Mode supports floating, fixed mode and accumulator (where the dot engine is bypassed). In Tensor mode, the DSP block uses a dot engine (dot product) as seen below. AI uses dot product, for example, when it performs convolution.



The dot engine when zoomed in more, but still seen at a high level looks like:

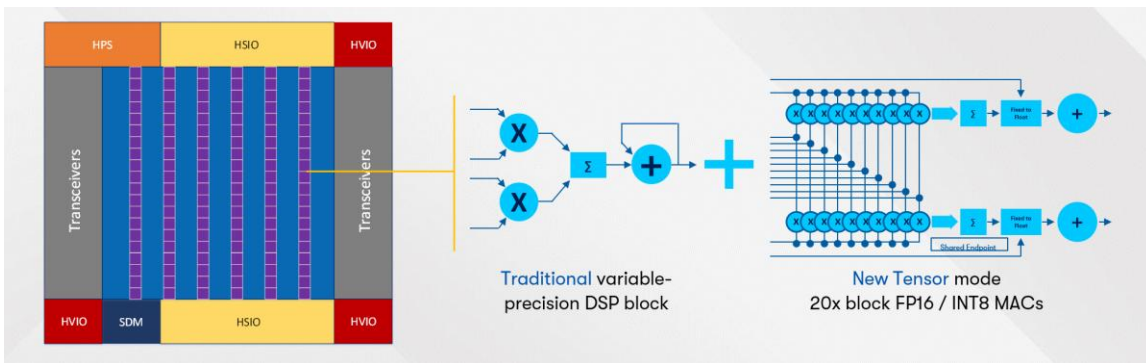


This means that the Agilex 3 DSP blocks can be configured to support 2 sums of ten Int8*Int8 multiplier tensor fixed point and floating-point computation. An example of dot vector is $(3, 4, 5) * (6, 7, 8) = 3*6+4*7+5*8=86$.

The Agilex 3 devices, which are the first power and cost optimized FPGAs infused with DSP and AI, have

- integrated AI compute with flexible FPGA logic
- focus on matrix/vector-mult pattern
- introduce new block FP16 format maximizing power/area efficiency
- completely backward compatible with existing FPGA DSP

This is represented by the diagram below



Earlier, it mentioned the new block FP16 format. Although this floating-point precision is less common than variations such as fp12 and it can increase resources, "it can improve accuracy especially for models that are not retrained for low precision"

10.3.4 Identify resources available for device on AXC3000

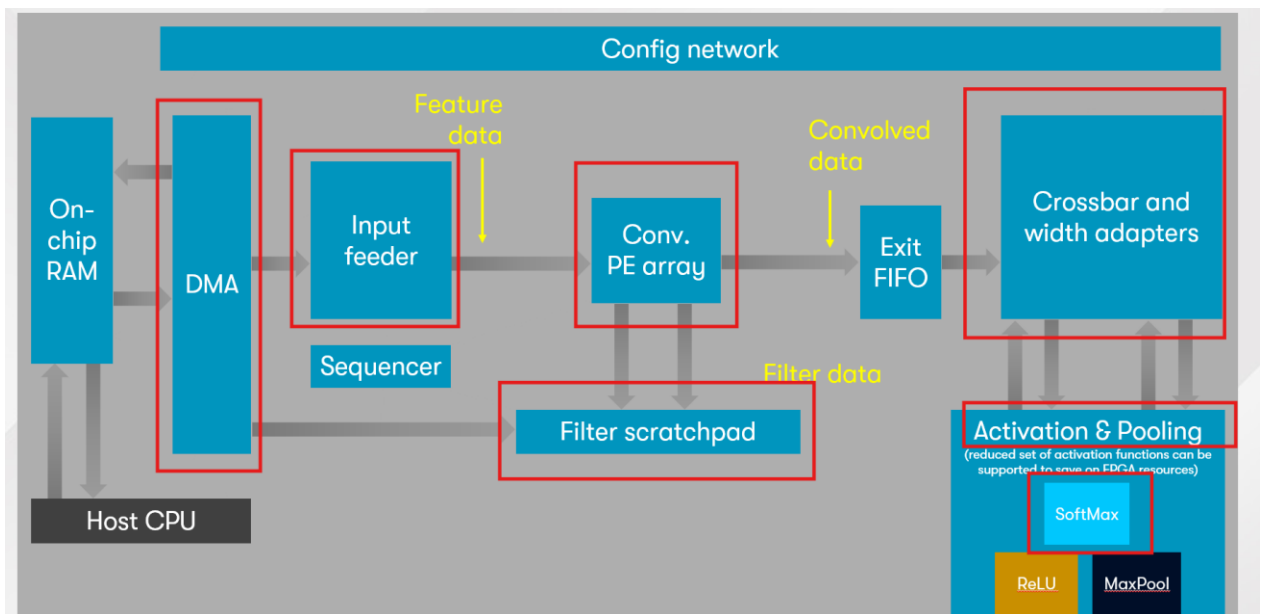
The AXC3000 dev board which is used for this workshop is pictured below:



The AXC3000 has an FPGA (Agilex 3) device, a HyperRAM, a QSPI configuration flash, 3-axis accelerometer, RGB LED, push buttons, DIP switch, CRUVI HS connector, and on-board programmer/debugger. The dev board allows quick evaluation of the FPGA.

10.4 Examine Agilex 3 performance .arch

Before describing the .arch file in detail, a top-level graphic of FPGA AI Suite IP is shown below. The instantiation of the various blocks are influenced by the settings in the .arch file.



10.4.1 Description of the .arch file. The .arch file consists of various parameters, where the following chart will provide details for each one

Parameter	What does it mean?
Family: 'AGX5'	Refers to the family that is being used. Even though the device being used is an Agilex 3, the Agilex 5 device is targeted. The architecture between the two devices is the same. There is also no AGX3 option.
K_vector : 64	The number of PE (processing elements) in the processing array. This number is a multiple of c_vector. This is the convolution engine.
C_vector- 32	Size of the dot product performed by each PE in the PE Array. Dot product occurs when you do convolution of 2 matrixes, where there is first the multiplication and then the addition of the numbers
Arch_precision: FP12AGX	Precision of the features and weights in the PE. There are different options for floating-point.

Stream_buffer_depth : 32768	On chip cache for the feature image. The larger the values, the more the logic and block memory will increase but the performance will increase.
Output_channels_max:16384	
enable_debug : false	Toggles the FPGA AI Suite debug network to allow forwarding of read requests from the CSR (control and status register) to one of the many externally attached debug modules. This is generally not required for production
enable_eltwise_mult : true	Enables the Elementwise multiplication layer. An example of elementwise would be $[2\ 3] * [6, 7]$ equals $2*6 = 12$ and $3*7 = 21$ thus the matrix would be $[12, 21]$
filter_size_width_max : 28	Maximum (width) size of the convolution filter.
filter_size_height_max : 28	Maximum (height) size of the convolution filter.
pe_array { num_interleaved_features : 12 num_interleaved_filters : 1 exit_fifo_depth : 1024 enable_scale: true }	This defines the processing element array, where the PE Array is used to calculate the dot products. "To support the layers with bias values, the PE array uses a threaded accumulator that is time multiplexed to handle multiple accumulations. Each accumulation corresponds to an output filter and feature. For Agilex 5 (thus Agilex 3), the num_interleaved features is set to 12, and the number of num_interleave filter needs to be 1. For Agilex 5, the number of num_interleaved_features times num_interleaved_filters must be greater to or equal to 12. The exit_fifo_depth is the depth of the PE Array exit FIFO. When this depth has large values, it may reduce the incidence of stalling but at the cost of area. Typically, the setting for the fifo_depth is not changed
filter_scratchpad { filter_depth : 512 bias_scale_depth : 512 }	The filter_scratchpad "parameters define the on-chip cache used for the filter weights, bias values, and scale values (if scale and bias are enabled)." The filter_depth "defines the size of the on-chip cache used to store convolution filter weights. Larger values use more FPGA resources but might increase fps" The architecture optimizer is typically used for the filter_depth number. The bias_scale_depth "parameter defines the size of the on-chip cache used to store feature bias and scale weights, if the corresponding support is enabled in the PE Array. Larger values use more FPGA resources but might increase inference throughput."
dma { csr_addr_width : 11 csr_data_bytes : 4 ddr_addr_width : 32 ddr_burst_width : 4 ddr_data_bytes : 32 ddr_read_id_width : 2 }	Defines the attributes that are connected to the external AXI interface
activation { generic_aux_parameters { k_vector : 32 } enable_clamp : false enable_round_clamp : true enable_leaky_relu : false	Defines what activation features to be enabled in this case, the enable_round_clamp, enable_sigmoid and enable_prelu are the ones that are set to true.

<pre> enable_sigmoid : true enable_prelu : true } </pre>	
<pre> pool { generic_aux_parameters { k_vector: 4 } max_window_height: 13 max_window_width: 13 max_stride_vertical: 4 max_stride_horizontal: 4 } </pre>	Defines the downsampling that will be used. There are different types of pooling, and this module defines the max pool. The parameters of the max_window_height and width define the max dimension. The stride refers to the “steps” that that pool will move across.
<pre> xbar { xbar_k_vector : 32 max_input_interfaces : 5 max_output_interfaces : 5 xbar_ports { xbar_aux_port { name: 'activation' input_connection : 'xbar_in_port' } xbar_aux_port { name: 'pool' input_connection : 'xbar_in_port' input_connection : 'activation' } } } </pre>	This defines a crossbar that connects the components of the PE array, auxiliary modules, input feeder (holding data for the next conv layer), and output.
<pre> custom_aux_primitive { generic_aux_parameters { k_vector : 1 } layer_type : 'softmax' name : 'softmax' type : ELTWISE_WITH_MEMORY max_num_channels : 2048 } </pre>	To enable the softmax, the parameter of custom_aux_primitive is needed
<pre> config_network { modules { module { name: 'filter_reader' cross_clock : 1 } module { name: 'input_feeder_mux' } } ... } </pre>	“These parameters define the configuration network that connects internal components within the IP. Typically, this section is changed only minimally when modifying the examples in the example_architectures/ directory.”

10.4.2 Enter the following command in the Ubuntu shell

```

$ dla_compiler \
--march $COREDLA_ROOT/example_architectures/AGX5_Performance.arch \
--network-file ./Digit-classifier.xml \

```

```
--foutput-format=open_vino_hetero \
--o digit-classifier-bin.bin \
--dumpdir work \
--fanalyze-performance --fanalyze-area
```

The command specifies that the **AGX5_Performance.arch** file will determine the build parameters. The output format (open_vino_hetero) will be a binary file. The **dumpdir** specifies that the output directory is **work**. The **fanalyze** performance and area means that performance and the area of the device will be analyzed and reported.

Note: When starting to work with FPGA AI Suite, an example reference design for that device family should be used first, as was done in this case. This will help provide a first estimate of the fps and the resources.

10.4.3 Review performance and resource usage

The screenshot of the results can be seen below.

```
main_graph_0 reported throughput: 18559.5 fps
TOTAL DDR SPACE REQUIRED = 0.333984 MB
  DDR INPUT & OUTPUT BUFFER SIZE = 0.0483398 MB
  DDR CONFIG BUFFER SIZE = 0.00488281 MB
  DDR FILTER BUFFER SIZE = 0.280762 MB
  DDR INTERMEDIATE BUFFER SIZE = 0 MB
NOTE: THIS ESTIMATE ASSUMES 1x I/O BUFFER. THE COREDLA RUNTIME DEFAULTS TO 5
TOTAL DDR TRANSFERS REQUIRED = 0.338013 MB
  DDR FILTER READS REQUIRED = 0.285156 MB
  DDR FEATURE READS REQUIRED = 0.0478516 MB
  DDR FEATURE WRITES REQUIRED = 0.00012207 MB
NUMBER OF DDR FEATURE READS = 1
MINIMUM AVERAGE DDR BANDWIDTH REQUIRED = 6273.35 MB/s
ASSUMED DDR BANDWIDTH PER IP INSTANCE = 6400 MB/s
-----
Performance Estimator Throughput Breakdown
Arch: kvec64xcvec32_i12x1_fp12agx_sb32768_xbark32_actk32_poolk4
Number of DLA instances = 1
Number of DDR Banks per DLA instance = 1
CoreDLA Target Fmax = 350 MHz
PE Target Fmax = 350 MHz
Batch Size = 1
PE-only Conv Throughput No DDR = 20833 fps
PE-only Conv Throughput = 20833 fps
Overall Throughput Inf PE Buf Depth (zero MPBW) = 19781 fps
Overall Throughput Zero PE Buf Depth (zero MPBW) = 19849 fps
Overall Throughput Inf PE Buf Depth = 19478 fps
Overall Throughput Zero PE Buf Depth = 19700 fps
-----
FINAL THROUGHPUT = 18559.5 fps
FINAL THROUGHPUT PER FMAX (CoreDLA) = 53.0272 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 53.0272 fps/MHz
Executing area estimate
Estimated area:
ALMs: 74222
ALUTs: 57384
Registers: 187891
DSPs: 266
M20Ks: 1162
Memory ALMs: 1960
M20K Width/Depth Breakdown:
(40,512): 546
(10,2048): 516
DSP Configuration: Tensor FP with ChainOut
```

Since this is the performance reference design, note in particular, the fps and the number of resources as seen below.

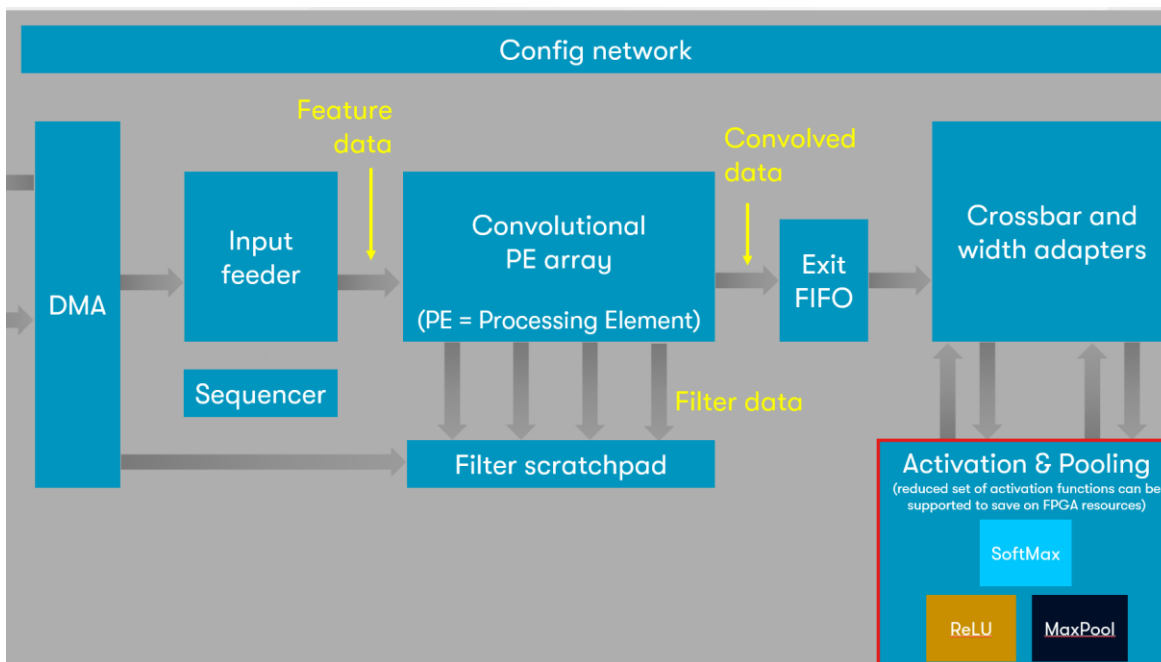
```
FINAL THROUGHPUT = 18559.5 fps
FINAL THROUGHPUT PER FMAX (CoreDLA) = 53.0272 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 53.0272 fps/MHz
Executing area estimate
Estimated area:
ALMs: 74222
ALUTs: 57384
Registers: 187891
DSPs: 266
M20Ks: 1162
Memory ALMs: 1960
M20K Width/Depth Breakdown:
(40,512): 546
(10,2048): 516
DSP Configuration: Tensor FP with ChainOut
```

This means the performance is very fast i.e. 18,559.5 fps

It is important to note is that the reference design uses many ALMs more than the Agilex 3 device on the dev board has. The Agilex 3 has 34,000 ALMs vs the result of 74,222 ALMs.

The Question might be: What can you do to reduce the amount of logic resources to have it fit into an Agilex 3 device? The .arch file can be customized with different settings. One example of this is that in the latest version of Quartus 2025.1.1, there is a public beta version of a lightweight layout transform. This lightweight transform is an area-optimized version of the hardware layout transform, which was released in the FPGA AI Suite 2024.3.

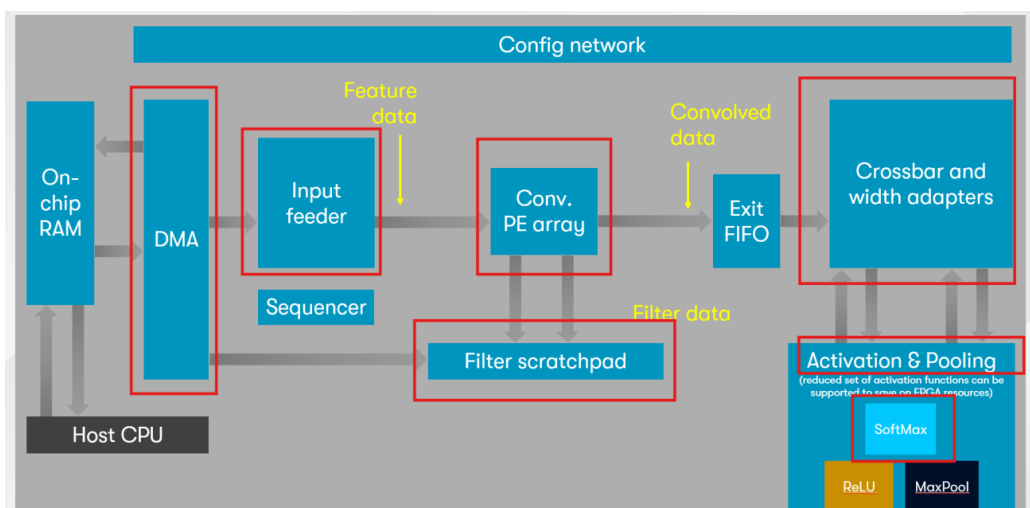
Overall, with different architecture settings, the FPGA AI Suite provides a tradeoff between features and performance. The IP can be scaled as large or small. As an example, if all activation blocks (shown below) are selected, this will mean there could be less space for the convolution array.



In the following step, using this new .arch file (and with other parameters in the FPGA AI Suite) the number of resources will be dramatically reduced to fit an Agilex 3 device instead.

10.5 Examine Agilex 3 Streaming Ddrfree softmax.arch

A top-level block diagram of DDR-free can be seen below. The red blocks refer to the features in the following .arch file that are defined. Various blocks will be configured differently than with the previously used Performance.arch file. This will allow the Agilex 3 to be targeted.



10.5.1 Description of parameters in Agilex 3 Streaming Dddrfree Softmax.arch file..

The following details explain the different parameters within AGX5-Streaming_Ddrfree_Softmax.arch file, where the .arch file defines the structure of the neural network.

Ddrfree, for reference, means that there is no external DDR connected to the device. The lines in the file with # at the starting of the line are comments only.

Parameter	What does it mean?
Family: 'AGX5'	Refers to the family that is being used. Even though the device being used is an Agilex 3, the Agilex 5 device is targeted. The architecture between the two devices is the same. There is also no AGX3 option.
K_vector : 4	The number of PE (processing elements) in the processing array. The smallest number is 4, which is what is being used here. This number is a multiple of c_vector. This is the convolution engine.
C_vector- 4	Size of the dot product performed by each PE in the PE Array. The number 4 is the smallest unit that is used here. Dot product occurs when you do convolution of 2 matrixes, where there is first the multiplication and then the addition of the numbers
Arch_precision: FP11	Precision of the features and weights in the PE. There are different options for floating point, but fp11 is the smallest fp.
Stream_buffer_depth : 2048	On chip cache for the feature image. The larger the values, the more the logic and block memory will increase but the performance will increase.
enable_parameter_rom : true	Enables storing graph parameters in on-chip memory, which requires input and output streaming to be enabled and configured. Since this is Ddrfree, this option is set to true
enable_debug : false	Toggles the FPGA AI Suite debug network to allow forwarding of read requests from the CSR (control and status register) to one of the many externally attached debug modules. This is generally not required for production
enable_eltwise_mult : true	Enables the Elementwise multiplication layer. An example of elementwise would be $\begin{bmatrix} 2 & 3 \end{bmatrix} * \begin{bmatrix} 6 & 7 \end{bmatrix}$ equals $2*6 = 12$ and $3*7 = 21$ thus the matrix would be $\begin{bmatrix} 12, 21 \end{bmatrix}$
filter_size_width_max : 16	Maximum (width) size of the convolution filter.
filter_size_height_max : 16	Maximum (height) size of the convolution filter.
input_stream_interface { enable: true data_width: 128 fifo_depth : 128	Defines the incoming data stream. The AXI-4 stream bus width is set to 128. The buffer size between the source data and the processing element is set to 128.

}	
<pre>output_stream_interface { enable: true data_width : 128 fifo_depth : 128 }</pre>	Defines the outgoing data stream. The AXI-4 stream bus width is set to 128. The buffer size between the processing element and output dat is set to 128.
<pre>enable_lightweight_layout_transform : true # 96 => 12 pixels in parallel lightweight_layout_transform_params { channels: 1 input_bus_width : 128 input_type : F16 enable_bias_scale : false }</pre>	For Agilex 3, the layout transform is set to lightweight layout, because this is area-optimized version of the hardware layout transform, which was introduced in the FPGA AI Suite 2024.3. The public beta version of this transform was released for Quartus 2025.1.1. Other device families such as Agilex 5 which have more resources, the non-lightweight transform can be used. The bus width is set the same as the input and output stream interface.
<pre>pe_array { num_interleaved_features : 12 num_interleaved_filters : 1 exit_fifo_depth : 512 }</pre>	This defines the processing element array, where the PE Array is used to calculate the dot products. “To support the layers with bias values, the PE array uses a threaded accumulator that is time multiplexed to handle multiple accumulations. Each accumulation corresponds to an output filter and feature. For Agilex 5 (thus Agilex 3), the num_interleaved_features are set to 12, and the number of num_interleave filter needs to be 1. For Agilex 5, the number of num_interleaved_features times num_interleaved filters must be greater to or equal to 12. The exit_fifo_depth is the depth of the PE Array exit FiFO. When this depth has large values, it may reduce the incidence of stalling but at the cost of area. Typically, the setting for the fifo_depth is not changed
<pre>filter_scratchpad { filter_depth : 1532 bias_scale_depth : 26 }</pre>	The filter_scratchpad “parameters define the on-chip cache used for the filter weights, bias values, and scale values (if scale and bias are enabled).” The filter_depth “defines the size of the on-chip cache used to store convolution filter weights. Larger values use more FPGA resources but might increase fps” The architecture optimizer is typically used for the filter_depth number. The bias_scale_depth “parameter defines the size of the on-chip cache used to store feature bias and scale weights, if the corresponding support is enabled in the PE Array. Larger values use more FPGA resources but might increase inference throughput.”
<pre>dma { csr_addr_width : 11 csr_data_bytes : 4 ddr_addr_width : 32 ddr_burst_width : 4 ddr_data_bytes : 64 ddr_read_id_width : 2 }</pre>	Defines the attributes that are connected to the external AXI interface
<pre>activation { generic_aux_parameters { k_vector : 4 } enable_clamp : false enable_leaky_relu : false</pre>	Defines what activation features to be enabled in this case, the enable_relu is the only one that is set to true, thus the non-linear rectified linear unit will be enabled

<pre> enable_sigmoid : false enable_relu : true } </pre>	
<pre> pool { generic_aux_parameters { k_vector: 4 } max_window_height: 3 max_window_width: 3 max_stride_vertical: 4 max_stride_horizontal: 4 } </pre>	Defines the downsampling that will be used. There are different types of pooling, and this module defines the max pool. The parameters of the max_window_height and width define the max dimension. The stride refers to the “steps” that that pool will move across.
<pre> xbar { xbar_k_vector : 4 max_input_interfaces : 5 max_output_interfaces : 5 xbar_ports { xbar_aux_port { name: 'activation' input_connection : 'xbar_in_port' } xbar_aux_port { name: "pool" input_connection: "xbar_in_port" input_connection: "activation" } } } </pre>	This defines a crossbar that connects the components of the PE array, auxiliary modules, input feeder (holding data for the next conv layer), and output.
<pre> custom_aux_primitive { generic_aux_parameters { k_vector : 1 } layer_type : 'softmax' name : 'softmax' type : ELTWISE_WITH_MEMORY max_num_channels : 16 } </pre>	To enable the softmax, the parameter of custom_aux_primitive is needed
<pre> config_network { config_cache_depth : 641 modules { module { name: 'filter_reader' cross_clock : 1... } } } </pre>	” These parameters define the configuration network that connects internal components within the IP. Typically, this section is changed only minimally when modifying the examples in the example_architectures/ directory.”

10.5.2 Enter the following command in the Ubuntu shell

```

dla_compiler \
--march ./AGX5_Streaming_Ddrfree_Softmax.arch \
--network-file ./Digit-classifier.xml \
--foutput-format=open_vino_hetero \

```

```
--o digit-classifier-bin.bin \
--fplugin=HETERO:FPGA --fanalyze-performance --fanalyze-area \
--ffolding-option 0 \
--dumpdir work
```

This command tells the dla compiler to use the new, custom .arch file (AGX5_Streaming_DDRfree_softmax.arch) with the OpenVino network topology file (digit-classifier.xml). The output will be a bin file.

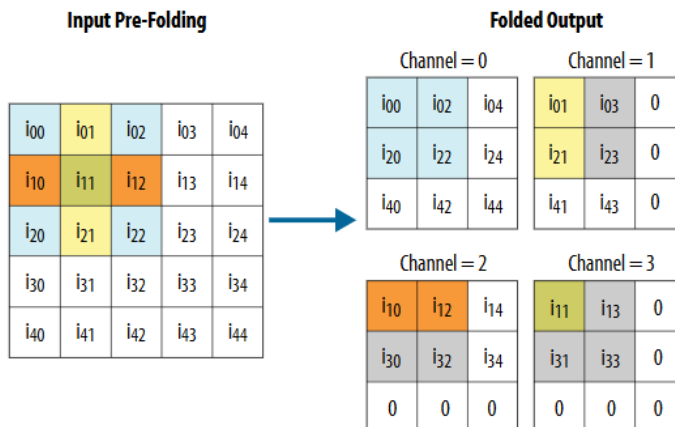
The **-fplugin=Hetero:FPGA option** means that it will try to use the FPGA for all the inference operations. Should there be layers that the FPGA does not support, it will report an error. It will analyze both performance and area.

FFolding option of 0 means that there will be no folding applied by the convolution of the network.

There might be a question of what does folding mean and why would it be used/not used?

Folding means that the input data is split into smaller (folded) subsets.

An example of folding can illustrate this better. The matrix 5 x 5 on the left is pre-folded, where the filter (colored boxes) is 3*3. Assume that the stride-height and stride-width = 2. This means the smaller (folded) subsets will have the matrixes height and width that are smaller than the original filter (5 x 5) -size of the stride (2 x 2) = 3 x 3 new matrix size.



The colors in the new filters refer to the filters, while the grey boxes correspond to padding for the filters. Now for the reason using/not using folding in the FPGA AI Suite. When no folding is used, this means that the FPGA AI Suite will not be as efficient, but it will be useful for debugging.

10.5.3 Review performance and resource usage. Compare to performance

```
main_graph_0 reported throughput: 4168.01 fps
TOTAL DDR SPACE REQUIRED = 0 MB
  DDR INPUT & OUTPUT BUFFER SIZE = 0 MB
  DDR CONFIG BUFFER SIZE = 0 MB
  DDR FILTER BUFFER SIZE = 0 MB
  DDR INTERMEDIATE BUFFER SIZE = 0 MB
NOTE: THIS ESTIMATE ASSUMES 1x I/O BUFFER. THE COREDLA RUNTIME DEFAULTS TO 5
TOTAL DDR TRANSFERS REQUIRED = 0 MB
  DDR FILTER READS REQUIRED = 0 MB
  DDR FEATURE READS REQUIRED = 0 MB
  DDR FEATURE WRITES REQUIRED = 0 MB
NUMBER OF DDR FEATURE READS = 0
MINIMUM AVERAGE DDR BANDWIDTH REQUIRED = 0 MB/s
ASSUMED DDR BANDWIDTH PER IP INSTANCE = 0 MB/s

-----
Performance Estimator Throughput Breakdown
Arch: kvec4xcvec4_i12x1_fp11_sb2048_xbark4_actk4_poolk4
Number of DLA instances = 1
Number of DDR Banks per DLA instance = 0
CoreDLA Target Fmax = 350 MHz
PE Target Fmax = 350 MHz
Batch Size = 1
PE-only Conv Throughput No DDR = 4176 fps
Overall Throughput Inf PE Buf Depth (zero MPBW) = 4163 fps
Overall Throughput Zero PE Buf Depth (zero MPBW) = 4173 fps
Overall Throughput Inf PE Buf Depth = 4163 fps
Overall Throughput Zero PE Buf Depth = 4173 fps

-----
FINAL THROUGHPUT = 4168.01 fps (this does not consider input and output streamer)
FINAL THROUGHPUT PER FMAX (CoreDLA) = 11.9086 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 11.9086 fps/MHz
Executing area estimate
Estimated area:
ALMs: 10930
ALUTs: 14066
Registers: 31479
DSPs: 16
M20Ks: 117
Memory ALMs: 1150
M20K Width/Depth Breakdown:
(40,512): 8
(10,2048): 18
DSP Configuration: Tensor FP with ChainOut
```

Note specifically from the above screenshot, the performance and resource numbers as seen in the red boxes below

```
FINAL THROUGHPUT = 4168.01 fps (this does not consider input and output streamer)
FINAL THROUGHPUT PER FMAX (CoreDLA) = 11.9086 fps/MHz
FINAL THROUGHPUT PER FMAX (PE) = 11.9086 fps/MHz
Executing area estimate
Estimated area:
ALMs: 10930
ALUTs: 14066
Registers: 31479
DSPs: 16
M20Ks: 117
Memory ALMs: 1150
```

The performance has decreased from 18,559 fps to 4,168 fps. However, the more significant change is the area. The number of the ALMS is now 10,930 which is a significant reduction from 74,222 ALMs. This implementation will now fit into a Agilex 3 device, which has 34,000 ALMs. As an example, you could also see that there are 117 memories being used and the Agilex 3 device has 262 memory blocks.

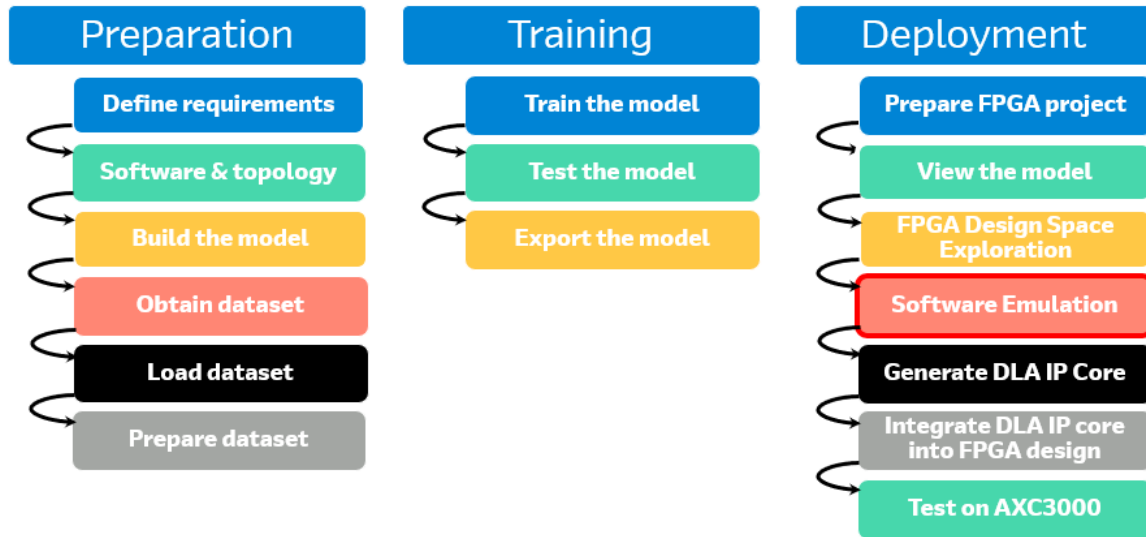
Agilex™ 3 FPGAs and SoCs C-Series Product Table					
altera An Intel Company					
Version 2025.03.12					
Product Line	A3C025 ¹	A3C050	A3C065	A3C100	A3C135
Specification Options ²	Y, Z	Y, Z	Y, Z	W, Y, Z	W, Y, Z
Logic elements (LEs)	25,075	47,200	65,490	100,300	135,110
Adaptive logic modules (ALMs)	8,500	16,000	22,200	34,000	45,800
ALM registers	34,000	64,000	88,800	136,000	183,200
M20K memory blocks	65	123	169	262	353

Overall, using different parameters in the arch file shows performance and area tradeoffs. Using the dla_compiler script means that you can quickly try different parameters and review if area or performance are what you need without doing further, more timing consuming steps to see that the results do not work.

11 Software Emulation using FPGA AI Suite

OpenVINO provides software runtime libraries to measure the accuracy of the model that has been generated

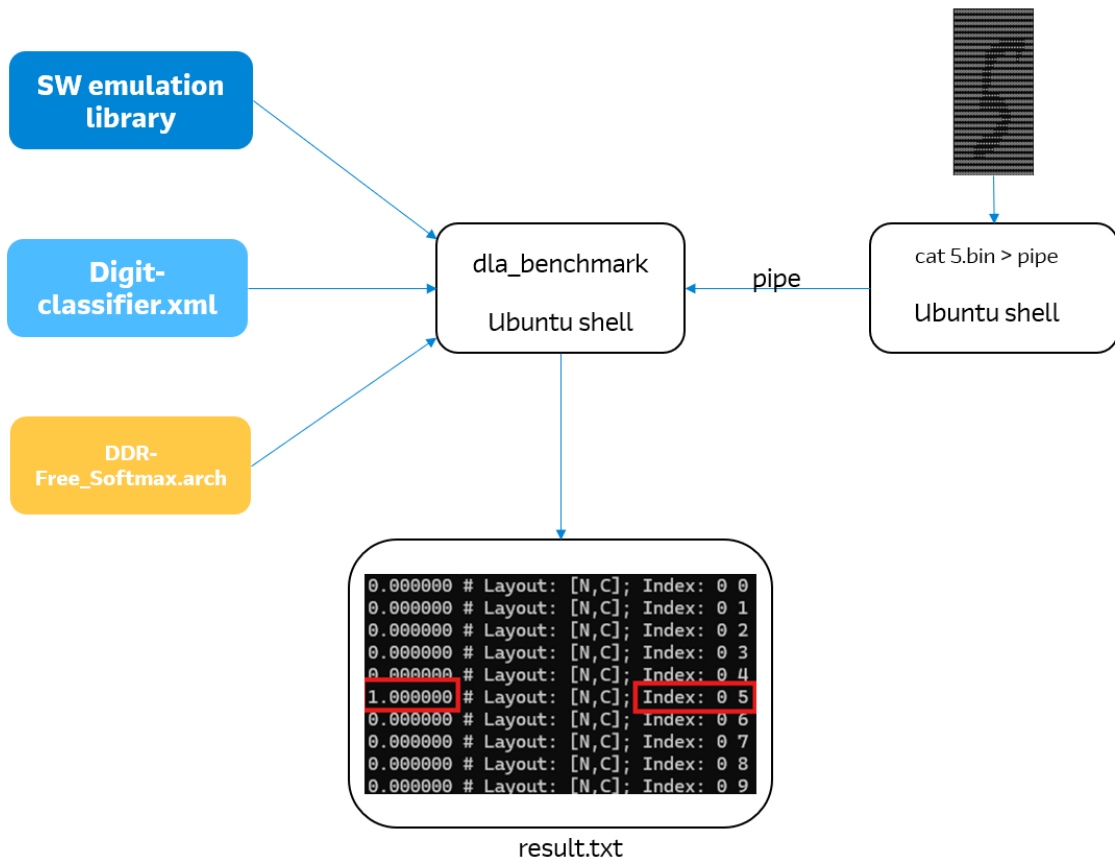
We will use an image of digits extracted from the MNIST database to verify the emulation functionality.



The emulation is a CPU-executed, bit-accurate software model of the FPGA AI Suite IP exposed as an OpenVINO plugin. It allows the user to run the FPGA AI Suite runtime without any FPGA board to validate numerics and pipelines. It is slower than hardware; a typical single-image ResNet-50 run take minutes. Numerics usually match hardware very accurately. It can be accessed from OpenVINO C++ and Python.

In this instance the `dla_benchmark` command line function uses the model xml and the .arch architecture file to represent the AI IP functionality. It emulates the graph with the emulation library. Once invoked it waits for an input image to be streamed using the linux pipe mechanism. The results are written to results.txt

The runtime emulation flow is shown in the diagram below.



```
$ cd $COREDLA_WORK/work
```

11.3 Benchmark the emulation version of the model

11.3.1 `dla_benchmark` is the FPGA AI Suite utility for measuring performance of IR models on a Deep Learning Accelerator (DLA). It waits for an image of a digit to be streamed in and then runs it through the emulated model. The result is written to a file named 'result.txt'

11.3.2 Execute `dla_benchmark`. Enter the following command in the Ubuntu shell.

```
$COREDLA_WORK/runtime/build_Release/dla_benchmark/dla_benchmark \
-b 1 \
-niter 1 \
-nireq 1 \
-m ../Digit-classifier.xml \
-d HETERO:FPGA \
-streaming_input_pipe pipe \
-arch_file ../AGX5_Streaming_Ddrfree_Softmax.arch \
-dump_output \
-folding_option 0 \
-plugins emulation -dump_output
```

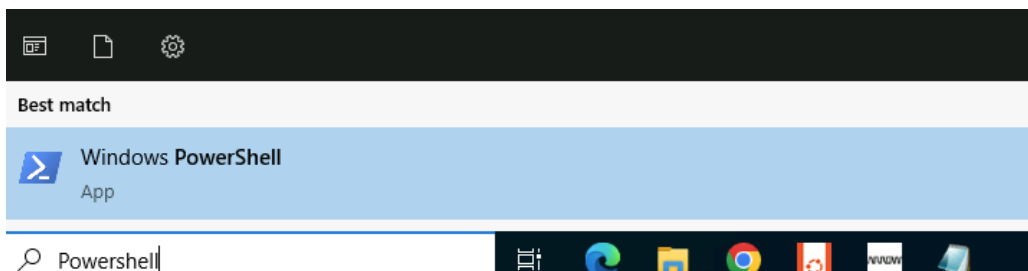
11.3.3 `dla_benchmark` is now running as a process in Ubuntu waiting for bin data input. It is specifically waiting for a file named pipe to stream the image from.

11.3.4 The table below details the parameters specified when running

Command	Explanation
<code>-b 1</code>	Batch size. Batch size of 1 is usually best
<code>-niter 1</code>	Number of images to process in each batch
<code>-nireq 1</code>	Number of simultaneous inference requests that are sent to the FPGA.
<code>-m ../Digit-classifier.xml \</code>	Points to the .xml file. The .bin file has the same name, and it is in the same directory.
<code>-d HETERO:FPGA \</code>	Tell the FPGA is the target

-streaming_input_pipe pipe \	
-arch_file ../AGX5_Streaming_Ddrfree_Softmax.arch \	Points to the .arch file that will be used
-dump_output \	
-folding_option 0 \	Input preprocessing step to reduce the amount of zero padding
-plugins emulation -dump_output	

11.3.5 Open a second Ubuntu Shell. Type 'PowerShell' into the windows search bar. Select Windows PowerShell.



11.3.6 Type 'wsl -d Ubuntu-22.04' in the PowerShell

11.3.7 Ubuntu is now available in the second shell.

11.3.8 Stream the image of the digit

```
$ cd ~/coredla_work/work
```

```
$ cat ../data/array_hwc_fp16_5.bin > pipe
```

11.3.9 Wait for about 5-10 seconds for the process to complete.

```
PASSED
Total time = 203.157 ms
Destructed
This function, GetActiveHWTimeMs, is not implemented for emulator device

[Step 11/12] Dumping statistics report
count:      1 iterations
system duration: 18381.9335 ms
latency:    18381.9242 ms
system throughput: 0.0544 FPS
number of hardware instances: 1
number of network instances: 1
[Step 12/12] Dumping the output values
[ INFO ] Dumping result of main_graph to ./result.txt
[ INFO ] Dumping per-batch result (raw output) of main_graph to ./result_{batch}.bin
[ INFO ] Dumping inference meta data of main_graph to ./result_meta.json
((opencvino_env) ) altera@BQ6BH74:~/coredla_work/work$
```

11.3.10 Review the results.

\$ cat result.txt

```
altera@BQ6BH74:~/coredla_work/work$ cat ../data/array_hwc_fp16_5.bin > pipe
altera@BQ6BH74:~/coredla_work/work$ cat result.txt
0.000000 # Layout: [N,C]; Index: 0 0 start of 20
0.000000 # Layout: [N,C]; Index: 0 1
0.000000 # Layout: [N,C]; Index: 0 2
0.000000 # Layout: [N,C]; Index: 0 3
0.000000 # Layout: [N,C]; Index: 0 4
1.000000 # Layout: [N,C]; Index: 0 5
0.000000 # Layout: [N,C]; Index: 0 6
0.000000 # Layout: [N,C]; Index: 0 7
0.000000 # Layout: [N,C]; Index: 0 8
0.000000 # Layout: [N,C]; Index: 0 9 end of 20, see result_tensor_boundaries.txt for details
```

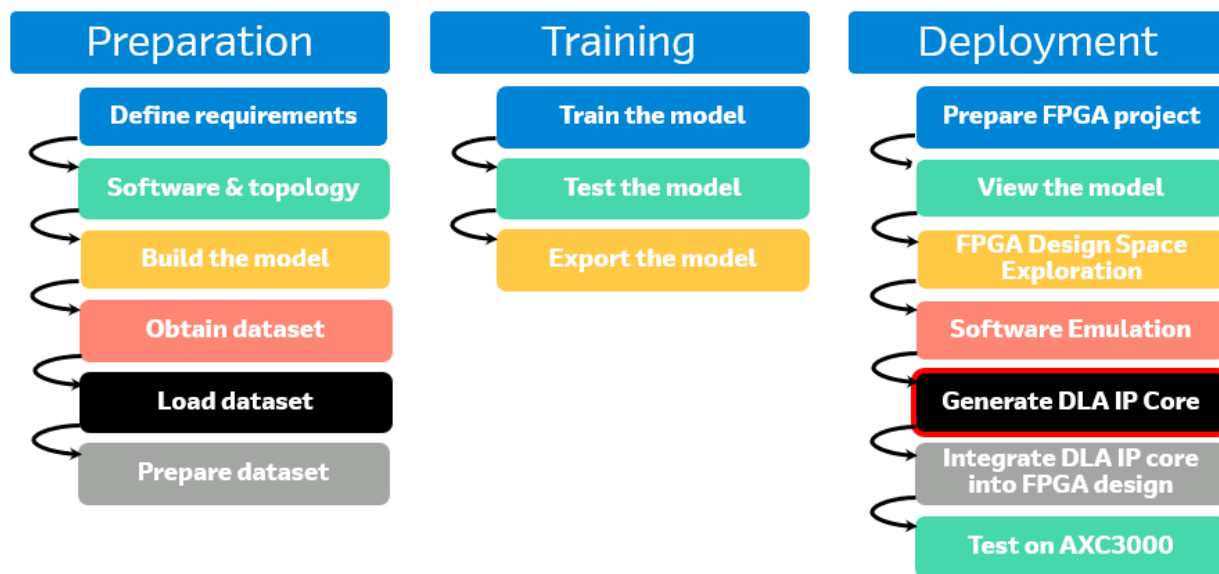
The first highlighted value represents the output of the Softmax function. The second is the category index. This indicates a probability of 100% that the digit is a '5'.

11.3.11 Repeat but stream a different digit (0 for example). Repeat steps 11.3.2 and 11.3.10.

```
altera@BQ6BH74:~/coredla_work/work$ cat result.txt
1.000000 # Layout: [N,C]; Index: 0 0 start of 20
0.000000 # Layout: [N,C]; Index: 0 1
0.000000 # Layout: [N,C]; Index: 0 2
0.000000 # Layout: [N,C]; Index: 0 3
0.000000 # Layout: [N,C]; Index: 0 4
0.000000 # Layout: [N,C]; Index: 0 5
0.000000 # Layout: [N,C]; Index: 0 6
0.000000 # Layout: [N,C]; Index: 0 7
0.000000 # Layout: [N,C]; Index: 0 8
0.000000 # Layout: [N,C]; Index: 0 9 end of 20, see result_tensor_boundaries.txt for details
```

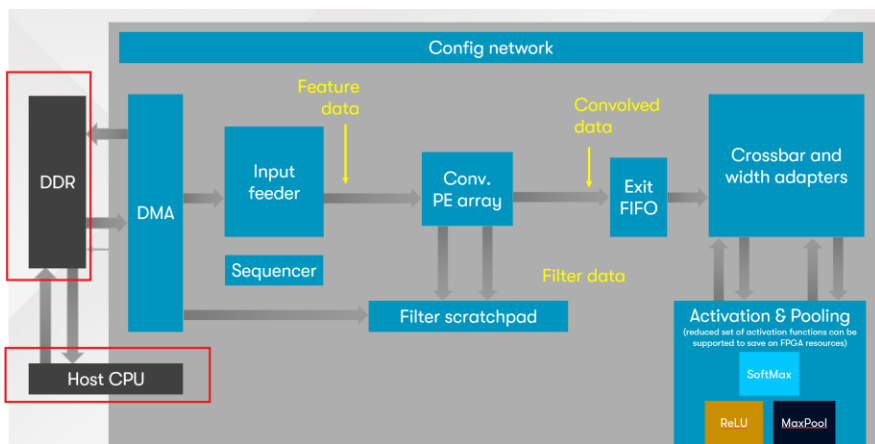
Repeat again, but this time use the digit 8. The results are not good. Look at the digit using step 11.1.2 (but for array_hwc_fp16_8.bin). What happened?

12 Generate DLA IP Core

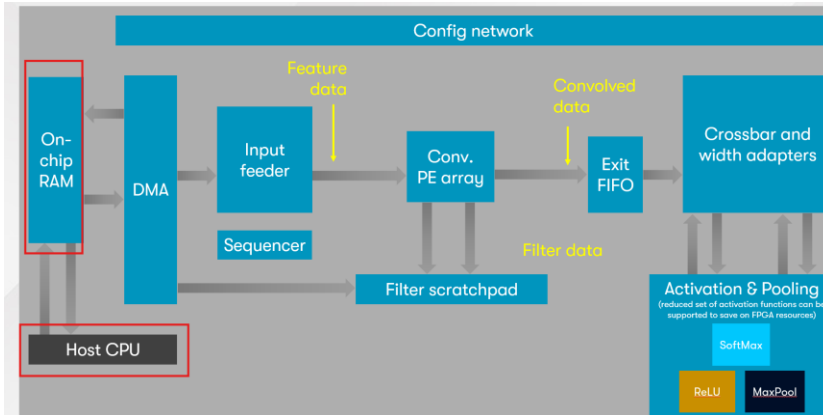


When the DLA IP Core runs it requires access to the graph (model) weights and biases. It also needs to access the configuration instructions. It employs different storage methodologies, for this data, depending on the chosen architecture. When DDR is present this data is stored in a contiguous graph buffer in DDR memory. It is allocated a startup time. It can be dynamically modified by the host. This means that the same DLA IP core can support many different graphs (models) in one FPGA design

In a hostless/ DDR-free configuration, the data is stored in on-chip memory blocks located within the DLA IP core. They are stored as parameter ROMs that are generated by the `dla_compiler`. This architecture cannot easily support different graphs but is much more efficient in its use of Agilex 3 resources. This is illustrated in the diagrams below.



There is an external DDR memory connected to the FPGA, while DDR-free is represented by the diagram below. Note the difference is in the first case, the graph weights are stored externally while in the second case, the graph weights are stored internally.



12.1 Generate the Parameter ROM for the DLA IP Core

THE DLA IP core includes internal Read Only Memory (ROM) memory that will be pre-initialized with the models weights and biases parameter values. It will also include the configuration instructions. These are extracted from the xml intermediate file and written to a binary file.

12.1.1 Generate the Parameter ROM. Enter the following into the Ubuntu shell.

```
$ cd $COREDLA_WORK

$ dlac \
--network-file ./Digit-classifier.xml \
--march ./AGX5_Streaming_Ddrfree_Softmax.arch \
--foutput-format=open_vino_hetero \
--o digit_classifier-bin.bin \
--fplugin HETERO:FPGA \
--dumpdir ./dlac-out/ \
--ffolding-option 0 \
--fanalyze-performance --fanalyze-area
```

12.2 Generate the DLA IP Core

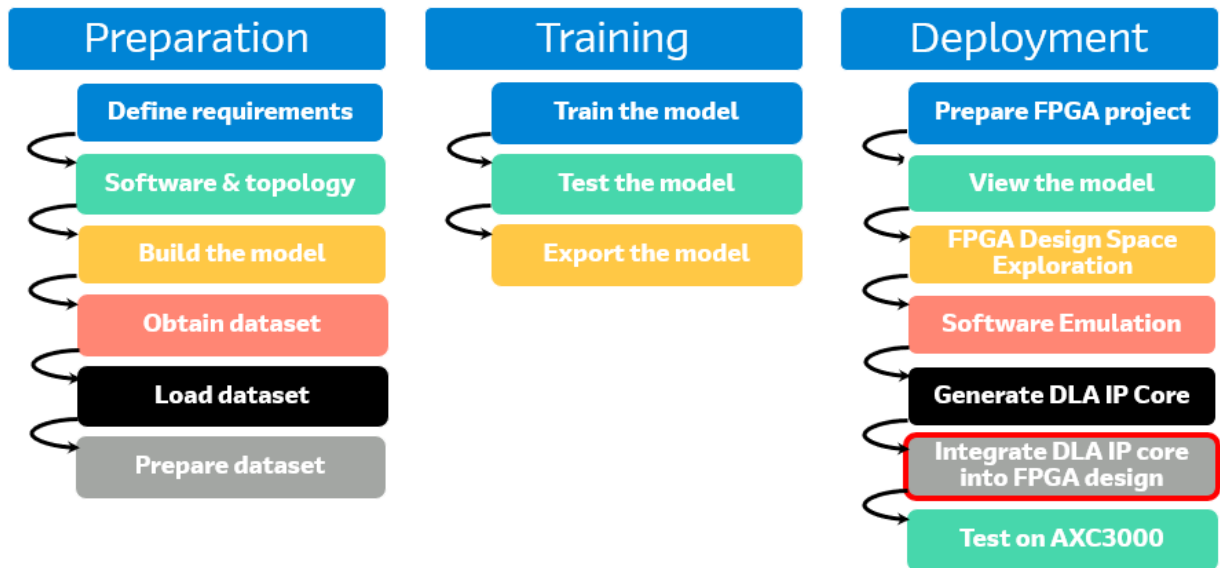
12.2.1 Enter the following into the Ubuntu shell.

```
dla_create_ip \
--flow create_ip \
--arch ./AGX5_Streaming_Ddrfree_Softmax.arch \
--ip_dir ./build-agx5-lenet/coredla_ip \
--parameter_rom_dir ./dlac-out/parameter_rom/ --licensed
```

The generated parameter ROM files are listed below. The are created as Memory Initialization Files (MIF). The files are used to define the initial contents of the parameter ROM on-chip memories in the FPGA at Quartus compile time.

 ddrfree_bias_scale_hw_0.mif	9/7/2025 6:59 PM	MIF File	1 KB
 ddrfree_bias_scale_hw_1.mif	9/7/2025 6:59 PM	MIF File	1 KB
 ddrfree_bias_scale_hw_2.mif	9/7/2025 6:59 PM	MIF File	1 KB
 ddrfree_bias_scale_hw_3.mif	9/7/2025 6:59 PM	MIF File	1 KB
 ddrfree_config.mif	9/7/2025 6:59 PM	MIF File	14 KB
 ddrfree_filter_emu.mif	9/7/2025 6:59 PM	MIF File	316 KB
 ddrfree_filter_hw_0.mif	9/7/2025 6:59 PM	MIF File	62 KB
 ddrfree_filter_hw_1.mif	9/7/2025 6:59 PM	MIF File	62 KB
 ddrfree_filter_hw_2.mif	9/7/2025 6:59 PM	MIF File	62 KB
 ddrfree_filter_hw_3.mif	9/7/2025 6:59 PM	MIF File	62 KB

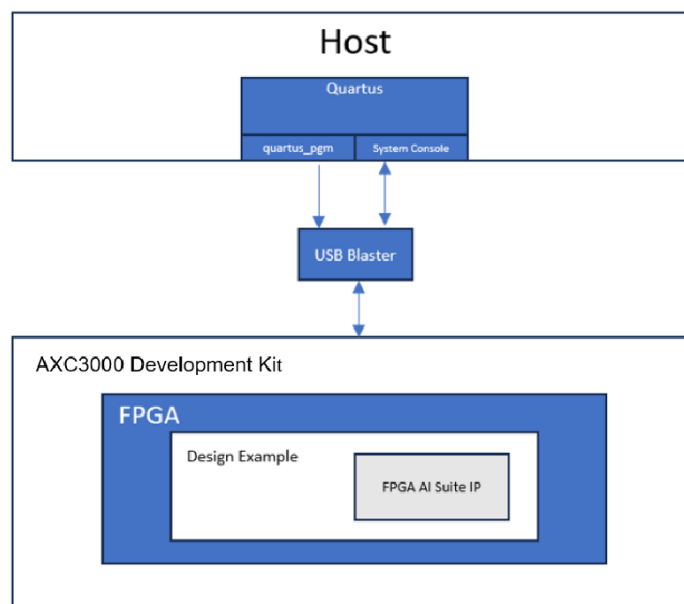
13 Integrate DLA IP Core into FPGA



The FPGA design is based on the Agilex 7 DDR-Free example design. This is a DDR-Free, hostless design. It has the following features

- DDR-Free operation
- Host-less operation (runtime free)
- Streaming of input features
- Streaming of inference results

The design was modified in Quartus to support an Agilex 3 device on the AXC3000 development kit.

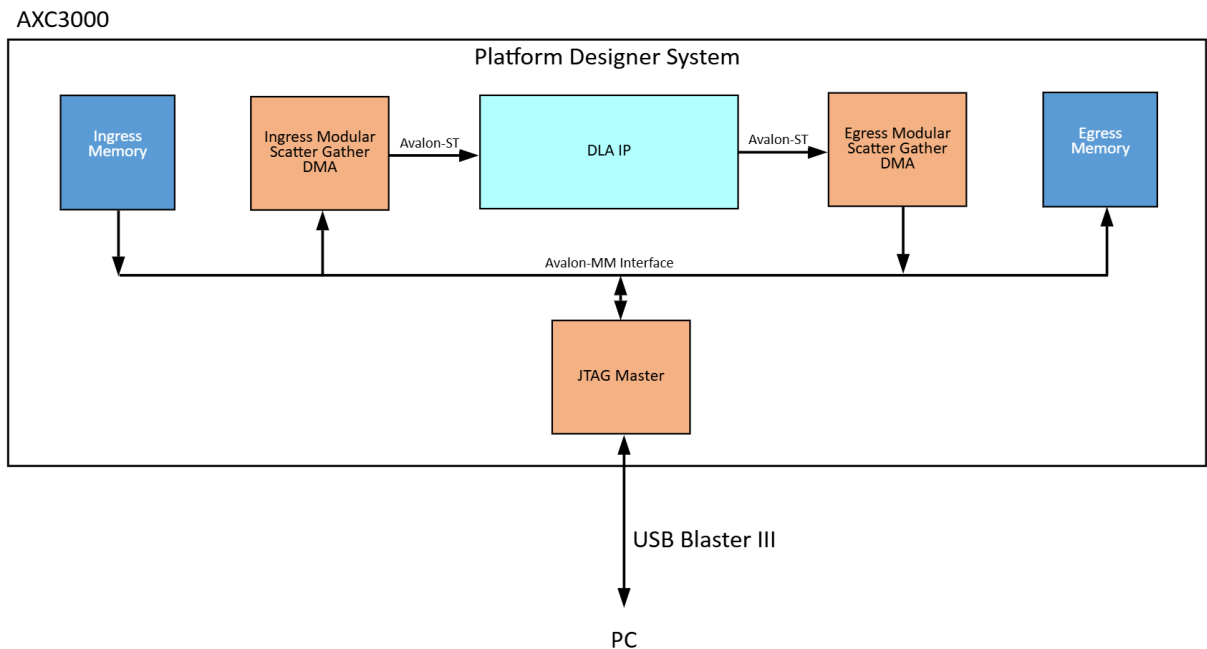


The architecture file was edited to reference the Agilex 3 family of devices. No other changes were required.

This design was chosen to show the AI features of Agilex 3. It utilizes less FPGA resources than the other Hosted designs and fits with ease in the AXC3000 development kit. Later versions of FPGA AI Suite will provide Agilex 3 example designs.

A detailed view of the example design is shown in the following diagram.

Features (images) are downloaded into the Ingress Memory from the PC via the JTAG Master. They are read from memory by the Ingress MSGDMA and streamed into the AI accelerator (DLA IP). Results are streamed from the accelerator into the Egress Memory by the Egress SGDMA. The JTAG Master uploads them to the PC.



13.1 Copy the CoreDLA IP to the example project

13.1.1 `$ cp -R build-agx5-lenet/coredla_ip \`
`/mnt/c/altera_workshops/axc3000/FPGA_AI_Suite_lab/build-agx5-lenet/hw/`

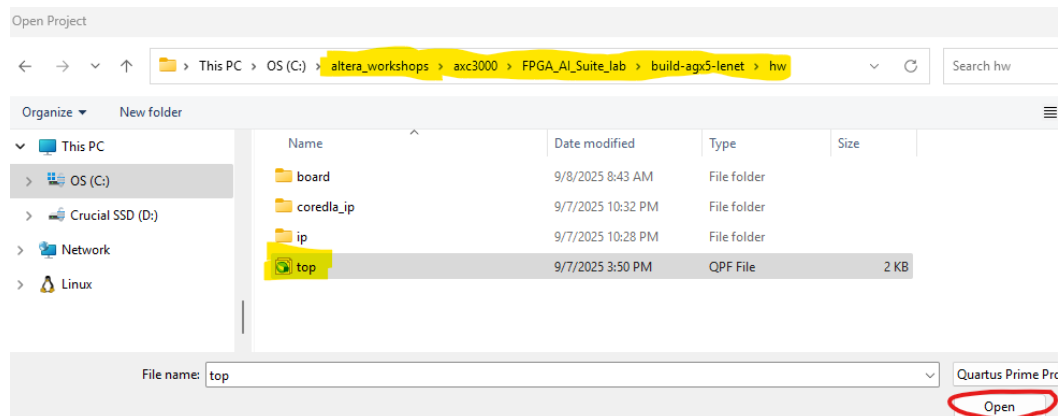
13.2 Open the Quartus project

13.2.1 From the Start menu or the Desktop, **open Quartus Prime Pro 25.1.1** software.

13.2.2 Open the project. File → Open Project

13.2.3 Navigate to `c:\altera_workshops\axc3000\FPGA_AI_Suite_lab\build-agx5-lenet\hw`.

13.2.4 Select `top.qpf`. Click Open.



13.3 Review the Quartus Project

13.3.1 Open the Top-level file

13.4 Review the Platform Designer System

13.4.1 Platform Designer is a system integration tool within Quartus. It enables designers to select IP blocks from its library and connect them to form a functioning system. Once the user has entered the design it will generate the source code that Quartus will compile into an FPGA bitstream.

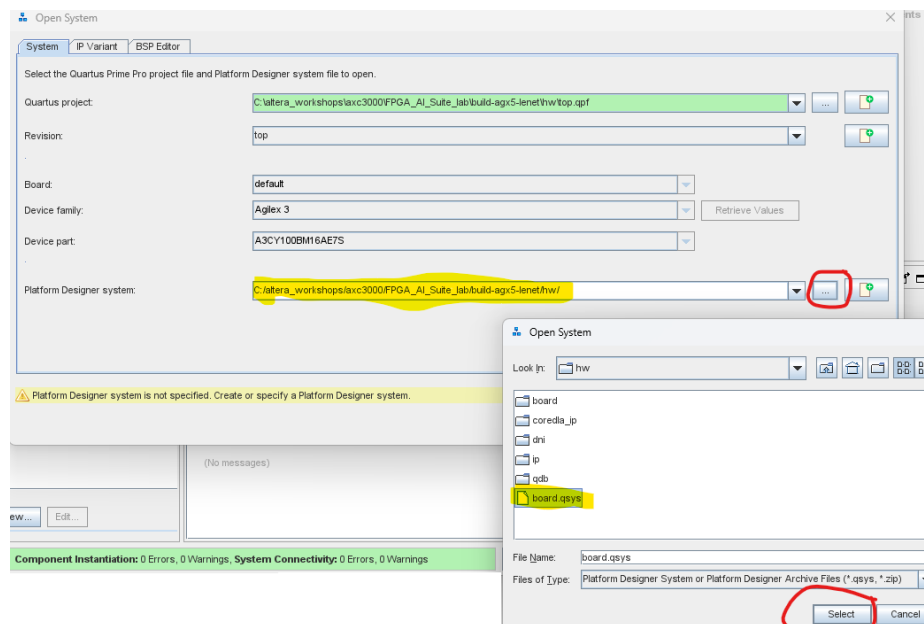
13.4.2 Open Platform Designer. Tools → Platform Designer

13.4.3 The Open System dialogue window will open by default.

13.4.4 Navigate to c:\altera_workshops\axc3000\FPGA_AI_Suite_lab\build-agx5-lenet\hw

13.4.5 Select board.qsys.

13.4.6 Press Select. Press Open. Press Close.

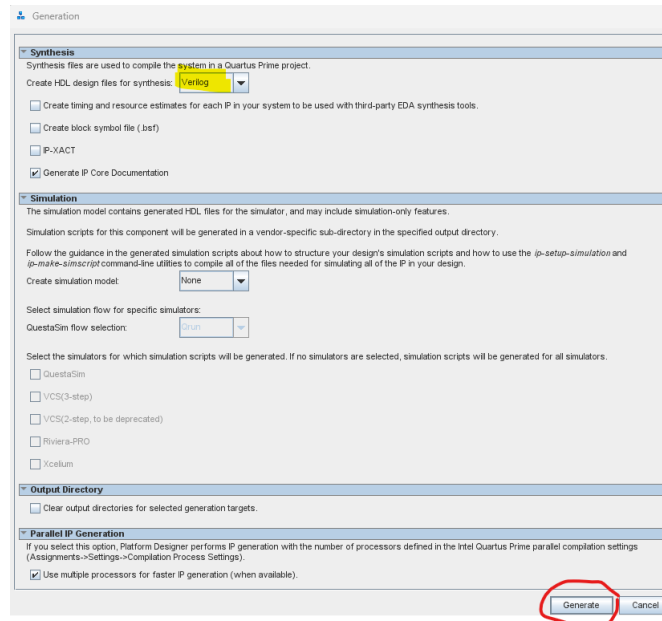


13.5 Generate the system

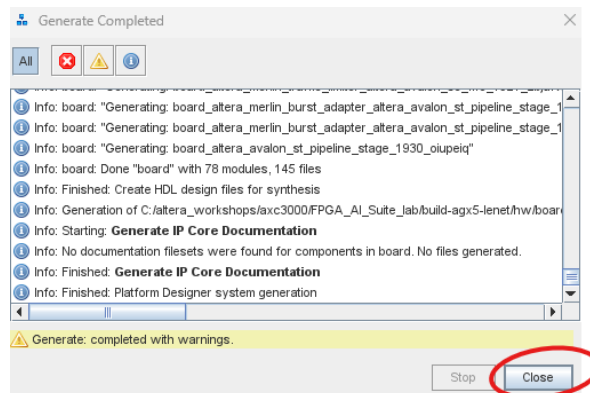
13.5.1 Generate → Generate HDL

13.5.2 Create HDL design files for synthesis should be set to Verilog.

13.5.3 Press Generate. This will take a few minutes while the Verilog code is being generated.



13.5.4 Press Close



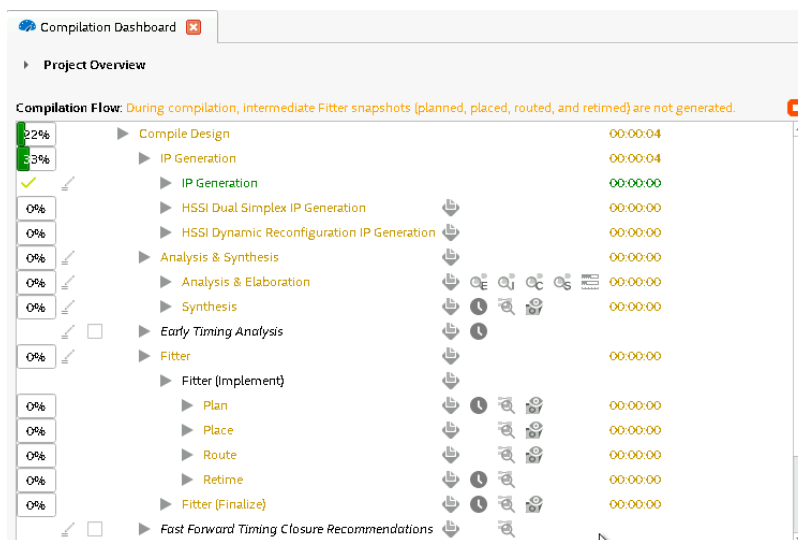
13.5.5 Exit Platform Designer. File → Exit

13.5.6 Compile the design in Quartus.

13.6 Compile the Design

Generate a bitstream for the FPGA that will implement the inference function of the trained LeNet-5 model.

13.6.1 Start the compilation. Processing → Start Compilation

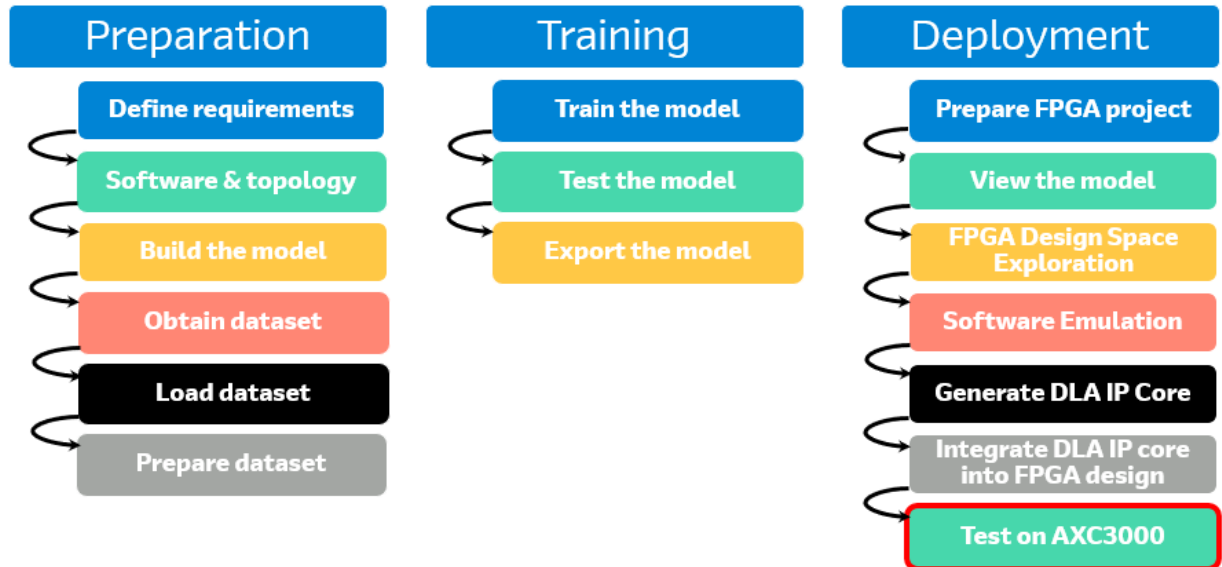


Device utilization is shown in the Fitter Summary below

Fitter Summary	
Filter <<Filter>> (use !<string> to invert filter)	
Fitter Status	Successful - Fri Sep 26 21:32:26 2025
Quartus Prime Version	25.1.1 Build 125 07/31/2025 SC Pro Edition
Revision Name	top
Top-level Entity Name	top
Family	Agilex 3
Device	A3CY100BM16AE7S
Timing Models	Final
Power Models	Final
Device Status	Preliminary
Logic utilization (in ALMs)	10,126 / 34,000 { 30 % }
Total dedicated logic registers	23325
Total pins	1 / 254 { < 1 % }
Total block memory bits	744,130 / 5,365,760 { 14 % }
Total RAM Blocks	85 / 262 { 32 % }
Total DSP Blocks	16 / 138 { 12 % }
Total GTS Transceiver Channels	0 / 4 { 0 % }
Total HSSI Ethernet Channels	0 / 1 { 0 % }
Total HSSI PCIEs	0 / 1 { 0 % }
Total HSSI HPS	0 / 1 { 0 % }
Total PLLs	1 / 4 { 25 % }

14 Test the Design

The final stage is to test the FPGA that contains the AI mode.

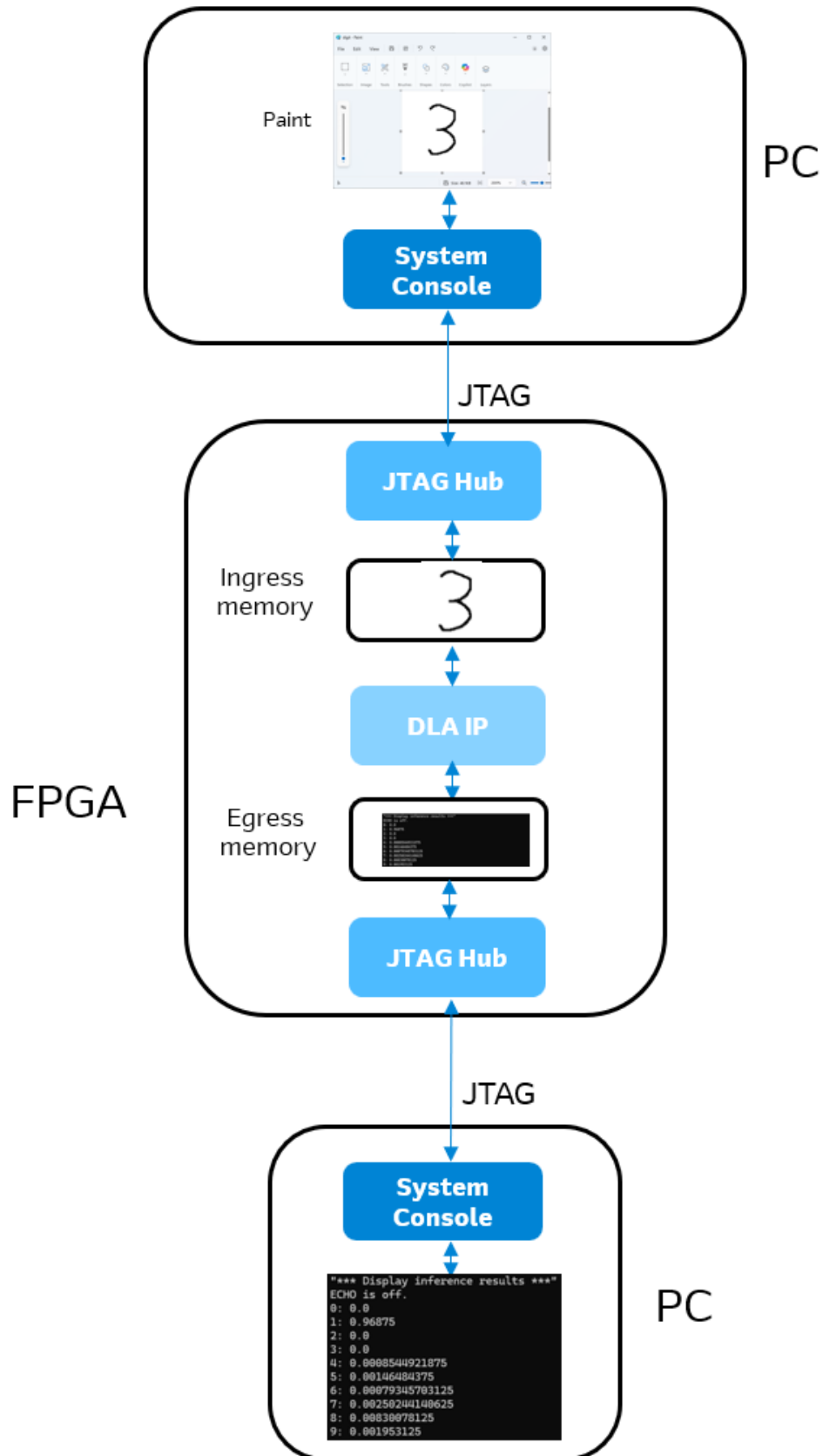


14.1 Test Structure

A block diagram of the test environment is shown below.

Digits are created in Paint and saved as a BMP file. They are converted to a raw binary format, and this is displayed to the user in the command shell. The file is transported via System Console and JTAG to the Ingress memory in the FPGA. It is streamed into the AI accelerator, and the results are streamed out to the Egress memory.

The results are transported back to the PC via JTAG and System Console. They are post processed and then displayed to the user in the command shell.



14.2 Design verification

Use the following guide to determine which verification section to complete

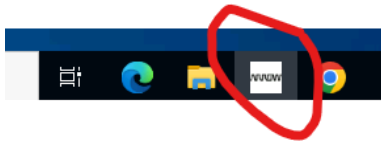
- If the design was compiled with CloudLabs, and will be verified using the Arrow board farm, then complete Section 15.
- If the design was compiled with CloudLabs but will be verified locally, then complete Section 16.
- If the design was compiled locally, and will be verified locally, then complete Section 17.

15 Verify the Design Remotely

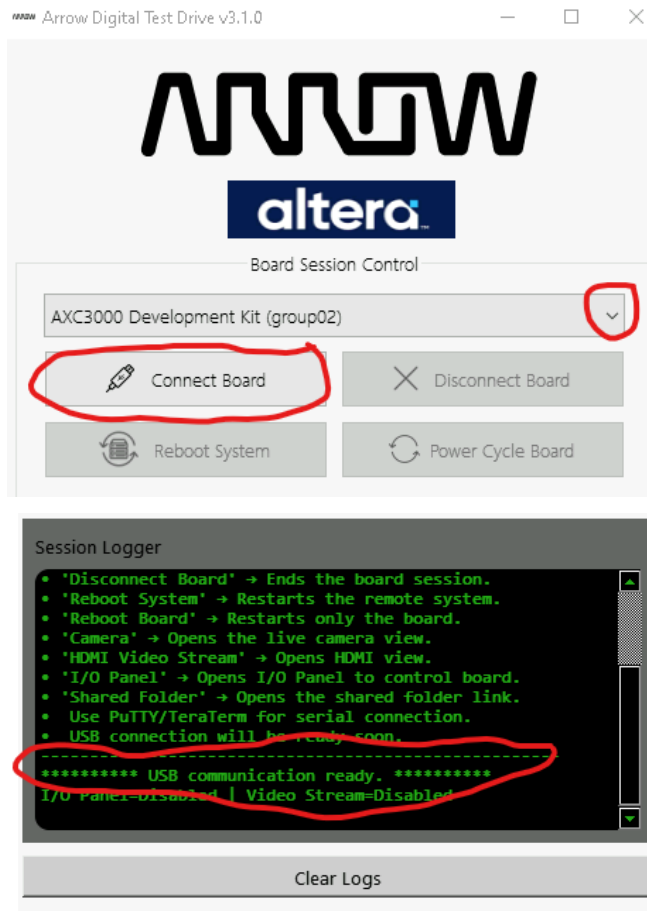
If the design was built using CloudLabs and will be verified using the board farm, then complete this section.

15.1 Connect to the Board Farm

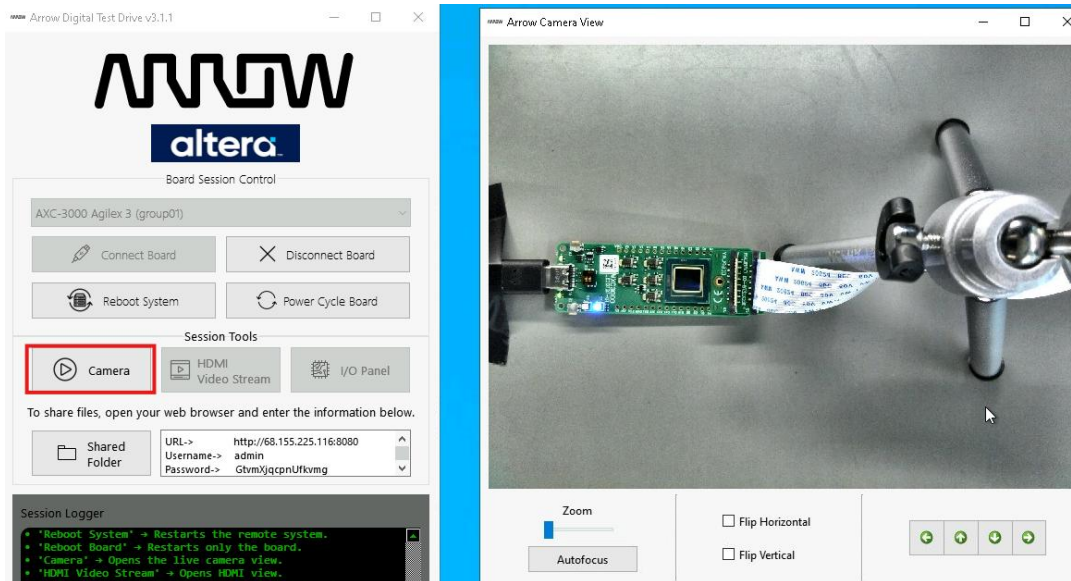
15.1.1 Connect to the remote AXC3000 Board using the ArrowDTD tool. Click on the ArrowDTD icon to launch the tool.



15.1.2 Select an available board from the dropdown menu and click **Connect Board**.



15.1.3 Open the Camera View of the board.

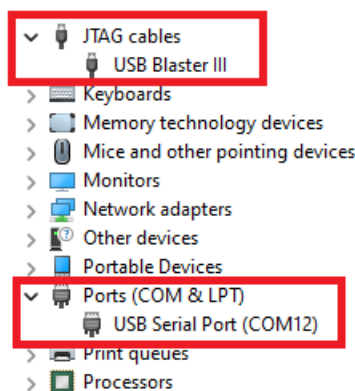


15.2 Program the FPGA

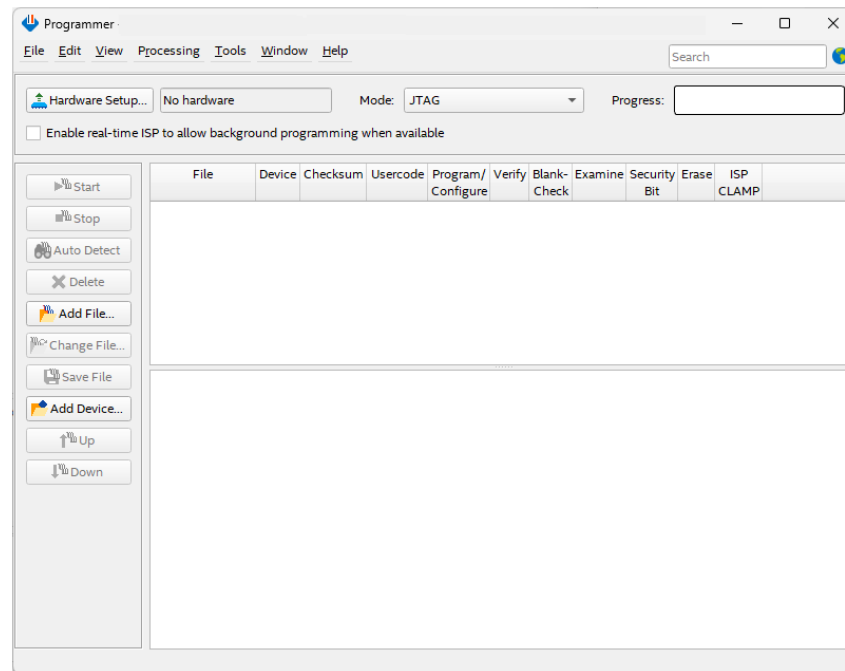
The next step is to program the design into the FPGA on the board. During the Assembler stage of the compilation, a programming file was generated.

15.2.1 Connect your AXC3000 Board to your PC using a USB C cable.

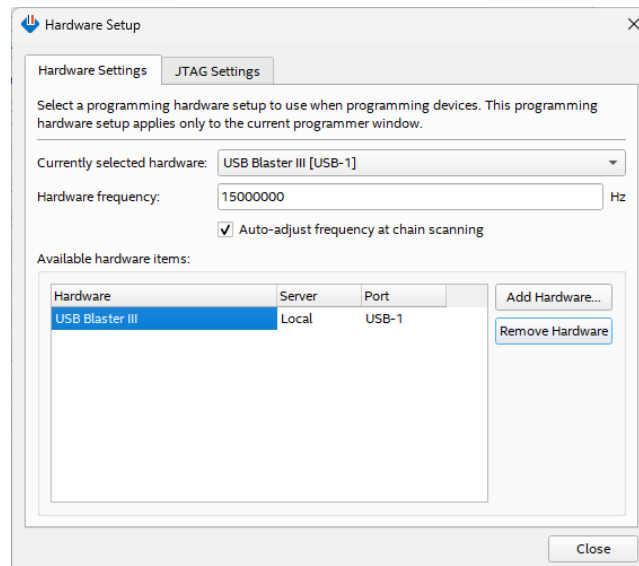
Since the USB Blaster III should be already installed, the Window's Device Manager should display the following entries, highlighted in red (COM port number may differ depending on your PC):



15.2.2 From the Start menu open Altera 25.1.1.125 Pro Edition --> Programmer.

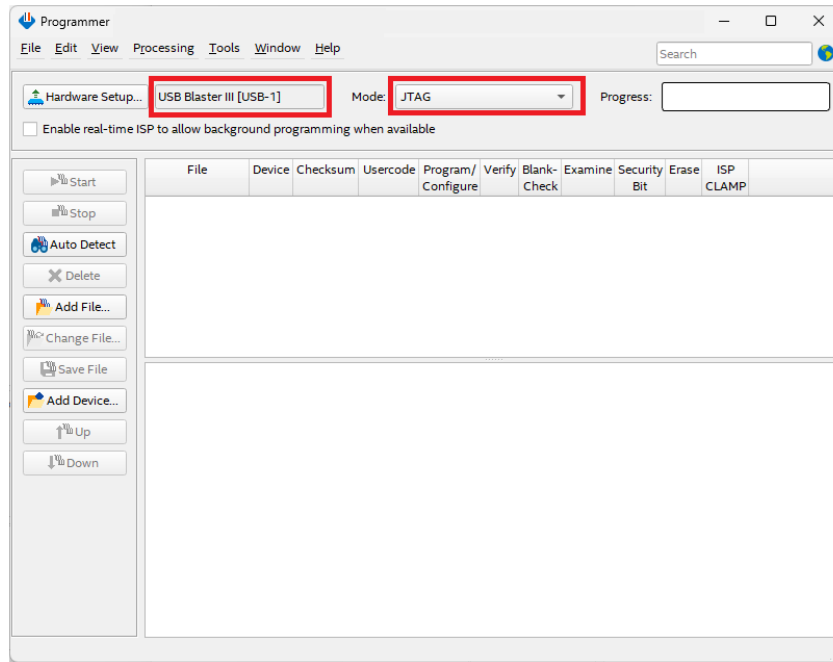


15.2.3 Click **Hardware Setup...** and double click **USB Blaster III** entry in the Hardware Setup window. **Currently selected hardware** should now show USB Blaster III [USB0] (the USB port number may be different depending on your PC).



15.2.4 Click **Close**.

15.2.5 Make sure the hardware setup is Altera-USB-Blaster-III [USB0] and the mode is JTAG. Click **Auto Detect**.



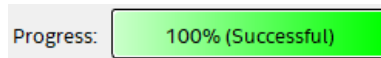
15.2.6 **Double click** the first <none> line item to choose the programming file

15.2.7 The sof file will be populated as a line item. If not, navigate to the **sub folder** output_files, and select the .sof file.

15.2.8 Click **Open**.

15.2.9 Make sure the Programmer shows the correct file and the correct part in the JTAG chain and check the **Program/Configure** checkbox.

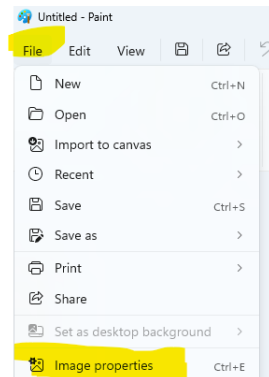
15.2.10 Click **Start** to program the board. When the configuration is complete, the Progress bar should show 100% (Successful).



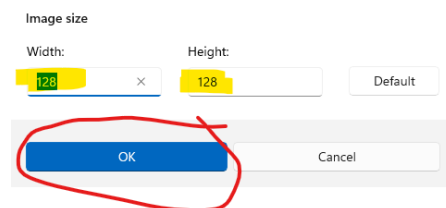
15.3 Paint a digit

15.3.1 Open Paint 

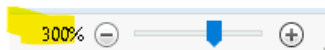
15.3.2 Define the size of the Image. File → Image properties



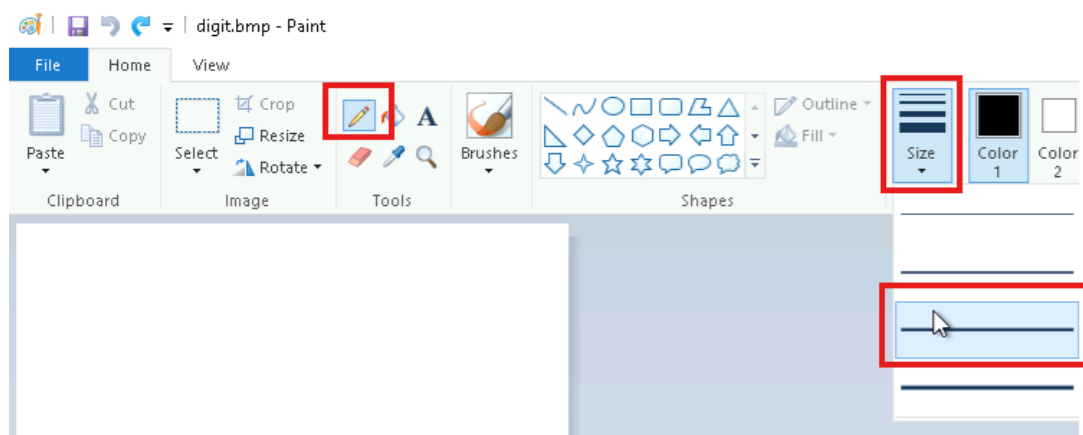
15.3.3 Set the size to 128 x 128 pixels



15.3.4 Set the zoom to 300%



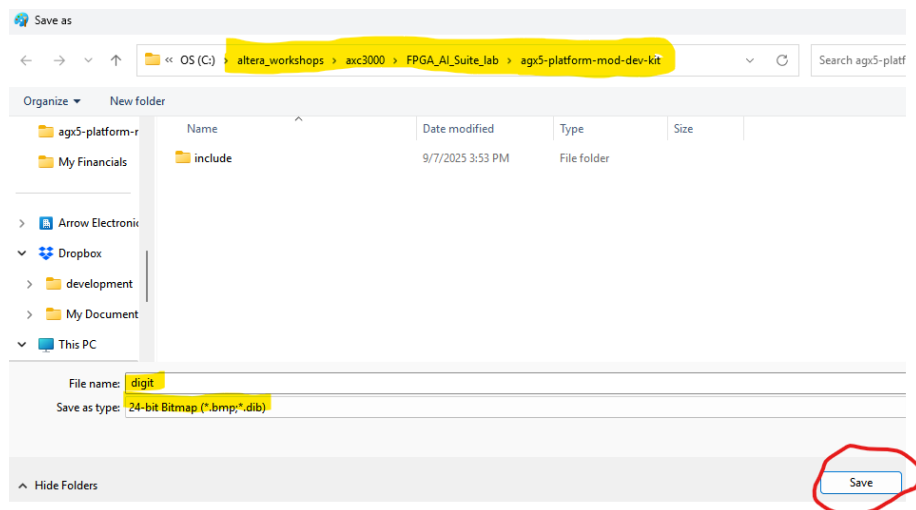
15.3.5 Select the Pencil and set it to a pixel width of 3.



15.3.6 Draw digit between 0 and 9.



15.3.7 Save the file to the path shown below. File Save. Specify **digit** as the filename.



15.4 Classify the Image

The image will be converted from a 128 x 128 pixel, 3 color channel, 24-bit BMP format to 28 x 28 pixel, Floating Point, 16-bit monochrome binary format. It will then be displayed for verification.

The image will be sent via JTAG to a memory block in the Agilex 3 device on the AXC3000 board. From there it will stream through the FPGA AI Suite custom IP

block, and the inference results will be written to another onchip memory block. All JTAG operations are performed by the System Console utility.

The results are read from the Agilex 3 device via JTAG and written to a file name output1.bin. This will be post processed and written to a file named result_hw.txt. The results will be displayed in the Command Prompt shell.

This process is implemented by a batch file. The contents are displayed below. Each line can be individually implemented in the Command shell

```
echo off
python convert_bmp_2_bin_fp16.py digit.bmp digit.bin
echo
echo **** Display image ****
echo
python display-fp16-file.py digit.bin
echo
echo **** Run inference on FPGA using system console ****
echo
system-console.exe --script=system_console_script.tcl --input digit.bin --num_inferences 1 --output_shape {10 1 1} --functional --arch ../AGX5_Streaming_Ddrfree_Softmax.arch
echo
echo **** Post process inference results and display them ****
echo
python streaming_post_processing.py output1.bin
echo
echo **** Display inference results ****
echo
type result_hw.txt
```

15.4.1 Open the Command shell

15.4.2 Navigate to

C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit

15.4.3 Type demo.bat. The inference and post processing will take up to 20 seconds. Most of the delay is spent opening the JTAG channel.

```
c:\>cd C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit
C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit>demo.bat
```


15.4.4 Review the results in the Command shell

```

"*** Display inference results ***"
ECHO is off.
0: 0.0
1: 0.96875
2: 0.0
3: 0.0
4: 0.0008544921875
5: 0.00146484375
6: 0.00079345703125
7: 0.00250244140625
8: 0.00830078125
9: 0.001953125

```

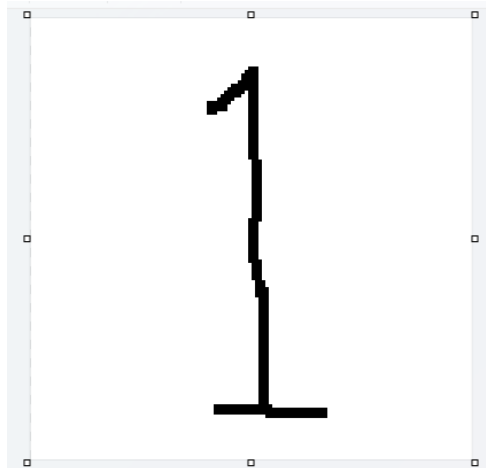
This classification effort indicated that there is a 96.875% chance that the digit was a one.

15.5 Classify a different digit

15.5.1 Erase the digit. Click on Undo. 

15.5.2 Hover above the digit and right click. This will erase the digit.

15.5.3 Select the Pencil tool and draw a new digit.



15.5.4 Save the digit. File → Save.

15.5.5 Run the demo.bat batch file again in the Command shell

15.5.6 This time the results are quite different. The classification is 32.8% for a one, 53.13% for a two, and 12.5% for a three. Why are the results so different this time? Was the model trained with data that represented European style digits?

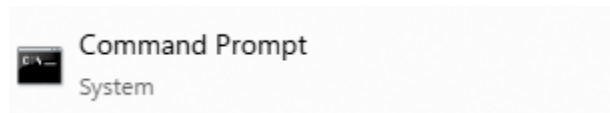
16 Verify the Design Locally (CloudLabs build)

If the design was built using CloudLabs but being verified locally then complete this section.

16.1 Clone the workshop repository

Download Python and tcl scripts necessary for the verification process.

16.1.1 Open a Command prompt



16.1.2 Change directory and clone the repository

```
$ cd c:\
```

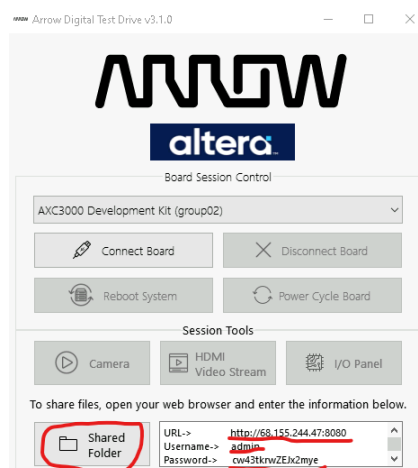
```
$ git clone https://github.com/ArrowElectronics/altera\_workshops.git
```

16.2 File Download

16.2.1 Click on the ArrowDTD icon in CloudLabs to launch the tool

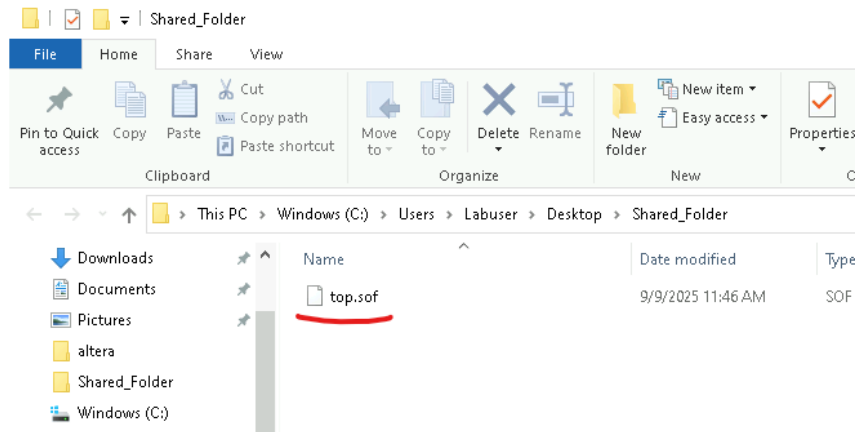


16.2.2 Open the **Shared Folder** utility in the CloudLab VM. Open a browser in the local PC. Navigate to the **URL** indicated. Use the provided username and password.



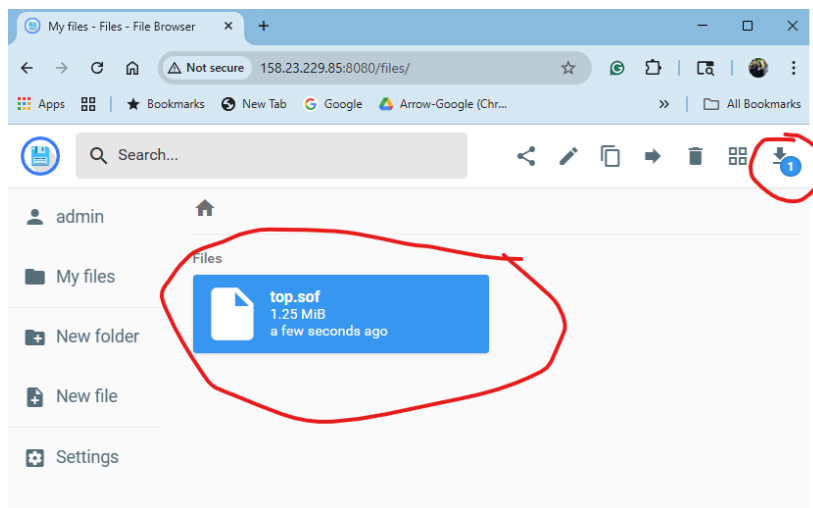
16.2.3 Drag and drop the following file into the **Shared** folder.

c:\altera_workshops\axc3000\FPGA_AI_Suite_lab\build-agx-lenet\hw\output_files\top.sof



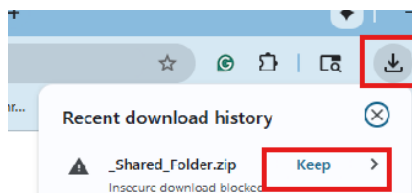
16.2.4 This will be reflected in the **browser** on the local PC.

16.2.5 **Download** the SOF file to C:\altera_workshops\axc3000\FPGA_AI_Suite_lab



The download might be blocked.

16.2.6 Press the download button. If the file was blocked click **Keep**.

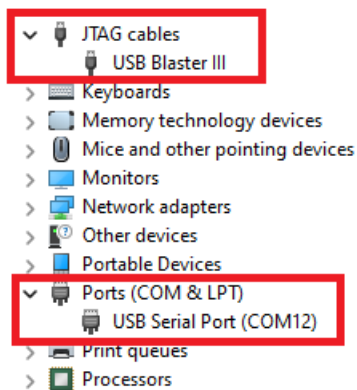


16.3 Program the FPGA

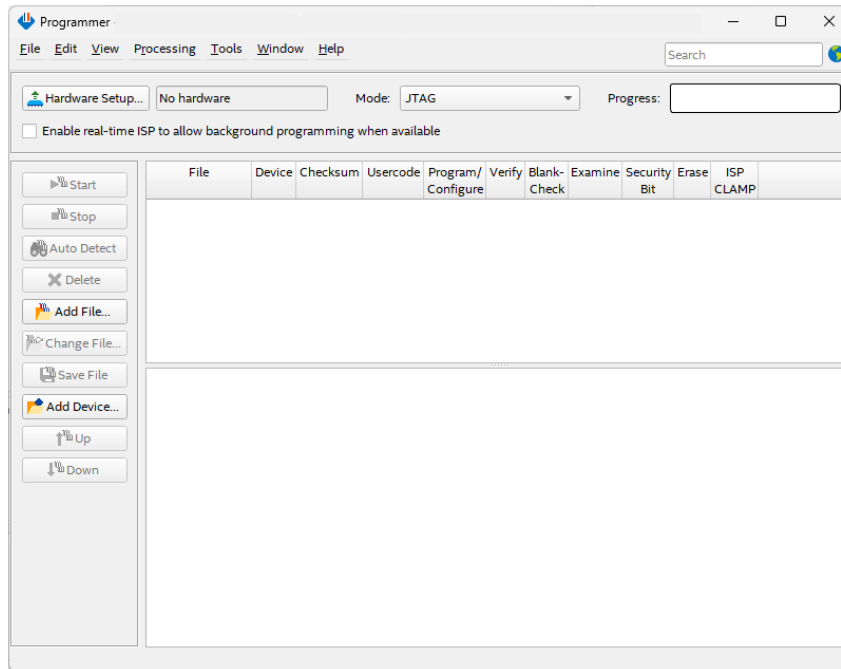
The next step is to program the design into the FPGA on the board. During the Assembler stage of the compilation, a programming file was generated.

16.3.1 Connect your AXC3000 Board to your PC using a USB C cable.

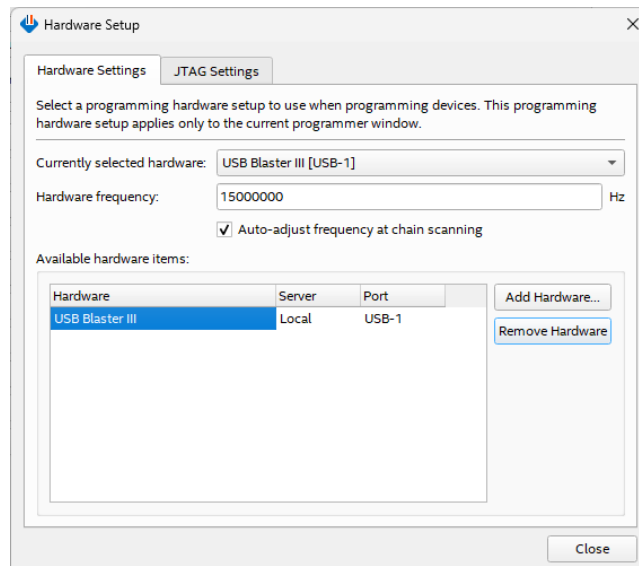
Since the USB Blaster III should be already installed, the Window's Device Manager should display the following entries, highlighted in red (COM port number may differ depending on your PC):



16.3.2 From the Start menu open Altera 25.1.1.125 Pro Edition --> Programmer.

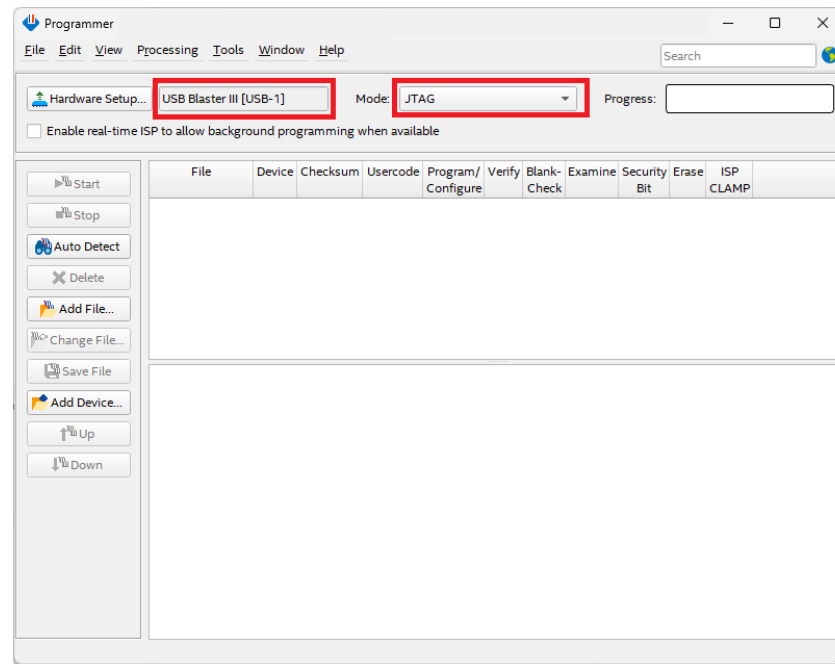


16.3.3 Click **Hardware Setup...** and double click **USB Blaster III** entry in the Hardware Setup window. **Currently selected hardware** should now show USB Blaster III [USB0] (the USB port number may be different depending on your PC).



16.3.4 Click **Close**.

16.3.5 Make sure the hardware setup is Altera-USB-Blaster-III [USB0] and the mode is JTAG. Click **Auto Detect**.



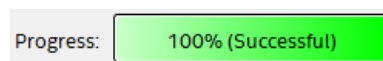
16.3.6 **Double click** the first <none> line item to choose the programming file

16.3.7 Navigate to the folder where the sof file was downloaded and select the .sof file

16.3.8 Click **Open**.

16.3.9 Make sure the Programmer shows the correct file and the correct part in the JTAG chain and check the **Program/Configure** checkbox.

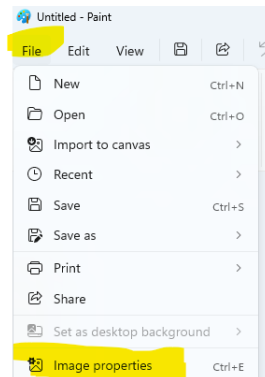
16.3.10 Click **Start** to program the board. When the configuration is complete, the Progress bar should show 100% (Successful).



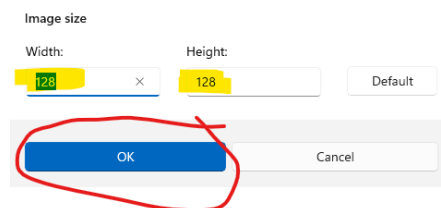
16.4 Paint a digit

16.4.1 Open Paint  Paint App

16.4.2 Define the size of the Image. File → Image properties



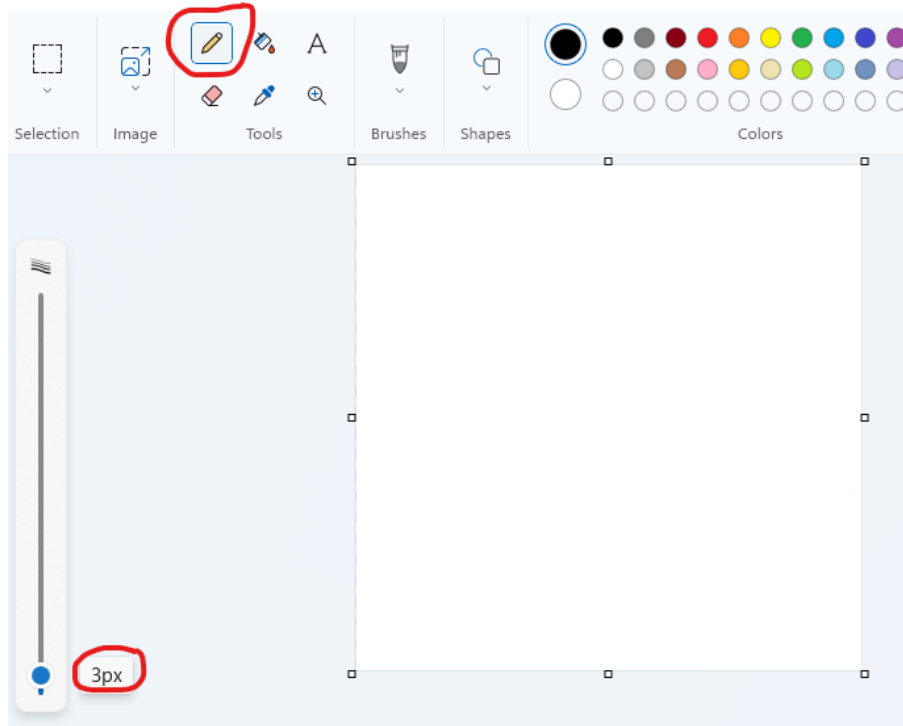
16.4.3 Set the size to 128 x 128 pixels



16.4.4 Set the zoom to 300%



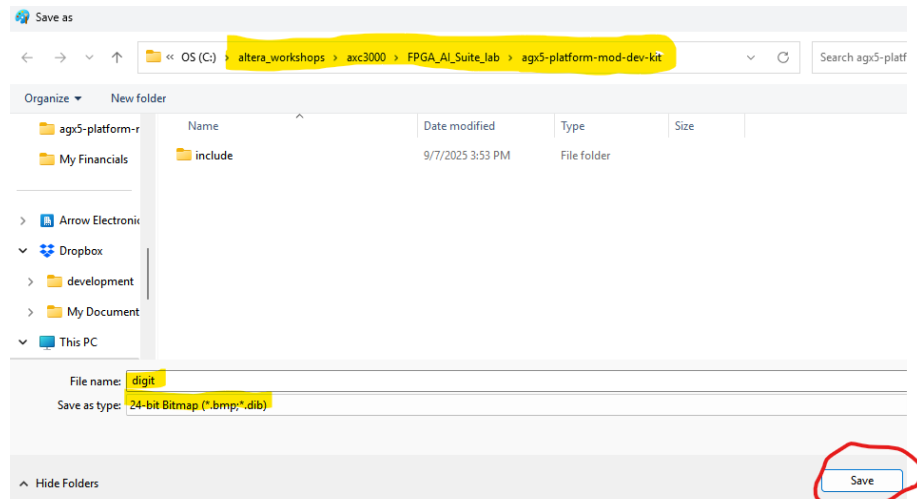
16.4.5 Select the Pencil and set it to a pixel width of 3.



16.4.6 Draw digit between 0 and 9.



16.4.7 Save the file. File Save. Specify **digit** as the filename.



16.5 Classify the Image

The image will be converted from a 128 x 128 pixel, 3 color channel, 24-bit BMP format to 28 x 28 pixel, Floating Point, 16-bit monochrome binary format. It will then be displayed for verification.

The image will be sent via JTAG to a memory block in the Agilex 3 device on the AXC3000 board. From there it will stream through the FPGA AI Suite custom IP block, and the inference results will be written to another onchip memory block. All JTAG operations are performed by the System Console utility.

The results are read from the Agilex 3 device via JTAG and written to a file name output1.bin. This will be post processed and written to a file named result_hw.txt. The results will be displayed in the Command Prompt shell.

This process is implemented by a batch file. The contents are displayed below. Each line can be individually implemented in the Command shell

```

echo off

python convert_bmp_2_bin_fpl6.py digit.bmp digit.bin

echo
echo **** Display image ****
echo
python display-fpl6-file.py digit.bin

echo
echo **** Run inference on FPGA using system console ****
echo
system-console.exe --script=system_console_script.tcl --input digit.bin --num_inferences 1 --output_shape [10 1 1] --functional --arch ../AGX5_Streaming_Ddrfree_Softmax.arch

echo
echo **** Post process inference results and display them ****
echo
python streaming_post_processing.py output1.bin

echo
echo **** Display inference results ****
echo
type result_hw.txt

```

16.5.1 Open the Command shell

16.5.2 Navigate to

C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit

16.5.3 Type demo.bat. The inference and post processing will take up to 20 seconds. Most of the delay is spent opening the JTAG channel.

```

c:\>cd C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit
C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit>demo.bat

```

16.5.4 Review the results in the Command shell

```

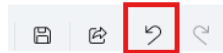
**** Display inference results ****
ECHO is off.
0: 0.0
1: 0.96875
2: 0.0
3: 0.0
4: 0.0008544921875
5: 0.00146484375
6: 0.00079345703125
7: 0.00250244140625
8: 0.00830078125
9: 0.001953125

```

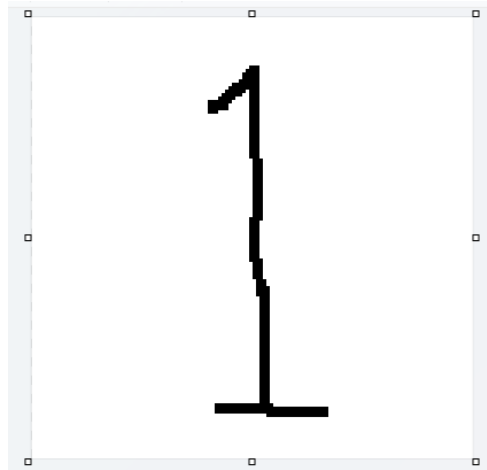
This classification effort indicated that there is a 96.875% chance that the digit was a one.

16.6 Classify a different digit

16.6.1 Erase the digit. Click on **Undo**.



16.6.2 Select the Pencil tool and draw a new digit.



16.6.3 Save the digit. File → Save.

16.6.4 Run the demo.bat batch file again in the Command shell

16.6.5 This time the results are quite different. The classification is 32.8% for a one, 53.13% for a two, and 12.5% for a three. Why are the results so different this time? Was the model trained with data that represented European style digits?

17 Verify the Design Locally (Local build)

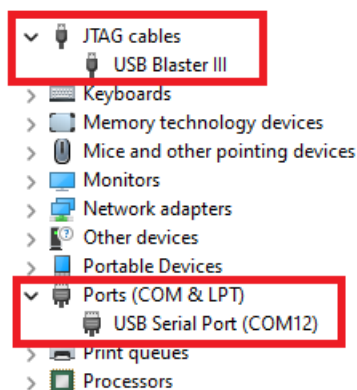
If the design was built locally then complete this section.

17.1 Program the FPGA

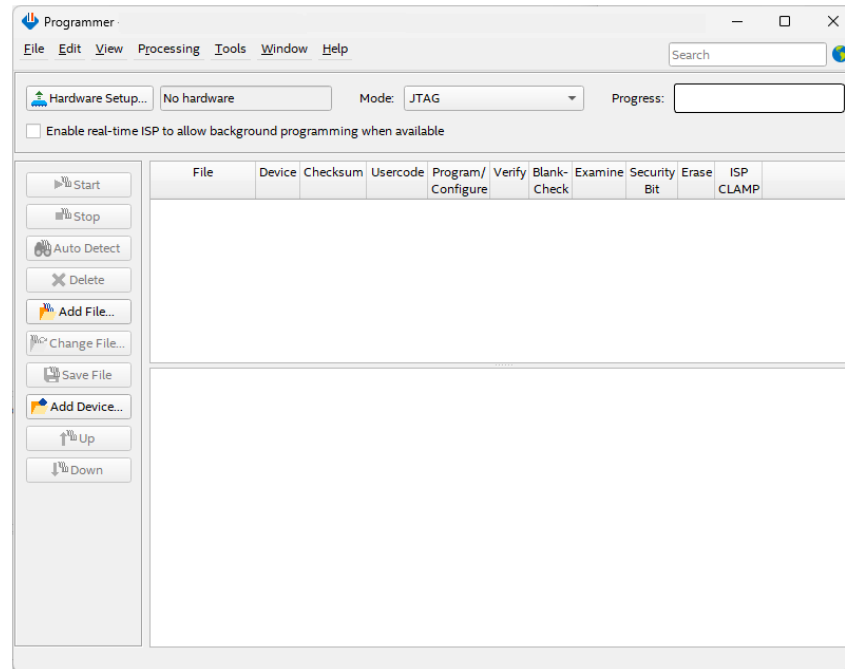
The next step is to program the design into the FPGA on the board. During the Assembler stage of the compilation, a programming file was generated.

17.1.1 Connect your AXC3000 Board to your PC using a USB C cable.

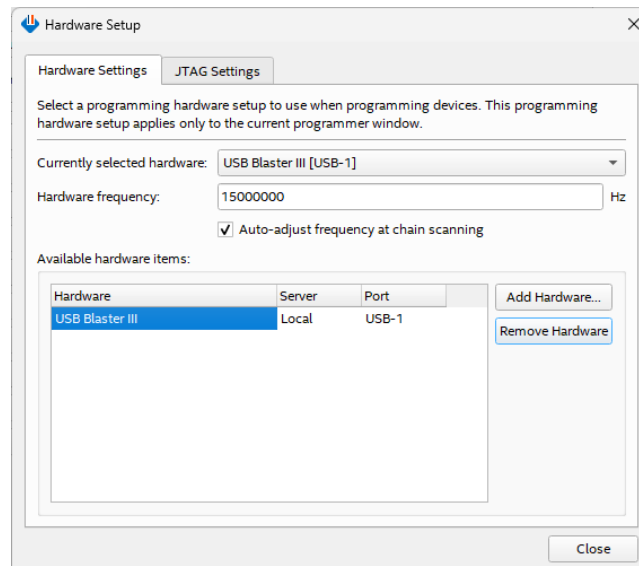
Since the USB Blaster III should be already installed, the Window's Device Manager should display the following entries, highlighted in red (COM port number may differ depending on your PC):



17.1.2 From the Start menu open Altera 25.1.1.125 Pro Edition --> Programmer.

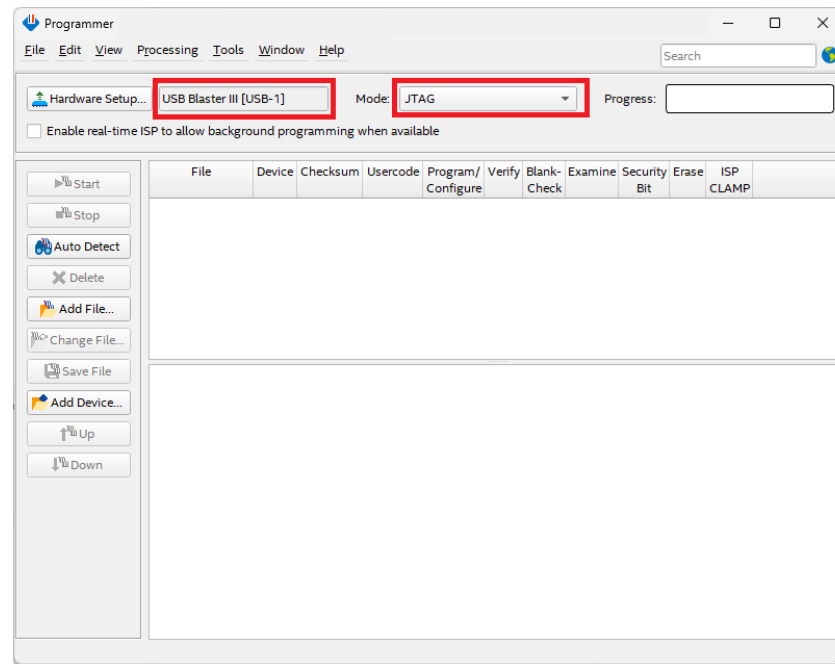


17.1.3 Click Hardware Setup... and double click USB Blaster III entry in the Hardware Setup window. Currently selected hardware should now show USB Blaster III [USB0] (the USB port number may be different depending on your PC).



17.1.4 Click Close.

17.1.5 Make sure the hardware setup is Altera-USB-Blaster-III [USB0] and the mode is JTAG. Click Auto Detect.



17.1.6 Double click the first <none> line item to choose the programming file

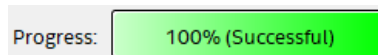
17.1.7 **Navigate** to the folder listed below and select the .sof file

C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\build-agx5-lenet\hw\output_files

17.1.8 Click **Open**

17.1.9 Make sure the Programmer shows the correct file and the correct part in the JTAG chain and check the Program/Configure checkbox.

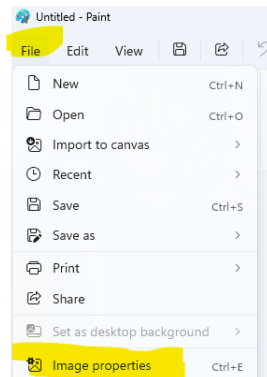
17.1.10 Click Start to program the board. When the configuration is complete, the Progress bar should show 100% (Successful).



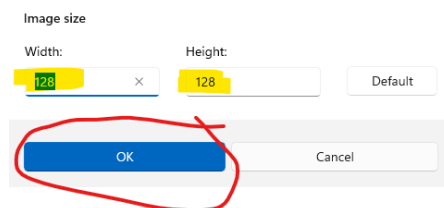
17.2 Paint a digit

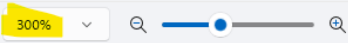
17.2.1 Open Paint  Paint App

17.2.2 Define the size of the Image. File → Image properties

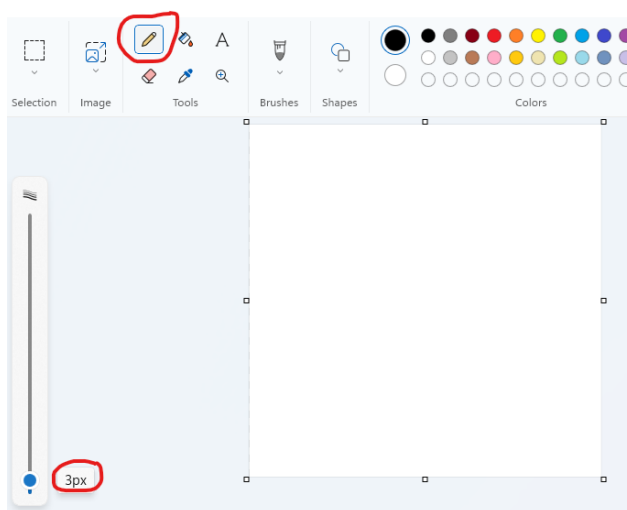


17.2.3 Set the size to 128 x 128 pixels



17.2.4 Set the zoom to 300% 

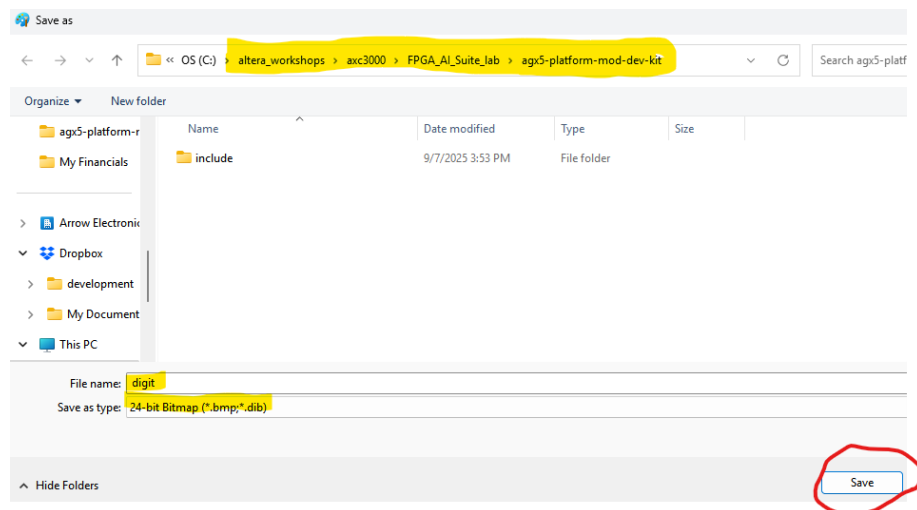
17.2.5 Select the Pencil and set it to a pixel width of 3.



17.2.6 Draw digit between 0 and 9.



17.2.7 Save the file. File Save. Specify digit as the filename.



17.3 Classify the Image

The image will be converted from a 128 x 128 pixel, 3 color channel, 24-bit BMP format to 28 x 28 pixel, Floating Point, 16-bit monochrome binary format. It will then be displayed for verification.

The image will be sent via JTAG to a memory block in the Agilex 3 device on the AXC3000 board. From there it will stream through the FPGA AI Suite custom IP block, and the inference results will be written to another onchip memory block. All JTAG operations are performed by the System Console utility.

The results are read from the Agilex 3 device via JTAG and written to a file name output1.bin. This will be post processed and written to a file named result_hw.txt. The results will be displayed in the Command Prompt shell.

This process is implemented by a batch file. The contents are displayed below. Each line can be individually implemented in the Command shell

```
echo off

python convert_bmp_2_bin_fp16.py digit.bmp digit.bin

echo
echo "**** Display image ****"
echo
python display_fp16-file.py digit.bin

echo
echo "**** Run inference on FPGA using system console ****"
echo
system-console.exe --script=system_console_script.tcl --input digit.bin --num_inferences 1 --output_shape [10 1 1] --functional --arch ../AGX5_Streaming_Ddrfree_Softmax.arch

echo
echo "**** Post process inference results and display them ****"
echo
python streaming_post_processing.py output1.bin

echo
echo "**** Display inference results ****"
echo
type result_hw.txt
```

17.3.1 Open the Command shell

17.3.2 Navigate to

C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit

17.3.3 Type demo.bat. The inference and post processing will take up to 20 seconds. Most of the delay is spent opening the JTAG channel.

```
c:\>cd C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit
C:\altera_workshops\axc3000\FPGA_AI_Suite_lab\agx5-platform-mod-dev-kit>demo.bat
```

17.3.4 Review the results in the Command shell

```
**** Display inference results ****
ECHO is off.
0: 0.0
1: 0.96875
2: 0.0
3: 0.0
4: 0.0008544921875
5: 0.00146484375
6: 0.00079345703125
7: 0.00250244140625
8: 0.00830078125
9: 0.001953125
```

This classification effort indicated that there is a 96.875% chance that the digit was a one.

17.4 Classify a different digit

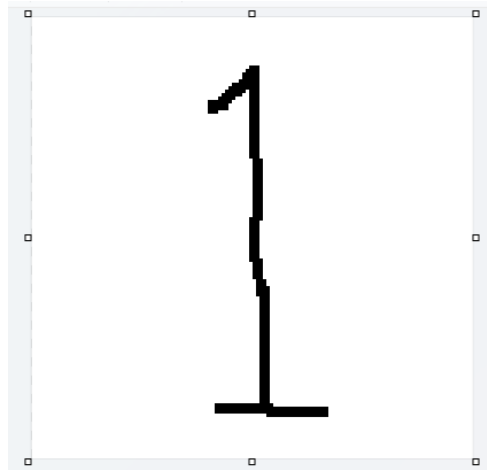




 e digit. Click on **Undo**.
 




17.4.2 Select the Pencil tool and draw a new digit.



17.4.3 Save the digit. File → Save.

17.4.4 Run the demo.bat batch file again in the Command shell

17.4.5 This time the results are quite different. The classification is 32.8% for a one, 53.13% for a two, and 12.5% for a three. Why are the results so different this time? Was the model trained with data that represented European style digits?

18 Congratulations on working through this workshop!

You worked through this workshop where you were exposed to considerable information on the AI flow including

- Design requirements
- Software (FPGA AI Suite) and network topology
- Obtaining a dataset
- Loading (and modifying) a dataset
- Changing a model
- Training a model
- Testing the model to evaluate what works and what does not with your own handwriting
- Visualization of the model with the output file, ONNX
- Simulation
- Inference of the Model by importing a model in FPGA
- Testing the FPGA (for those who have actual boards)

19 Legal Disclaimer

ARROW ELECTRONICS

EVALUATION BOARD LICENSE AGREEMENT

By using this evaluation board or kit (together with all related software, firmware, components, and documentation provided by Arrow, "Evaluation Board"), You ("You") are agreeing to be bound by the terms and conditions of this Evaluation Board License Agreement ("Agreement"). Do not use the Evaluation Board until You have read and agreed to this Agreement. Your use of the Evaluation Board constitutes Your acceptance of this Agreement.

PURPOSE

The purpose of this evaluation board is solely intended for evaluation purposes. Any use of the Board beyond these purposes is on your own risk. Furthermore, according to the applicable law, the offering Arrow entity explicitly does not warrant, guarantee or provide any remedies to you with regard to the board.

LICENSE

Arrow grants You a non-exclusive, limited right to use the enclosed Evaluation Board offering limited features only for Your evaluation and testing purposes in a research and development setting. Usage in a live environment is prohibited. The Evaluation Board shall not be, in any case, directly or indirectly assembled as a part in any production of Yours as it is solely developed to serve evaluation purposes and has no direct function and is not a finished product.

EVALUATION BOARD STATUS

The Evaluation Board offers limited features allowing You only to evaluate and test purposes. The Evaluation Board is not intended for consumer or household use. You are not authorized to use the Evaluation Board in any production system, and it may not be offered for sale or lease, or sold, leased or otherwise distributed for commercial purposes.

OWNERSHIP AND COPYRIGHT

Title to the Evaluation Board remains with Arrow and/or its licensors. This Agreement does not involve any transfer of intellectual property rights ("IPR") for evaluation board. You may not remove any copyright or other proprietary rights notices without prior written authorization from Arrow or its licensors.

RESTRICTIONS AND WARNINGS

Before You handle or use the Evaluation Board, You shall comply with all such warnings and other instructions and employ reasonable safety precautions in using the Evaluation Board. Failure to do so may result in death, personal injury, or property damage.

You shall not use the Evaluation Board in any safety critical or functional safety testing, including but not limited to testing of life supporting, military or nuclear applications. Arrow expressly disclaims any responsibility for such usage which shall be made at Your sole risk.

WARRANTY

Arrow warrants that it has the right to provide the evaluation board to you. This warranty is provided by Arrow in lieu of all other warranties, written or oral, statutory, express or implied, including any warranty as to merchantability, non-infringement, fitness for any particular purpose, or uninterrupted or error-free operation, all of which are expressly disclaimed. The evaluation board is provided “as is” without any other rights or warranties, directly or indirectly.

You warrant to Arrow that the evaluation board is used only by electronics experts who understand the dangers of handling and using such items, you assume all responsibility and liability for any improper or unsafe handling or use of the evaluation board by you, your employees, affiliates, contractors, and designees.

LIMITATION OF LIABILITIES

In no event shall Arrow be liable to you, whether in contract, tort (including negligence), strict liability, or any other legal theory, for any direct, indirect, special, consequential, incidental, punitive, or exemplary damages with respect to any matters relating to this agreement. In no event shall arrow’s liability arising out of this agreement in the aggregate exceed the amount paid by you under this agreement for the purchase of the evaluation board.

IDENTIFICATION

You shall, at Your expense, defend Arrow and its Affiliates and Licensors against a claim or action brought by a third party for infringement or misappropriation of any patent, copyright, trade secret or other intellectual property right of a third party to the extent resulting from (1) Your combination of the Evaluation Board with any other component, system, software, or firmware, (2) Your modification of the Evaluation Board, or (3) Your use of the Evaluation Board in a manner not permitted under this Agreement. You shall indemnify Arrow and its Affiliates and Licensors against and pay any resulting costs and damages finally awarded against Arrow and its Affiliates and Licensors or agreed to in any settlement, provided that You have sole control of the defense and settlement of the claim or action, and Arrow cooperates in the defense and furnishes all related evidence under its control at Your expense. Arrow will be entitled to participate in the defense of such claim or action and to employ counsel at its own expense.

RECYCLING

The Evaluation Board is not to be disposed as an urban waste. At the end of its life cycle, differentiated waste collection must be followed, as stated in the directive 2002/96/EC. In all the countries belonging to the European Union (EU Dir.

2002/96/EC) and those following differentiated recycling, the Evaluation Board is subject to differentiated recycling at the end of its life cycle, therefore: It is forbidden to dispose the Evaluation Board as an undifferentiated waste or with other domestic wastes. Consult the local authorities for more information on the proper disposal channels. An incorrect Evaluation Board disposal may cause damage to the environment and is punishable by the law.